Computational Mathematics II (MATH2731)

Dr Andrew Krause & Dr Denis Patterson, Durham University

2026-09-01

Table of contents

Introduction **3**
Content . . . 3
Weekly workflow and summative assessment . . . 4
Lab reports . . . 4
E-assessments . . . 5
Project . . . 5
Lectures, computing drop-ins & project workshops . . . 6
Contact details and Reading Materials . . . 7
Acknowledgements . . . 7

1 Numerical and Symbolic Computing **8**
1.1 Fixed-point numbers . . . 9
1.2 Floating-point numbers . . . 9
1.3 Significant figures . . . 12
1.4 Rounding error . . . 12
1.5 Loss of significance . . . 14
1.6 Example: Weather Modelling . . . 16
1.7 Exact and symbolic computing . . . 17
Knowledge checklist . . . 18

2 Continuous Functions **19**
2.1 Interpolation . . . 19
2.1.1 Polynomial Interpolation: Motivation . . . 19
2.1.2 Taylor series . . . 20
2.1.3 Polynomial Interpolation . . . 22
2.1.4 Lagrange Polynomials . . . 25
2.1.5 Interpolation Error . . . 28
2.1.6 Node Placement: Chebyshev nodes . . . 30
2.2 Nonlinear Equations . . . 33
2.2.1 Interval Bisection . . . 34
2.2.2 Fixed point iteration . . . 37
2.2.3 Orders of convergence . . . 40
2.2.4 Newton's method . . . 43
2.2.5 Newton's method for systems . . . 47
2.2.6 The secant method . . . 50

Knowledge checklist . . . 52

3 Linear Algebra **53**
3.1 Systems of Linear Equations . . . 53
3.2 Triangular systems . . . 54
3.3 Gaussian elimination . . . 56
3.4 LU decomposition . . . 58
3.5 Pivoting . . . 62
3.6 Vector norms . . . 64
3.7 Matrix norms . . . 66
3.8 Conditioning . . . 70
3.9 Iterative methods . . . 74
Knowledge checklist . . . 81

4 Calculus **82**
4.1 Differentiation . . . 82
4.1.1 Basics . . . 82
4.1.2 Higher-order finite differences . . . 84
4.1.3 Rounding error . . . 85
4.1.4 Differentiating noisy data . . . 86
4.1.5 Richardson extrapolation . . . 87
4.2 Numerical integration . . . 90
4.2.1 Newton-Cotes formulae . . . 91
4.2.2 Composite Newton-Cotes formulae . . . 93
4.2.3 Exactness . . . 95
4.2.4 Gaussian quadrature . . . 98
Knowledge checklist . . . 104

5 Differential Equations **106**
5.1 Basic Concepts . . . 106
5.1.1 Preliminary ODE theory . . . 108
5.2 Finite Difference Methods . . . 109
5.2.1 The van der Pol Oscillator . . . 109
5.3 Stability of Finite Difference Schemes . . . 112
5.3.1 The Dahlquist Problem . . . 113
5.3.2 Stability Regions and A-Stability . . . 114
5.3.3 Convergence . . . 116
5.4 Higher-Order Methods . . . 117
5.4.1 Single-step (Runge-Kutta) methods . . . 117
5.4.2 Linear multistep methods . . . 118
5.5 Beyond the Basics . . . 119

6 Further Topics **120**
6.1 Partial Differential Equations . . . 120
6.1.1 The heat equation and boundary conditions . . . 120
6.1.2 Spatial discretisation: finite differences . . . 121
6.1.3 The method of lines formulation . . . 121
6.1.4 Time discretisation and stability . . . 122
6.1.5 Summary . . . 123
6.2 Monte Carlo Methods . . . 123
6.2.1 Pseudorandom numbers . . . 124
6.2.2 Testing generators: the chi-squared test . . . 126
6.2.3 Sampling from other distributions . . . 128
6.2.4 The Monte Carlo estimator . . . 129
6.2.5 Why Monte Carlo? The curse of dimensionality . . . 131
6.2.6 Variance reduction: importance sampling . . . 133
6.2.7 Application: random walkers solve the heat equation . . . 135
Knowledge checklist . . . 138

Projects Showcase **139**
Outstanding Projects 2025/2026 (selection) . . . 140

Introduction

Welcome to Computational Mathematics II!

This course aims to help you build skills and knowledge in using modern computational methods to do and apply mathematics. It will involve a blend of hands-on computing work and mathematical theory—this theory will include aspects of numerical analysis, computational algebra, and other topics within scientific computing. These areas consist of studying the mathematical properties of the computational representations of mathematical objects (numerical values as well as symbolic manipulations). The computing skills developed in this module will be valuable in all subsequent courses in your degree at Durham and well beyond. We will also introduce you to the use (and abuse) of various computational tools invaluable for doing mathematics, such as AI and searchable websites. While we will encourage you throughout to use all the tools at your disposal, it is **imperative that you understand the details and scope of what you are doing!** You will also develop your communication, presentation, and group-work skills through the various assessments involved in the course – more on that below!

This module has **no final exam**. In fact, there are no exams of any kind. Instead, the summative assessment and associated final grade are entirely based on coursework undertaken during the term. This means that you should expect to spend more time on this course during the term relative to your other modules. We believe this workload distribution is a better way to train the skills we are trying to develop, and as a bonus, you will not need to worry about this course any further once the term ends!

Content

The module's content is divided into six chapters of roughly equal length; some will focus slightly more on theory, while others have a more practical and hands-on nature.

- **Chapter 1: Introduction to Computational Mathematics**
  - Programming basics (including GitHub, and numerical versus symbolic computation)
  - LaTeX, Overleaf, and presenting lab reports
  - Finite-precision arithmetic, rounding error, symbolic representations

- **Chapter 2: Continuous Functions**
  - Interpolation using polynomials – fitting curves to data (Lagrange polynomials, error estimates, convergence, and Chebyshev nodes)
  - Solving nonlinear equations (bisection, fixed-point iteration, Newton's method)
- **Chapter 3: Linear Algebra**
  - Solving linear systems numerically (LU decomposition, Gaussian elimination, conditioning) and symbolically
  - Applications: PageRank, computer graphics
- **Chapter 4: Calculus**
  - Numerical differentiation (finite differences)
  - Numerical integration (quadrature rules, Newton-Cotes formulae)
- **Chapter 5: Ordinary Differential Equations (ODEs)**
  - Numerically approximating solutions of ODEs
  - Timestepping: explicit and implicit methods
  - Stability and convergence order
- **Chapter 6: Selected Further Topics**
  - Intro. to partial differential equations (the method of lines)
  - Monte Carlo methods: pseudorandom numbers and testing them, sampling, error estimates, the curse of dimensionality, and variance reduction

---

Weekly workflow and summative assessment

The final grade for this module is determined as follows:

- **Weekly lab reports (weeks 1-6)** – 25%
- **Weekly e-assessments (weeks 1-6)** – 25%
- **Project (weeks 7-10)** – 50%

Lab reports

Each week for the first six weeks of the course, we will release a short set of exercises based on the lectures from the previous week. Students will be expected to submit a brief report (1-2 pages A4, including figures) with their solutions to the set of exercises – the report will consist of written answers and figures/plots. The reports will be evaluated for correctness and quality

of the presentation and communication (quality of figures, clarity of argumentation, etc.). **The lab report for a given week will be due at noon on Monday of the following week** (e.g., week one's lab report is due on Monday of week two and so on). Solutions and generalised feedback will be provided to the class on common mistakes and issues arising in each report. Students can also seek detailed feedback on their submission from the lecturers during drop-in sessions and office hours. The first lab report (due on Monday of Week 2) will be formative. There will be five summative lab reports in total, and **your mark is based on your four highest-scoring submissions.**

E-assessments

Each week for the first six weeks of the course, we will release a set of e-assessments based on the lectures from the previous week. These exercises are designed to complement the lab reports by focusing exclusively on coding skills. The e-assessments will involve submitting code auto-marked by an online grading tool, and hence give immediate feedback. As with the lab reports, **the set of e-assessments for a given week will be due at noon on Monday of the following week**. The first set of e-assessments is formative and there will be five summative sets of e-assessments in total; **your mark is based on your four highest-scoring submissions.**

Project

The single largest component of the assessment for this module is the project. **Weeks 7-10 of this course focus exclusively on project work with lectures ending in Week 6.** We will be releasing more detailed instructions on the project submission format and assessment criteria separately, but briefly, the main aspects of the project are as follows:

- There will be approximately eight different project options to choose from across different areas of mathematics (e.g., pure, applied, probability, mathematical physics, etc.); each project has a distinct member of the Maths Department as supervisor.
- Students will submit their preferred project options (ranked choice preferences) in Week 4 of the term and be allocated to projects by the end of Week 6 (there are maximum subscription numbers for each option to ensure equity of supervision).
- Each project consists of two parts: a **guided component** that is completed as part of a small group and an **extension component** that is open-ended and completed as an individual. Group allocations will be done by the lecturers.
- Each group will jointly submit a five-page report for the guided component of the project, and this is worth 50% of the project grade.
- Each student will also submit a three-page report and a six-minute video presentation on their extension component. This submission is worth 50% of the project grade.

In Weeks 7-10 of the term, lectures will be replaced by project workshop sessions during which students can discuss their project with the designated supervisor. This will be an opportunity to discuss progress, ask questions, and seek clarification. Each student only needs to attend the one project drop-in weekly session relevant to their project. Computing drop-in sessions will continue as scheduled in the first six weeks to provide additional support for coding pertinent tasks for the projects – there will be two timetabled computing drop-ins per week and students are encouraged to attend at least one of them.

Lectures, computing drop-ins & project workshops

Lectures will primarily present, explain, and discuss new material (especially theory), but will also feature computer demonstrations of the algorithms and numerical methods. As such, students are encouraged to bring their laptops to lectures to run the examples themselves. Students must bring a laptop or device capable of running code to the computer drop-ins to work on the e-assessments and lab reports.

Table 1: Chapter 1 is covered in a single lecture; Chapters 2–6 each run over three consecutive lectures, so each chapter starts partway through one week and finishes at the start of the next.

	Lecture 1	Lecture 2	Lecture 3	Other activities
Week 1	Chapter 1	Chapter 2	Chapter 2	2 computing drop-ins
Week 2	Chapter 2	Chapter 3	Chapter 3	2 computing drop-ins
Week 3	Chapter 3	Chapter 4	Chapter 4	2 computing drop-ins
Week 4	Chapter 4	Chapter 5	Chapter 5	2 computing drop-ins
Week 5	Chapter 5	Chapter 6	Chapter 6	2 computing drop-ins
Week 6	Chapter 6	Project preparation	Project preparation	2 computing drop-ins
Week 7	–	–	–	1 project workshop, 2 computing drop-ins
Week 8	–	–	–	1 project workshop, 2 computing drop-ins
Week 9	–	–	–	1 project workshop, 2 computing drop-ins
Week 10	–	–	–	1 project workshop, 2 computing drop-ins

Contact details and Reading Materials

If you have questions or need clarification on any of the above, please speak to us during lectures and drop-in sessions. Alternatively, email one or both of us at denis.d.patterson@durham.ac.uk or andrew.krause@durham.ac.uk.

The lecture notes are designed to be sufficient and self-contained. Hence, students do not need to purchase a textbook to complete the course successfully. References for additional reading will also be given at the end of each chapter.

The following texts may be useful supplementary references for students wishing to read further into topics from the course:

- Burden, R. L., & Faires, J. D. (1997). *Numerical Analysis* (6th ed.). Pacific Grove, CA: Brooks/Cole Publishing Company.
- Süli, E., & Mayers, D. F. (2003). *An Introduction to Numerical Analysis*. Cambridge: Cambridge University Press.

Acknowledgements

We are indebted to Prof. Anthony Yeates (Durham) whose numerical analysis notes formed the basis of several chapters of the coures notes.

1 Numerical and Symbolic Computing

The goal of this chapter is to explore and begin to answer the following question:

> *How do we represent numbers and symbolic expressions on a computer?*

In MATLAB, the following comparison returns `0` (false):

```
0.1 + 0.2 == 0.3
```

The numbers 0.1, 0.2 and 0.3 cannot be stored exactly in binary, and the tiny rounding errors mean that the two sides differ by about 5.6×10^{-17}. By the end of this chapter, you will understand why.

Integer arithmetic on a computer is exact, provided the results stay within some maximum size.

If 1 bit (binary digit) is used to store the sign $\pm$, then the remaining 63 bits can store the magnitude, and the largest possible number is

$$1 \times 2^{62} + 1 \times 2^{61} + \ldots + 1 \times 2^{1} + 1 \times 2^{0} = 2^{63} - 1.$$

> **Note**
>
> Integers in Python have arbitrary precision, so they can grow as large as memory allows. In most other languages, integers have a fixed size and exceeding the range causes an **overflow**: in Java, for example, the result wraps around to a negative number, while MATLAB's fixed-size integer types (such as `int64`) saturate at the largest representable value.

In contrast to the integers, only a subset of the real numbers within any given interval can be represented exactly.

Symbolic expressions representing a range of mathematical objects and operations can also be manipulated exactly using Computer Algebra Systems (CAS), although such operations are almost always much slower than numerical computations using integer or real-valued numbers. This trade-off between numerical and symbolic computation will be a key theme throughout the course.

1.1 Fixed-point numbers

In everyday life, we tend to use a **fixed-point** representation of real numbers:

$$x = \pm(d_1 d_2 \cdots d_{k-1}.d_k \cdots d_n)_\beta, \quad \text{where} \quad d_1, \ldots, d_n \in \{0, 1, \ldots, \beta - 1\}.$$

Here β is the base (e.g. 10 for decimal arithmetic or 2 for binary). The word *fixed* refers to the position of the point: a fixed-point system uses a fixed number of digits before and after the point, so the numbers it can represent are equally spaced, a distance $\beta^{-(n-k+1)}$ apart.

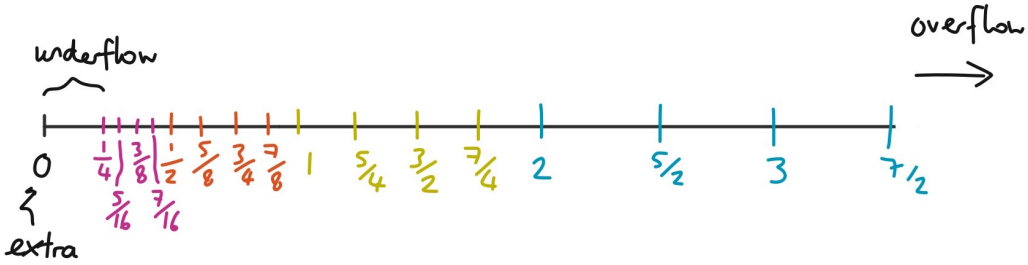
More generally, if we allow any number of digits and forbid leading zeros (other than a single 0 before the point for numbers less than 1), then every real number has a unique representation of this form. The only exception is that a number whose expansion terminates can also be written with an infinite trailing sequence of the digit $\beta - 1$.

1.2 Floating-point numbers

Computers use a **floating-point** representation. Fix a base $\beta \geq 2$, a number of digits m, and an exponent range $e_{\min} \leq e \leq e_{\max}$. Only numbers in the **floating-point number system** $F \subset \mathbb{R}$ can be represented exactly, where

$$F = \{ \pm (0.d_1 d_2 \cdots d_m)_\beta \beta^e \mid d_i \in \{0, 1, \ldots, \beta - 1\},\ e \in \mathbb{Z},\ e_{\min} \leq e \leq e_{\max}\}.$$

Here $(0.d_1 d_2 \cdots d_m)_\beta$ is called the **fraction** (or **significand** or **mantissa**), and e is the **exponent**. This can represent a much larger range of numbers than a fixed-point system of the same size, although at the cost that the numbers are not equally spaced. If we require $d_1 \neq 0$, then each non-zero number in F has a unique representation and F is called **normalised**. From now on, we assume that F is normalised, with the number 0 included separately.



> **Note**
>
> Notice that the spacing between numbers jumps by a factor β at each power of β. The largest possible number is $(0.111)_2 2^2 = (\frac{1}{2} + \frac{1}{4} + \frac{1}{8})(4) = \frac{7}{2}$. The smallest non-zero number

is $(0.100)_2 2^{-1} = \frac{1}{2}(\frac{1}{2}) = \frac{1}{4}$.

Here $\beta = 2$, and there are 52 bits for the fraction, 11 for the exponent, and 1 for the sign. The stored exponent field is $E = (e_1 e_2 \cdots e_{11})_2 \in [0, 2047]$, and a normalised number has the value

$$\pm(1.d_1 \cdots d_{52})_2 2^{E-1023} = \pm(0.1 d_1 \cdots d_{52})_2 2^{E-1022}.$$

When $\beta = 2$, the first digit of a normalised number is always 1, so it doesn't need to be stored in memory. The standard's **exponent bias** is 1023; in the $(0.1\cdots)_2$ form used in these notes it becomes 1022, so the exponents $e = E - 1022$ lie in the range -1022 to 1025. The two extreme values of the exponent field are reserved: $E = 0$ stores ± 0 and the subnormal numbers (see below), and $E = 2047$ stores $\pm\infty$ and `NaN` ("not a number"). Normalised numbers therefore have exponents from -1021 to 1024.

The smallest positive normalised number in this system is $(0.1)_2 2^{-1021} \approx 2.225 \times 10^{-308}$, and the largest number is $(0.1\cdots1)_2 2^{1024} \approx 1.798 \times 10^{308}$. In MATLAB, these are returned by `realmin` and `realmax`.

> **Note**
>
> IEEE stands for Institute of Electrical and Electronics Engineers. The IEEE 754 standard was first published in 1985 (and revised in 2008 and 2019), and is used for floating-point arithmetic by MATLAB, Python and almost all modern hardware. The automatic 1 is sometimes called the "hidden bit". The exponent bias avoids the need to store the sign of the exponent.

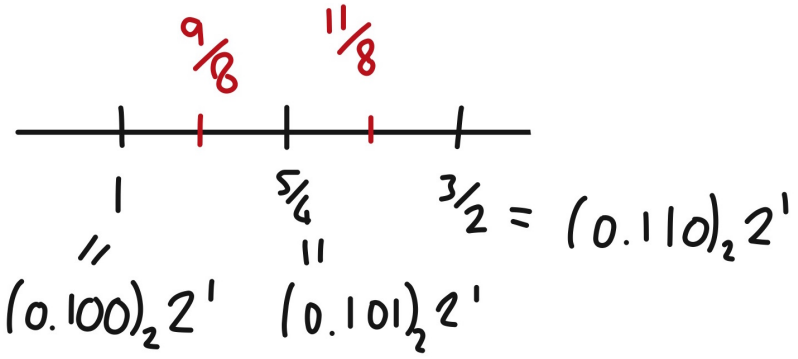
Numbers outside the finite set F cannot be represented exactly. If a calculation produces a non-zero result that is smaller in absolute value than the smallest normalised number, it is called **underflow**; IEEE arithmetic first represents such results using subnormal numbers (see below), and results smaller still are rounded to 0. If a result is larger in absolute value than the largest number in F, it is called **overflow**; in IEEE arithmetic (and hence in MATLAB), the result is $\pm\infty$, so that `2*realmax` returns `Inf`.

> **Note**
>
> In IEEE arithmetic, when the exponent field is $E = 0$, a zero fraction stores ± 0 (the sign bit gives the sign). The non-zero fractions with $E = 0$ store **subnormal** numbers, which have hidden bit 0 rather than 1. These fill the gap between 0 and the smallest normalised number (in our toy system above, the gap between 0 and $\frac{1}{4}$), so that underflow happens gradually.

> **Note**
>
> **Ariane 5 rocket failure (1996):** The maiden flight ended in failure. Only 40 seconds after initiation, at altitude 3700m, the launcher veered off course and exploded. The cause was a software exception during data conversion from a 64-bit float to a 16-bit integer. The converted number was too large to be represented (an overflow), causing an exception.

The mapping from $\mathbb{R}$ to F is called **rounding** and denoted $\mathrm{fl}(x)$. Usually it is simply the nearest number in F to x. If x lies exactly midway between two numbers in F, a method of breaking ties is required. The IEEE standard specifies *round to nearest even*—i.e., take the neighbour with last digit 0 in the fraction. Ties are then rounded up and down equally often, which avoids statistical bias or prolonged drift in one direction.

> **Note**
>
> **Vancouver stock exchange index:** In 1982, the index was established at 1000. By November 1983, it had fallen to 520, even though the exchange seemed to be doing well. Explanation: the index was rounded *down* to 3 digits at every recomputation. Since the errors were always in the same direction, they added up to a large error over time. Upon recalculation, the index doubled!



$\frac{9}{8} = (1.001)_2$ lies exactly midway between its neighbours $1 = (0.100)_2 2^1$ and $\frac{5}{4} = (0.101)_2 2^1$, so the tie-break rule applies: 1 has last digit 0, so $\frac{9}{8}$ is rounded down to 1. $\frac{11}{8} = (1.011)_2$ lies exactly midway between its neighbours $\frac{5}{4} = (0.101)_2 2^1$ and $\frac{3}{2} = (0.110)_2 2^1$; this time $\frac{3}{2}$ has last digit 0, so $\frac{11}{8}$ is rounded up to $\frac{3}{2}$.

1.3 Significant figures

When doing calculations without a computer, we often use the terminology of **significant figures**. To count the number of significant figures in a number x, start with the first non-zero digit from the left, and count all the digits thereafter, including final zeros if they are after the decimal point.

To round x to n s.f., replace x by the nearest number with n s.f. An approximation $\hat{x}$ of x is "correct to n s.f." if both $\hat{x}$ and x round to the same number to n s.f.

1.4 Rounding error

There are two common ways of measuring the error of an approximation in numerical analysis that we will use throughout the course:

Definition 1.1: Absolute and relative errors

If x is the true value of a number and $\hat{x}$ is an approximation, we define the **absolute error** of this approximation as the quantity

$$|\hat{x} - x|,$$

while the **relative error** of the approximation is given by

$$\frac{|\hat{x} - x|}{|x|},$$

assuming $x \neq 0$.

Note

Relative errors are often more useful because they are scale invariant. E.g., an error of 1 hour is irrelevant in estimating the age of a lecture theatre, but catastrophic in timing your arrival at the lecture.

If $|x|$ lies between the smallest non-zero number in F and the largest number in F, then we can write

$$\mathrm{fl}(x) = x(1+\delta), \quad \text{where} \quad \delta = \frac{\mathrm{fl}(x) - x}{x},$$

so that $|\delta|$ is the relative error incurred by rounding.

To bound $|\delta|$, write $x = (0.d_1 d_2 \cdots)_\beta \beta^e$ with $d_1 \neq 0$ and $e \in [e_{\min}, e_{\max}]$. If $x \notin F$, this significand will not terminate after m digits, but rounding changes it by at most $\frac{1}{2}\beta^{-m}$, so

$$|\mathrm{fl}(x) - x| \leq \tfrac{1}{2}\beta^{-m}\beta^e.$$

Since $d_1 \neq 0$, the significand of x is at least $(0.1)_\beta = \beta^{-1}$, so $|x| \geq \beta^{-1}\beta^e$. Therefore

$$|\delta| = \frac{|\mathrm{fl}(x) - x|}{|x|} \leq \frac{\frac{1}{2}\beta^{-m}\beta^e}{\beta^{-1}\beta^e} = \tfrac{1}{2}\beta^{1-m}.$$

The number $\epsilon_{\mathrm{M}} = \frac{1}{2}\beta^{1-m}$ is called the **unit roundoff**, and is independent of x. So the relative rounding error satisfies

$$|\delta| \leq \epsilon_{\mathrm{M}}.$$

> **Note**
>
> Terminology varies between sources. The distance between 1 and the next larger number in F is $\beta^{1-m} = 2\epsilon_{\mathrm{M}}$, and many texts (and MATLAB) call this quantity the **machine epsilon**. Typing `eps` in MATLAB returns $2^{-52} \approx 2.2204 \times 10^{-16}$, so the bound on the relative rounding error in double precision is `eps/2`.

For our system with $\beta = 2$, $m = 3$, we have $\epsilon_{\mathrm{M}} = \frac{1}{2} \cdot 2^{1-3} = \frac{1}{8}$. For IEEE double precision, we have $\beta = 2$ and $m = 53$ (including the hidden bit), so $\epsilon_{\mathrm{M}} = \frac{1}{2} \cdot 2^{1-53} = 2^{-53} \approx 1.11 \times 10^{-16}$.

When adding/subtracting/multiplying/dividing two numbers in F, the result will not be in F in general, so must be rounded.

Let us multiply $x = \frac{5}{8}$ and $y = \frac{7}{8}$. We have

$$xy = \tfrac{35}{64} = \tfrac{1}{2} + \tfrac{1}{32} + \tfrac{1}{64} = (0.100011)_2.$$

This has too many significant digits to represent in our system, so the best we can do is round the result to $\mathrm{fl}(xy) = (0.100)_2 = \frac{1}{2}$.

> **Note**
>
> In practice, the hardware effectively computes the exact result (using a few extra digits) and then rounds once, as we did in this example.

For $\circ = +, -, \times, \div$, IEEE standard arithmetic requires **correctly rounded** operations: the result must be the exact result rounded to F. Provided no overflow or underflow occurs, this means that

$$\mathrm{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq \epsilon_{\mathrm{M}}.$$

1.5 Loss of significance

You might think that the above guarantees the accuracy of calculations to within ϵ_{M}, but this is true only if x and y are themselves exact. In reality, we are probably starting from $\bar{x} = x(1+\delta_1)$ and $\bar{y} = y(1+\delta_2)$, with $|\delta_1|, |\delta_2| \leq \epsilon_{\mathrm{M}}$. In that case, there is an error even before we round the result, since

$$\begin{aligned} \bar{x} \pm \bar{y} &= x(1+\delta_1) \pm y(1+\delta_2) \\ &= (x \pm y)\left(1 + \frac{x\delta_1 \pm y\delta_2}{x \pm y}\right). \end{aligned}$$

If the correct answer $x \pm y$ is very small relative to $|x|$ and $|y|$ (for example, when subtracting two nearly equal numbers), then there can be an arbitrarily large relative error in the result, compared to the errors in the initial $\bar{x}$ and $\bar{y}$. In particular, this relative error can be much larger than ϵ_{M}. This is called **loss of significance** (or **catastrophic cancellation**), and is a major cause of errors in floating-point calculations.

To 4 s.f., the roots are

$$x_1 = 28 + \sqrt{783} = 55.98, \quad x_2 = 28 - \sqrt{783} = 0.01786.$$

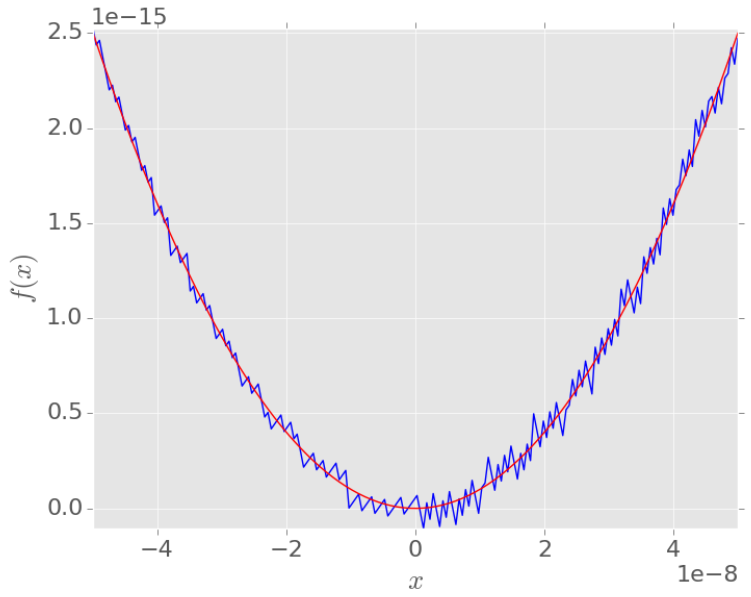
However, working to 4 s.f. we would compute $\sqrt{783} = 27.98$, which would lead to the results

$$\bar{x}_1 = 55.98, \quad \bar{x}_2 = 0.02000.$$

The smaller root is not correct to 4 s.f., because of catastrophic cancellation. One way around this is to note that $x^2 - 56x + 1 = (x - x_1)(x - x_2)$, so the product of the roots equals the constant term: $x_1 x_2 = 1$. Computing $x_2 = 1/x_1 = 1/55.98 = 0.01786$ then gives the correct answer.

> **Note**
>
> Note that the error crept in when we rounded $\sqrt{783}$ to 27.98, because this removed digits that would otherwise have been significant after the subtraction.

Let us plot this function in the range $-5 \times 10^{-8} \leq x \leq 5 \times 10^{-8}$ – even in IEEE double precision arithmetic we find significant errors, as shown by the blue curve:



The problem is that, for such small x, we have $e^x \approx 1 + x$ and $\cos(x) \approx 1$, so we are subtracting numbers of size about 1 to obtain an answer of size about $x^2 \leq 2.5 \times 10^{-15}$. The rounding errors in computing e^x and $\cos(x)$ are of size about $\epsilon_{\mathrm{M}} \approx 10^{-16}$, which is comparable to the answer itself.

The red curve shows the correct result approximated using the Taylor series

$$\begin{aligned} f(x) &= \left(1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots\right) - \left(1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \ldots\right) - x \\ &\approx x^2 + \frac{x^3}{6}. \end{aligned}$$

This avoids subtraction of nearly equal numbers. The error made by discarding the higher-order terms of the series is called the **truncation error**; here, the first neglected term is $x^5/120$, which is negligible for such small x.

> **Note**
>
> We will look in more detail at polynomial approximations, including Taylor series, in Chapter 2.

Note that floating-point arithmetic violates many of the usual rules of real arithmetic, such as $(a + b) + c = a + (b + c)$.

$$\begin{aligned} \mathrm{fl}\big[(5.9 + 5.5) + 0.4\big] &= \mathrm{fl}\big[\mathrm{fl}(11.4) + 0.4\big] = \mathrm{fl}(11 + 0.4) = 11, \\ \mathrm{fl}\big[5.9 + (5.5 + 0.4)\big] &= \mathrm{fl}\big[5.9 + 5.9\big] = \mathrm{fl}(11.8) = 12. \end{aligned}$$

In $\mathbb{R}$, the average of two numbers always lies between the numbers. But if we work to 3 decimal digits,

$$\mathrm{fl}\left(\frac{5.01+5.02}{2}\right) = \frac{\mathrm{fl}(10.03)}{2} = \frac{10.0}{2} = 5.00,$$

which lies outside the interval $[5.01, 5.02]$!

The moral of the story is that sometimes care is needed to ensure that we carry out a calculation accurately and as intended!

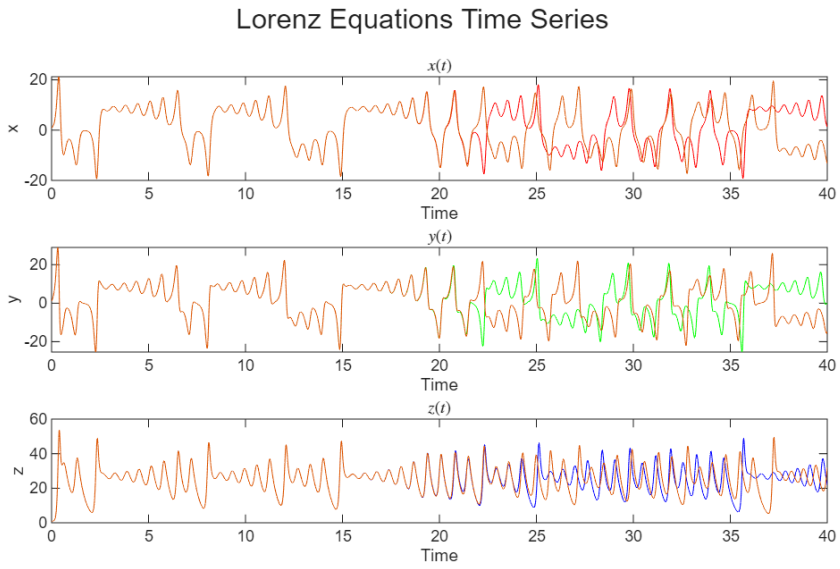
1.6 Example: Weather Modelling

A very simplistic description of an atmospheric fluid is given by the Lorenz system of differential equations:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y-x),\\ \frac{dy}{dt} &= x(\rho-z)-y,\\ \frac{dz}{dt} &= xy-\beta z,\end{aligned}$$

where σ, ρ, and β are positive constants. (Here β is a model parameter, unrelated to the floating-point base β used earlier in this chapter.)

We will describe how to solve such equations numerically in Chapter 5. For now, let's look at a simulation of these equations computed by just iterating a bunch of addition and multiplication steps:



Here we plot all three variables over a particular window of time (loosely corresponding to something between minutes to hours, depending on several details of the physics we are neglecting). For each variable, we have run two simulations with two different values of β. One simulation has $\beta = 8/3 = 2.66666666666\ldots$ and the other has $\beta = 8/3 + 10^{-10} = 2.66666666676\ldots$. Nevertheless, you can clearly see that this tiny difference in parameters (well below what is experimentally measurable) has a huge impact on the solution. The reason for this, and a key reason why weather is fundamentally hard to predict, is the existence of chaos in many models of physical phenomena. However, truncation errors and floating-point errors also contribute to the discrepancies in this simulation. Indeed, 8/3 cannot be stored exactly in binary floating point, and rounding errors of size about ϵ_{M} are introduced at every step; in a chaotic system these grow just like the difference in parameters, so even two runs with identical parameters will eventually diverge if they carry out the same operations in a different order.

The overall lesson of this chapter so far is that small errors can exist due to analytically-understood reasons such as the *truncation error* in a Taylor series approximation, or the *floating-point error* in the numerical methods used. Such errors can not only impact individual calculations, but they can accumulate and entirely change outcomes of simulations, as we will see throughout the course. Nevertheless, computational methods still underlie an enormous range of scientific fields, so understanding these sources of error (and how they can become amplified) is a central theme of this course.

1.7 Exact and symbolic computing

Symbolic computations require different data types from numerical ones; in MATLAB we can use the Symbolic Math Toolbox. In the following chapters, we will compare symbolic and numerical approaches to solving mathematical problems. One key difference is that symbolic computations are exact, but much more expensive to scale up to solve larger problems; for example, we will tackle numerical problems involving matrices of size $10^4 \times 10^4$ or larger, which would not be possible to successfully manipulate symbolically on a modern computer.

Compare the following commands:

```
1/3                          % double precision: displays 0.3333 (rounded)
sym(1)/3                     % symbolic: the exact fraction 1/3
vpa(sym(1)/3, 30)            % 1/3 evaluated to 30 significant digits
isAlways(sym(1)/10 + sym(2)/10 == sym(3)/10)   % returns 1 (true)
```

Unlike `0.1 + 0.2 == 0.3` from the start of the chapter, the symbolic comparison is exact and returns true.

Symbolic computations are used to check or do laborious analytical work, but also to rigorously prove mathematical theorems which would be too onerous to carry out by hand. For instance,

while the Lorenz equations above were first noted to have strange dependencies on initial conditions, it was only in 2002 that this was mathematically proved to be a chaotic system. Among other tools, the proof relied on interval arithmetic, which computes intervals that are guaranteed to contain the true result of a calculation, by rounding the lower endpoint of each interval down and the upper endpoint up.

Knowledge checklist

Key topics:

1. Integer, fixed-point and floating-point representations of numbers on computers.
2. Absolute and relative errors, rounding, and the unit roundoff (machine epsilon).
3. Overflow, underflow and loss of significance.
4. Symbolic and numerical representations.

Key skills:

- Understanding and distinguishing integer, fixed-point, and floating-point representations.
- Analysing the effects of rounding and machine epsilon in calculations.
- Diagnosing and managing rounding errors, overflow, and underflow.

2 Continuous Functions

The goal of this chapter is to explore and begin to answer the following question:

How do we represent and manipulate continuous functions on a computer?

2.1 Interpolation

2.1.1 Polynomial Interpolation: Motivation

The main idea of this section is to find a polynomial that approximates a general function f. But why polynomials? Polynomials have many nice mathematical properties but from the perspective of function approximation, the key one is the following: Any continuous function on a compact interval can be approximated to arbitrary accuracy using a polynomial (provided you are willing to go to a high enough degree).

Theorem 2.1: Weierstrass Approximation Theorem (1885)

For any $f \in C([0,1])$ and any $\epsilon > 0$, there exists a polynomial $p(x)$ such that

$$\max_{0 \le x \le 1} |f(x) - p(x)| \le \epsilon.$$

Note

This may be proved using an explicit sequence of polynomials, called Bernstein polynomials. By rescaling, the same result holds on any closed interval $[a, b]$.

If f is a polynomial of degree n,

$$f(x) = p_n(x) = a_0 + a_1 x + \ldots + a_n x^n,$$

then we only need to store the $n+1$ coefficients $a_0, \ldots, a_n$. Operations such as taking the derivative or integrating f are also convenient. On the other hand, polynomials are poor at approximating functions with singularities, such as $1/x$ near $x = 0$, since a polynomial is bounded on any bounded interval.

> **Note**
>
> To approximate functions like $1/x$, there is a well-developed theory of rational function interpolation, which is beyond the scope of this course.

In this chapter, we look for a suitable polynomial p_n by **interpolation**—that is, requiring $p_n(x_i) = f(x_i)$ at a finite set of points x_i, usually called **nodes**. Sometimes we will also require the derivative(s) of p_n to match those of f. This type of function approximation where we want to match values of the function that we know at particular points is very natural in many applications. For example, weather forecasts involve numerically solving huge systems of partial differential equations (PDEs), which means actually solving them on a discrete grid of points. If we want weather predictions between grid points, we must **interpolate**. Figure 2.1 shows the spatial resolutions of a range of current and past weather models produced by the UK Met Office.

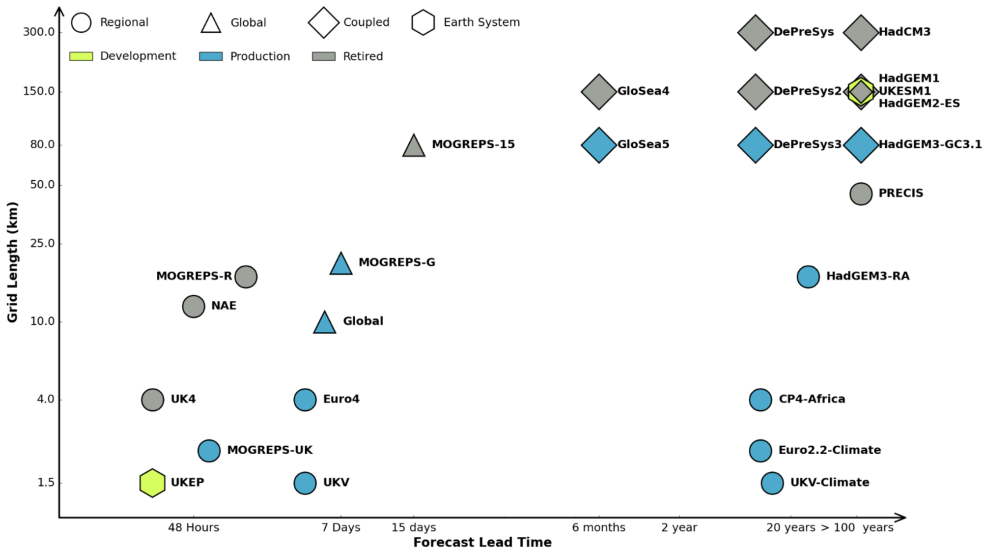


Figure 2.1: Chart showing a range of weather models produced by the UK Met Office. Even the highest spatial resolution models have more than 1.5km between grid points due to computational constraints.

2.1.2 Taylor series

A truncated Taylor series is (in some sense) the simplest interpolating polynomial since it uses only a single node x_0, although it does require p_n to match both f and some of its derivatives.

We can approximate this using a Taylor series about the point $x_0 = 0$, which is

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots.$$

This comes from writing

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + \dots,$$

then differentiating term-by-term and matching values at x_0:

$$\begin{aligned}
f(x_0) &= a_0, \\
f'(x_0) &= a_1, \\
f''(x_0) &= 2a_2, \\
f'''(x_0) &= 3(2)a_3, \\
&\vdots \\
\implies f(x) &= f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f'''(x_0)}{3!}(x - x_0)^3 + \dots.
\end{aligned}$$

So

$$\begin{aligned}
\text{1 term} &\implies f(0.1) \approx 0.1, \\
\text{2 terms} &\implies f(0.1) \approx 0.1 - \frac{0.1^3}{6} = 0.099833\dots, \\
\text{3 terms} &\implies f(0.1) \approx 0.1 - \frac{0.1^3}{6} + \frac{0.1^5}{120} = 0.09983341\dots.
\end{aligned}$$

The next term will be $-0.1^7/7! \approx -2 \times 10^{-11}$, which won't change the answer to 6 s.f.

> **Note**
>
> The exact answer is $\sin(0.1) = 0.09983341\dots$.

Mathematically, we can write the remainder as follows.

Theorem 2.2: Taylor's Theorem

Let f be $n+1$ times differentiable on (a, b), and let $f^{(n)}$ be continuous on $[a, b]$. If $x, x_0 \in [a, b]$ then there exists ξ between x_0 and x such that

$$f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1}.$$

The sum is called the **Taylor polynomial** of degree n, and the last term is called the **Lagrange form** of the remainder. Note that the unknown number ξ depends on x.

For $f(x) = \sin(x)$, we found the Taylor polynomial $x - x^3/3! + x^5/5!$. Since the x^6 term in the Taylor series of $\sin(x)$ is zero, this is also the Taylor polynomial of degree 6, $p_6(x)$. So we may use the remainder with $n = 6$, which involves $f^{(7)}(x) = -\cos(x)$:

$$|f(x) - p_6(x)| = \left| \frac{f^{(7)}(\xi)}{7!}(x - x_0)^7 \right|$$

for some ξ between x_0 and x. For $x = 0.1$, we have

$$|f(0.1) - p_6(0.1)| = \frac{1}{5040}(0.1)^7 |f^{(7)}(\xi)| \quad \text{for some } \xi \in [0, 0.1].$$

Since $|f^{(7)}(\xi)| = |\cos(\xi)| \leq 1$, we can say, before calculating, that the error satisfies

$$|f(0.1) - p_6(0.1)| \leq 1.984 \times 10^{-11}.$$

> **Note**
>
> The actual error is 1.984×10^{-11} (to 4 s.f.), so this is a tight estimate: $\cos(\xi) \approx 1$ for $\xi \in [0, 0.1]$.

Since this error arises from approximating f with a truncated series, rather than due to rounding, it is known as **truncation error**. Note that it tends to be lower if you use more terms (larger n), or if the function oscillates less (smaller $f^{(n+1)}$ on the interval (x_0, x)).

Error estimates like the Lagrange remainder play an important role in numerical analysis and computation. Note that Taylor's theorem does not tell us the value of ξ, so in practice we must take a worst-case estimate to get an error bound, i.e., use the maximum value of $|f^{(n+1)}|$ over the interval between x_0 and x.

> **Note**
>
> The proof of Taylor's theorem (and of the interpolation error theorem below) uses repeated application of Rolle's theorem: if f is continuous on $[a, b]$ and differentiable on (a, b), with $f(a) = f(b) = 0$, then there exists $\xi \in (a, b)$ with $f'(\xi) = 0$.

2.1.3 Polynomial Interpolation

The classical problem of **polynomial interpolation** is to find a polynomial

$$p_n(x) = a_0 + a_1 x + \ldots + a_n x^n = \sum_{k=0}^{n} a_k x^k$$

that interpolates our function f at $n+1$ distinct nodes $x_0, x_1, \ldots, x_n$. In other words, $p_n(x_i) = f(x_i)$ at each of the nodes x_i. Since the polynomial has $n+1$ unknown coefficients, we need $n+1$ conditions, one for each node.

Here we have two nodes x_0, x_1, and seek a polynomial $p_1(x) = a_0 + a_1 x$. Then the interpolation conditions require that

$$\begin{cases} p_1(x_0) = a_0 + a_1 x_0 = f(x_0) \\ p_1(x_1) = a_0 + a_1 x_1 = f(x_1) \end{cases} \quad \Longrightarrow \quad p_1(x) = \frac{x_1 f(x_0) - x_0 f(x_1)}{x_1 - x_0} + \frac{f(x_1) - f(x_0)}{x_1 - x_0} x.$$

For general n, the interpolation conditions require

$$\begin{array}{ccccccc} a_0 & +a_1 x_0 & +a_2 x_0^2 & +\ldots & +a_n x_0^n & = f(x_0), \\ a_0 & +a_1 x_1 & +a_2 x_1^2 & +\ldots & +a_n x_1^n & = f(x_1), \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ a_0 & +a_1 x_n & +a_2 x_n^2 & +\ldots & +a_n x_n^n & = f(x_n), \end{array}$$

so we have to solve

$$\underbrace{\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix}}_{V} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

The matrix V is called a **Vandermonde matrix**. Its determinant is $\det(V) = \prod_{0 \le i < j \le n} (x_j - x_i)$, which is non-zero provided the nodes are all distinct, so the system has a unique solution. This establishes an important result, where $\mathcal{P}_n$ denotes the space of all real polynomials of degree $\le n$.

Theorem 2.3: Existence/uniqueness

Given $n+1$ distinct nodes $x_0, x_1, \ldots, x_n$, there is a unique polynomial $p_n \in \mathcal{P}_n$ that interpolates $f(x)$ at these nodes.

We may also prove uniqueness by the following elegant argument.

Proof (Uniqueness part of Existence/Uniqueness Theorem): Suppose that in addition to p_n there is another interpolating polynomial $q_n \in \mathcal{P}_n$. Then the difference $r_n := p_n - q_n$ is also a polynomial with degree $\le n$. But we have

$$r_n(x_i) = p_n(x_i) - q_n(x_i) = f(x_i) - f(x_i) = 0 \quad \text{for } i = 0, \ldots, n,$$

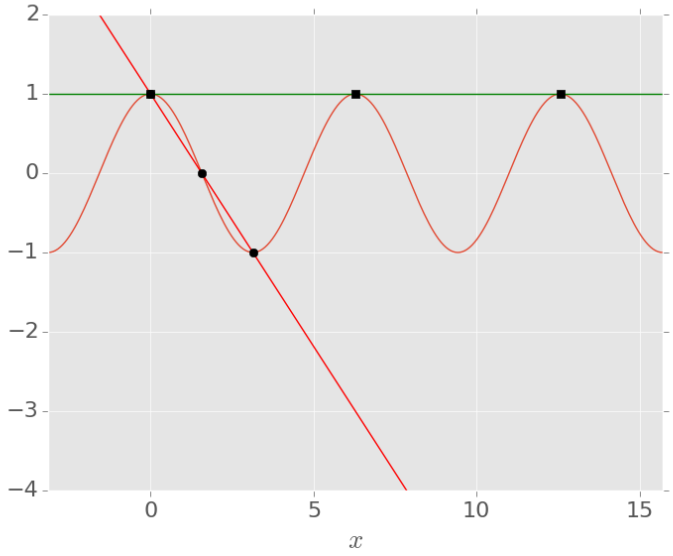
so $r_n(x)$ has $n+1$ roots. A non-zero polynomial of degree $\le n$ has at most n roots, so we must have $r_n(x) \equiv 0$, which implies that $q_n = p_n$.

Note

Note that the unique polynomial through $n+1$ points may have degree $< n$. This happens when $a_n = 0$ in the solution to the Vandermonde system above.

Example 2.1: Interpolate $f(x) = \cos(x)$ with $p_2 \in \mathcal{P}_2$ at the nodes $\{0, \frac{\pi}{2}, \pi\}$.

We have $x_0 = 0$, $x_1 = \frac{\pi}{2}$, $x_2 = \pi$, so $f(x_0) = 1$, $f(x_1) = 0$, $f(x_2) = -1$. Clearly the unique interpolant is a straight line $p_2(x) = 1 - \frac{2}{\pi}x$.
If we took the nodes $\{0, 2\pi, 4\pi\}$, we would get a constant function $p_2(x) = 1$.



One way to compute the interpolating polynomial would be to solve the Vandermonde system above, e.g. by Gaussian elimination. However, this is not recommended: as n grows, the Vandermonde matrix becomes very badly conditioned, meaning that small rounding errors in the data can produce large errors in the computed coefficients (we will make this precise in Chapter 3). In practice, we choose a different basis for p_n.

Note

The Vandermonde matrix arises when we write p_n in the **natural basis** $\{1, x, x^2, \ldots\}$, but we could also choose to work in some other basis...

Example 2.2: MATLAB demonstration: the Vandermonde approach

MATLAB's `vander` builds the Vandermonde matrix (with the columns in the order $x^n, \ldots, x, 1$, which is the order that `polyval` expects for the coefficients):

```
% Quadratic interpolant of cos(x) at three nodes
x = linspace(-pi/4, pi/4, 3)';
a = vander(x) \ cos(x);           % coefficients, highest power first
xx = linspace(-1, 1, 200);
figure
plot(xx, cos(xx), 'LineWidth', 3), hold on
plot(xx, polyval(a, xx), '--', 'LineWidth', 3)
plot(x, cos(x), 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k'), hold off
legend('f(x) = cos(x)', 'interpolant p_2(x)', 'nodes', 'Location', 'south')
xlabel('x'), ylabel('y'), set(gca, 'FontSize', 18), grid on

% The condition number grows rapidly with n
for n = [5 10 20 30]
    x = linspace(-1, 1, n+1)';
    fprintf('n = %2d, cond(V) = %.2e\n', n, cond(vander(x)));
end
```

2.1.4 Lagrange Polynomials

This uses a special basis of polynomials $\{\ell_k\}$ in which the interpolation equations reduce to the identity matrix. In other words, the coefficients in this basis are just the function values,

$$p_n(x) = \sum_{k=0}^{n} f(x_k)\ell_k(x).$$

Example 2.3: Linear interpolation again.

We can re-write our linear interpolant to separate out the function values:

$$p_1(x) = \underbrace{\frac{x - x_1}{x_0 - x_1}}_{\ell_0(x)} f(x_0) + \underbrace{\frac{x - x_0}{x_1 - x_0}}_{\ell_1(x)} f(x_1).$$

Then ℓ_0 and ℓ_1 form the necessary basis. In particular, they have the property that

$$\ell_0(x_i) = \begin{cases} 1 & \text{if } i = 0, \\ 0 & \text{if } i = 1, \end{cases} \qquad \ell_1(x_i) = \begin{cases} 0 & \text{if } i = 0, \\ 1 & \text{if } i = 1, \end{cases}$$

For general n, the $n+1$ **Lagrange polynomials** are defined as a product

$$\ell_k(x) = \prod_{\substack{j=0 \\ j\neq k}}^{n} \frac{x - x_j}{x_k - x_j}.$$

By construction, they have the property that

$$\ell_k(x_i) = \begin{cases} 1 & \text{if } i = k, \\ 0 & \text{otherwise.} \end{cases}$$

From this, it follows that the interpolating polynomial may be written as above.

> **Note**
>
> By the Existence/Uniqueness Theorem, the Lagrange polynomials are the *unique* polynomials in $\mathcal{P}_n$ with this property.

Example 2.4: Compute the quadratic interpolating polynomial to $f(x) = \cos(x)$ with nodes $\{-\frac{\pi}{4}, 0, \frac{\pi}{4}\}$ using Lagrange polynomials.

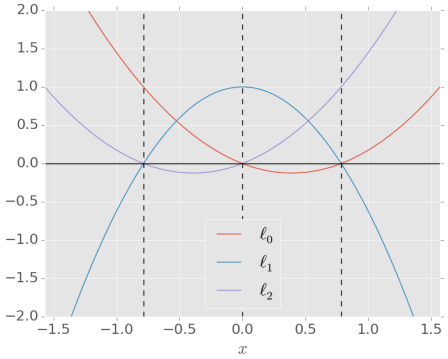
The Lagrange polynomials of degree 2 for these nodes are

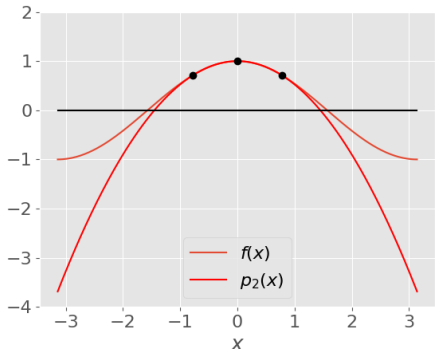
$$\begin{aligned}
\ell_0(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{x(x - \frac{\pi}{4})}{\frac{\pi}{4} \cdot \frac{\pi}{2}}, \\
\ell_1(x) &= \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x + \frac{\pi}{4})(x - \frac{\pi}{4})}{-\frac{\pi}{4} \cdot \frac{\pi}{4}}, \\
\ell_2(x) &= \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} \\
&= \frac{x(x + \frac{\pi}{4})}{\frac{\pi}{2} \cdot \frac{\pi}{4}}.
\end{aligned}$$

So the interpolating polynomial is

$$\begin{aligned}
p_2(x) &= f(x_0)\ell_0(x) + f(x_1)\ell_1(x) + f(x_2)\ell_2(x) \\
&= \tfrac{1}{\sqrt{2}}\tfrac{8}{\pi^2}x(x - \tfrac{\pi}{4}) - \tfrac{16}{\pi^2}(x + \tfrac{\pi}{4})(x - \tfrac{\pi}{4}) + \tfrac{1}{\sqrt{2}}\tfrac{8}{\pi^2}x(x + \tfrac{\pi}{4}) = \tfrac{16}{\pi^2}(\tfrac{1}{\sqrt{2}} - 1)x^2 + 1.
\end{aligned}$$

The Lagrange polynomials and the resulting interpolant are shown below:





Note

Lagrange polynomials were actually discovered by Edward Waring in 1776 and rediscovered by Euler in 1783, before they were published by Lagrange himself in 1795; a classic example of Stigler's law of eponymy!

Example 2.5: MATLAB demonstration: plotting the Lagrange basis

```
x = [-pi/4, 0, pi/4];              % nodes
xx = linspace(-1, 1, 200);
figure, hold on
for k = 1:length(x)
    others = x([1:k-1, k+1:end]);
    ell_k = prod((xx' - others) ./ (x(k) - others), 2);   % l_k evaluated on the grid
    plot(xx, ell_k, 'LineWidth', 3, 'DisplayName', sprintf('$\\ell_%d(x)$', k-1))
end
plot(x, zeros(size(x)), 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k', 'DisplayName', 'no
hold off
legend('Interpreter', 'latex', 'Location', 'southoutside', 'Orientation', 'horizontal')
xlabel('x'), set(gca, 'FontSize', 18), grid on
```

The Lagrange form of the interpolating polynomial is easy to write down, but changing any of the nodes means that the ℓ_k must all be recomputed from scratch, and similarly for adding a new node (moving to higher degree). An alternative basis, due to Newton, builds up p_n one node at a time so that adding a node is cheap; we will not need it in this course.

2.1.5 Interpolation Error

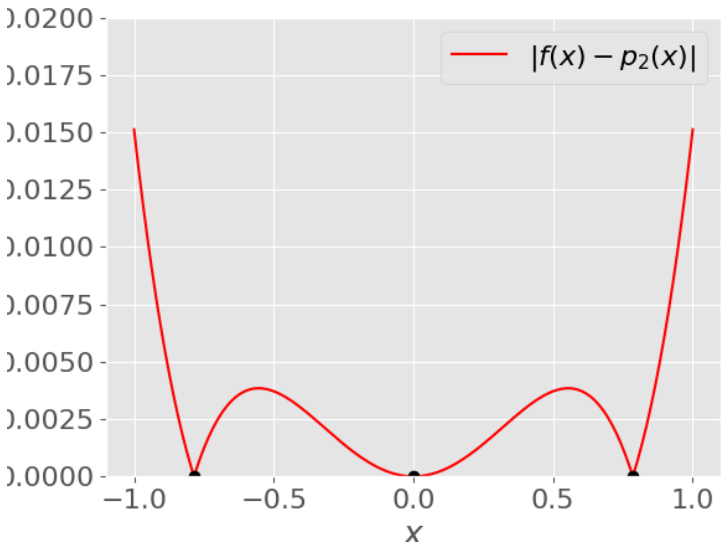
The goal here is to estimate the error $|f(x) - p_n(x)|$ when we approximate a function f by a polynomial interpolant p_n. Clearly this will depend on x.

Example 2.6: Quadratic interpolant for $f(x) = \cos(x)$ with $\{-\frac{\pi}{4}, 0, \frac{\pi}{4}\}$.

From Section 2.1.4, we have $p_2(x) = \frac{16}{\pi^2}\left(\frac{1}{\sqrt{2}} - 1\right)x^2 + 1$, so the error is

$$|f(x) - p_2(x)| = \left|\cos(x) - \frac{16}{\pi^2}\left(\frac{1}{\sqrt{2}} - 1\right)x^2 - 1\right|.$$

This is shown below:



Clearly the error vanishes at the nodes themselves, but note that it generally does better near the middle of the set of nodes — this is quite typical behaviour.

There is an error formula for interpolation that is analogous to Taylor's theorem (and its proof again uses Rolle's theorem).

Theorem 2.4: Cauchy's Interpolation Error Theorem

Let $p_n \in \mathcal{P}_n$ be the unique polynomial interpolating $f(x)$ at the $n+1$ distinct nodes $x_0, x_1, \ldots, x_n \in [a, b]$, and let f be continuous on $[a, b]$ with $n+1$ continuous derivatives

on (a, b). Then for each $x \in [a, b]$ there exists $\xi \in (a, b)$ such that

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)(x - x_1) \cdots (x - x_n).$$

This looks similar to the error formula for Taylor polynomials (see Taylor's Theorem). But now the error vanishes at multiple nodes rather than just at x_0.

From the formula, you can see that the error will be larger for a more "wiggly" function, where the derivative $f^{(n+1)}$ is larger. It might also appear that the error will go down as the number of nodes n increases; we will see in Section 2.1.6 that this is not always true.

Note

As in Taylor's theorem, note the appearance of an undetermined point ξ. This will prevent us knowing the error exactly, but we can make an estimate as before.

Example 2.7: Quadratic interpolant for $f(x) = \cos(x)$ with $\{-\frac{\pi}{4}, 0, \frac{\pi}{4}\}$.

For $n = 2$, Cauchy's Interpolation Error Theorem says that

$$f(x) - p_2(x) = \frac{f^{(3)}(\xi)}{6} x(x + \tfrac{\pi}{4})(x - \tfrac{\pi}{4}) = \tfrac{1}{6} \sin(\xi) x(x + \tfrac{\pi}{4})(x - \tfrac{\pi}{4}),$$

for some ξ in the smallest interval containing both x and the nodes. For $x \in [-1, 1]$, this means that $\xi \in [-1, 1]$.
For an upper bound on the error at a particular x, we can just use $|\sin(\xi)| \leq 1$ and plug in x.
To bound the maximum error within the interval $[-1, 1]$, let us maximise the polynomial $w(x) = x(x + \frac{\pi}{4})(x - \frac{\pi}{4})$. We have $w'(x) = 3x^2 - \frac{\pi^2}{16}$ so turning points are at $x = \pm\frac{\pi}{4\sqrt{3}}$. We have

$$w(-\tfrac{\pi}{4\sqrt{3}}) = 0.186\ldots, \quad w(\tfrac{\pi}{4\sqrt{3}}) = -0.186\ldots, \quad w(-1) = -0.383\ldots, \quad w(1) = 0.383\ldots.$$

So our error estimate for $x \in [-1, 1]$ is

$$|f(x) - p_2(x)| \leq \tfrac{1}{6}(0.383) = 0.0638\ldots$$

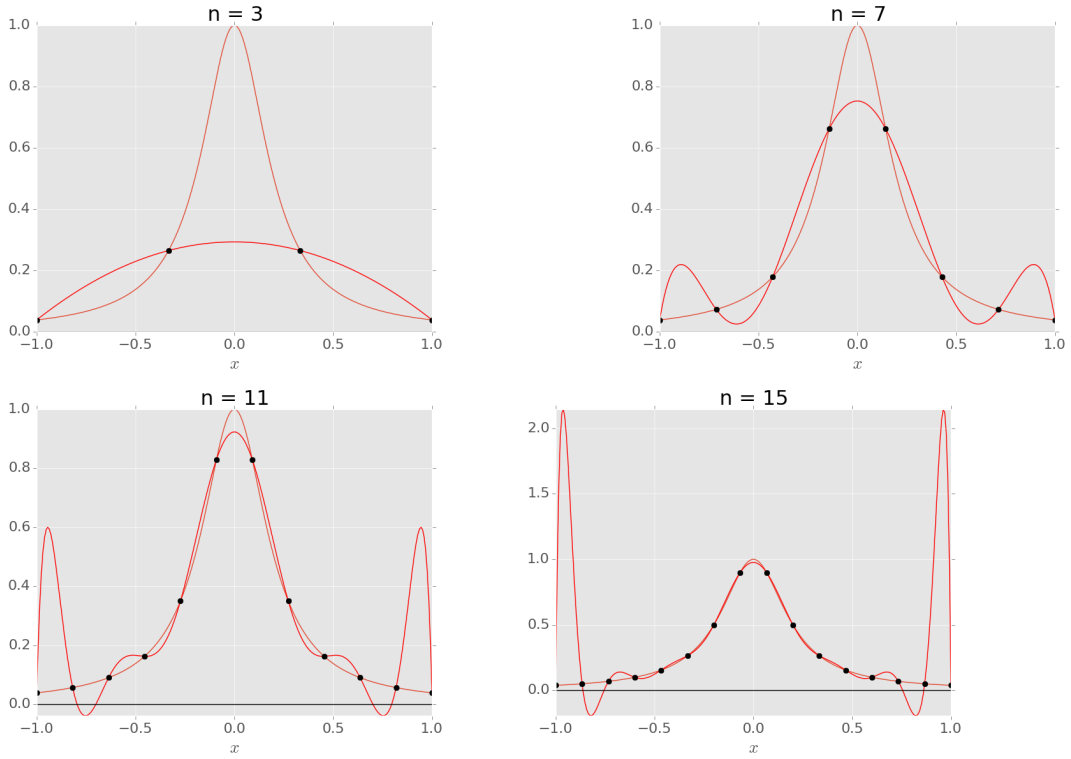
From the plot earlier, we see that this bound is satisfied (as it has to be), although not tight.

2.1.6 Node Placement: Chebyshev nodes

You might expect polynomial interpolation to *converge* as $n \to \infty$. Surprisingly, this is not the case if you take **equally-spaced** nodes x_i. This was shown by Runge in a famous 1901 paper.

Example 2.8: The Runge function $f(x) = 1/(1+25x^2)$ on $[-1, 1]$.

Here are illustrations of p_n for increasing n:



Notice that p_n is converging to f in the middle, but diverging more and more near the ends, even within the interval $[x_0, x_n]$. This is called the **Runge phenomenon**.

Note

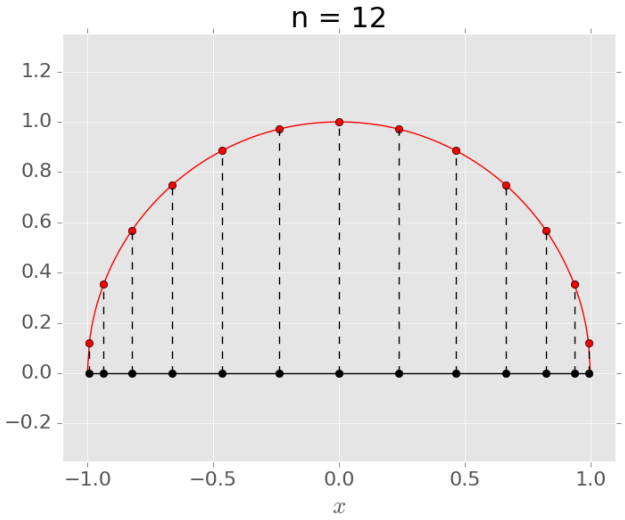
A full mathematical explanation for this divergence usually uses complex analysis — see Chapter 13 of *Approximation Theory and Approximation Practice* by L.N. Trefethen (SIAM, 2013). For a more elementary proof, see this StackExchange post.

The problem is (largely) coming from the **node polynomial**

$$w(x) = \prod_{i=0}^{n} (x - x_i),$$

which appears in the interpolation error formula. For equally spaced nodes, $|w(x)|$ is much larger near the ends of the interval than in the middle. We can avoid the Runge phenomenon by choosing different nodes x_i that are **not uniformly spaced**.

Since the problems are occurring near the ends of the interval, it would be logical to put more nodes there. A good choice is given by taking equally-spaced points on the unit circle $|z| = 1$, and projecting to the real line:



The points around the circle are

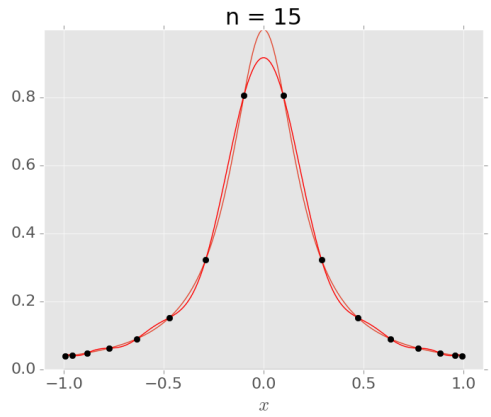
$$\phi_j = \frac{(2j+1)\pi}{2(n+1)}, \quad j = 0, \ldots, n,$$

so the corresponding **Chebyshev nodes** are

$$x_j = \cos\left[\frac{(2j+1)\pi}{2(n+1)}\right], \quad j = 0, \ldots, n.$$

Example 2.9: The Runge function $f(x) = 1/(1+25x^2)$ on $[-1, 1]$ using the Chebyshev nodes.

For $n = 3$, the nodes are $x_0 = \cos(\frac{\pi}{8})$, $x_1 = \cos(\frac{3\pi}{8})$, $x_2 = \cos(\frac{5\pi}{8})$, $x_3 = \cos(\frac{7\pi}{8})$.
Below we illustrate the resulting interpolant for $n = 15$:



Compare this to the example with equally spaced nodes.

In fact, the Chebyshev nodes are, in one sense, an optimal choice. They are the zeros of the **Chebyshev polynomial** $T_{n+1}(x) = \cos\left((n+1)\arccos(x)\right)$, and it can be shown that, among all choices of $n+1$ nodes in $[-1,1]$, they minimise $\max_{[-1,1]} |w(x)|$, giving $\max_{[-1,1]} |w(x)| = 2^{-n}$. Combined with the interpolation error theorem, this gives

$$\max_{[-1,1]} |f(x) - p_n(x)| \leq \frac{1}{2^n(n+1)!} \max_{[-1,1]} |f^{(n+1)}|.$$

So if all the derivatives of f are bounded by the same constant M on $[-1,1]$ (as for $\sin(x)$, $\cos(x)$ or e^x), the error tends to zero as $n \to \infty$.

i Note

For the Runge function, the derivatives $f^{(n+1)}$ grow very rapidly with n, so this bound alone does not explain the convergence seen above with Chebyshev nodes; a full explanation again uses complex analysis (see Trefethen's book).

Example 2.10: MATLAB demonstration: equally spaced vs Chebyshev nodes

```
f = @(x) 1 ./ (1 + 25*x.^2);      % Runge function
xx = linspace(-1, 1, 1000);
for n = [5 10 15 20]
    xe = linspace(-1, 1, n+1);                  % equally spaced nodes
    xc = cos((2*(0:n) + 1)*pi / (2*(n+1)));     % Chebyshev nodes
    pe = polyval(polyfit(xe, f(xe), n), xx);
    pc = polyval(polyfit(xc, f(xc), n), xx);
    fprintf('n = %2d: max error equispaced %.2e, Chebyshev %.2e\n', ...
        n, max(abs(f(xx) - pe)), max(abs(f(xx) - pc)));
end

% Side-by-side plots for n = 15
n = 15;
xe = linspace(-1, 1, n+1);
xc = cos((2*(0:n) + 1)*pi / (2*(n+1)));
nodes = {xe, xc};
titles = {'Equally spaced nodes', 'Chebyshev nodes'};
figure
for j = 1:2
    subplot(1, 2, j)
    xn = nodes{j};
    plot(xx, f(xx), 'LineWidth', 3), hold on
    plot(xx, polyval(polyfit(xn, f(xn), n), xx), '--', 'LineWidth', 3)
    plot(xn, f(xn), 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k'), hold off
    ylim([-0.5 2.5]), grid on
    title(sprintf('%s, n = %d', titles{j}, n))
    legend('Runge function f(x)', 'interpolant p_n(x)', 'nodes', 'Location', 'north')
    xlabel('x'), set(gca, 'FontSize', 18)
end
```

For larger n, MATLAB may warn that the polynomial is badly conditioned: `polyfit` works in the natural basis, so this is the Vandermonde problem from earlier.

2.2 Nonlinear Equations

How do we find roots of nonlinear equations?

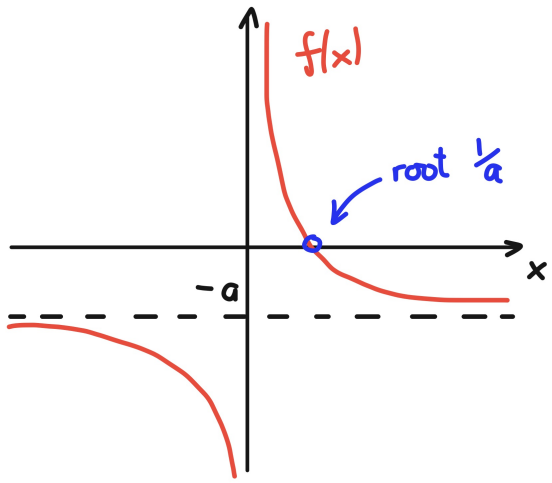
Given a general equation

$$f(x) = 0,$$

there will usually be no explicit formula for the root(s) x_*, so we must use an iterative method.

Rootfinding is a delicate business, and it is essential to begin by plotting a graph of $f(x)$, so that you can tell whether the answer you get from your numerical method is correct.

Example 2.11: $f(x) = \frac{1}{x} - a$, for $a > 0$.



Clearly we know the root is exactly $x_* = \frac{1}{a}$, but this will serve as a running example to test some of our methods.

2.2.1 Interval Bisection

If f is continuous and we can find an interval where it changes sign, then it must have a root in this interval. Formally, this is based on:

Theorem 2.5: Intermediate Value Theorem

If f is continuous on $[a, b]$ and c lies between $f(a)$ and $f(b)$, then there is at least one point $x \in [a, b]$ such that $f(x) = c$.

If $f(a)f(b) < 0$, then f changes sign at least once in $[a, b]$, so by the Intermediate Value Theorem there must be a point $x_* \in [a, b]$ where $f(x_*) = 0$.

We can turn this into the following iterative algorithm:

Algorithm 2.1: Interval bisection

Let f be continuous on $[a_0, b_0]$, with $f(a_0)f(b_0) < 0$.

- At each step, set $m_k = (a_k + b_k)/2$.
- If $f(a_k)f(m_k) \geq 0$ then set $a_{k+1} = m_k$, $b_{k+1} = b_k$, otherwise set $a_{k+1} = a_k$, $b_{k+1} = m_k$.
- Stop when $f(m_k) = 0$ or when $(b_k - a_k)/2$ is smaller than a chosen tolerance, and return m_k.

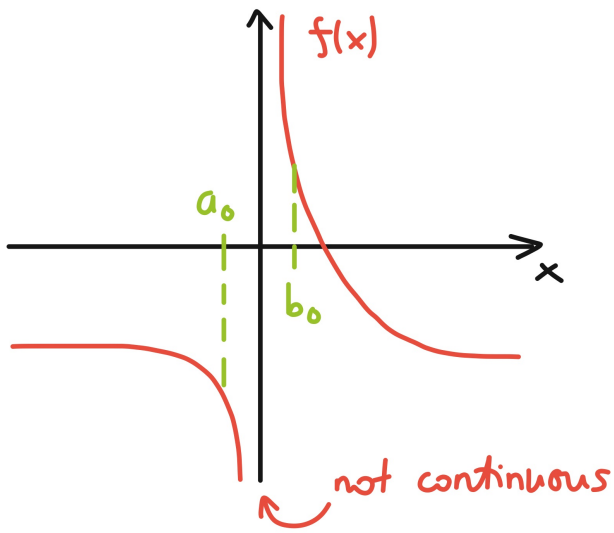
Example 2.12: $f(x) = \frac{1}{x} - 0.5$.

1. Try $a_0 = 1$, $b_0 = 3$ so that $f(a_0)f(b_0) = 0.5(-0.1666) < 0$. Now the midpoint is $m_0 = (1 + 3)/2 = 2$, with $f(m_0) = 0$. We are lucky and have already stumbled on the root $x_* = m_0 = 2$!

2. Suppose we had tried $a_0 = 1.5$, $b_0 = 3$, so $f(a_0) = 0.1666$ and $f(b_0) = -0.1666$, and again $f(a_0)f(b_0) < 0$. Now $m_0 = 2.25$, $f(m_0) = -0.0555$. We have $f(a_0)f(m_0) < 0$, so we set $a_1 = a_0 = 1.5$ and $b_1 = m_0 = 2.25$. The root must lie in $[1.5, 2.25]$. Now $m_1 = 1.875$, $f(m_1) = 0.0333$, and $f(a_1)f(m_1) > 0$, so we take $a_2 = m_1 = 1.875$, $b_2 = b_1 = 2.25$. The root must lie in $[1.875, 2.25]$. We can continue this algorithm, halving the length of the interval each time.

Since the interval halves in size at each iteration, and always contains a root, we are guaranteed to converge to a root provided that f is continuous on $[a_0, b_0]$. Stopping at step k, we get the minimum possible error by choosing m_k as our approximation.

Example 2.13: Same example with initial interval $[-0.5, 0.5]$.



In this case $f(a_0)f(b_0) < 0$, but there is no root in the interval. This is because f is not continuous on $[-0.5, 0.5]$ (it has a singularity at $x = 0$), so the Intermediate Value Theorem does not apply. Bisection will instead converge to the discontinuity at $x = 0$.

The rate of convergence is steady, so we can pre-determine how many iterations will be needed to converge to a given accuracy. After k iterations, the interval has length

$$|b_k - a_k| = \frac{|b_0 - a_0|}{2^k},$$

so the error in the mid-point satisfies

$$|m_k - x_*| \leq \frac{|b_0 - a_0|}{2^{k+1}}.$$

In order for $|m_k - x_*| \leq \delta$, it is enough that

$$\frac{|b_0 - a_0|}{2^{k+1}} \leq \delta \quad \Longleftrightarrow \quad \log|b_0 - a_0| - (k+1)\log(2) \leq \log(\delta) \quad \Longleftrightarrow \quad k \geq \frac{\log|b_0 - a_0| - \log(\delta)}{\log(2)} - 1.$$

Example 2.14: Previous example continued

With $a_0 = 1.5$, $b_0 = 3$, as in the above example, then for $\delta = \epsilon_{\mathrm{M}} = 1.1 \times 10^{-16}$ we would

need

$$k \geq \frac{\log(1.5) - \log(1.1 \times 10^{-16})}{\log(2)} - 1 \quad \implies k \geq 53 \text{ iterations.}$$

i Note

This convergence is pretty slow, but the method has the advantage of being very robust (i.e., use it if all else fails...). It has the more serious disadvantage of *only working in one dimension.*

2.2.2 Fixed point iteration

This is a very common type of rootfinding method. The idea is to transform $f(x) = 0$ into the form $g(x) = x$, so that a root x_* of f is a **fixed point** of g, meaning $g(x_*) = x_*$. To find x_*, we start from some initial guess x_0 and iterate

$$x_{k+1} = g(x_k)$$

until $|x_{k+1} - x_k|$ is sufficiently small. For a given equation $f(x) = 0$, there are many ways to transform it into the form $x = g(x)$. Only some will result in a convergent iteration.

Example 2.15: $f(x) = x^2 - 2x - 3$.

Note that the roots are -1 and 3. Consider some different rearrangements, with $x_0 = 0$.

(a) $g(x) = \sqrt{2x+3}$, gives $x_k \to 3$ [to machine accuracy after 33 iterations].

(b) $g(x) = 3/(x-2)$, gives $x_k \to -1$ [to machine accuracy after 33 iterations].

(c) $g(x) = (x^2 - 3)/2$, gives $x_k \to -1$ [but very slowly!].

(d) $g(x) = x^2 - x - 3$, gives $x_k \to \infty$.

(e) $g(x) = (x^2 + 3)/(2x - 2)$, gives $x_k \to -1$ [to machine accuracy after 5 iterations].

If instead we take $x_0 = 42$, then (a) and (b) still converge to the same roots, (c) now diverges, (d) still diverges, and (e) now converges to the other root $x_k \to 3$.

Example 2.16: MATLAB demonstration: comparing the rearrangements

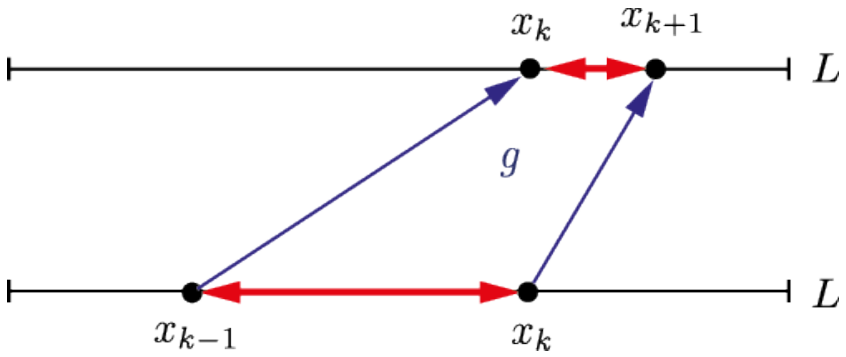
```
g = {@(x) sqrt(2*x + 3), @(x) 3./(x - 2), @(x) (x.^2 - 3)/2, ...
     @(x) x.^2 - x - 3,  @(x) (x.^2 + 3)./(2*x - 2)};
labels = 'abcde';
for j = 1:5
    x = 0;                                          % initial guess
    for k = 1:50
        x = g{j}(x);
    end
    fprintf('(%c): x_50 = %g\n', labels(j), x);
end
```

In this section, we will consider which iterations will converge, before addressing the *rate* of convergence in Section 2.2.3.

One way to ensure that the iteration will work is to find a **contraction mapping** g, which is a map $L \to L$ (for some closed interval L) satisfying

$$|g(x) - g(y)| \leq \lambda |x - y|$$

for some $\lambda < 1$ and for all x, $y \in L$. The sketch below shows the idea:



Theorem 2.6: Contraction Mapping Theorem

If g is a contraction mapping on $L = [a, b]$, then 1. There exists a unique fixed point $x_* \in L$ with $g(x_*) = x_*$. 2. For any $x_0 \in L$, the iteration $x_{k+1} = g(x_k)$ will converge to x_* as $k \to \infty$.

Proof (non-examinable)

To prove *existence*, consider $h(x) = g(x) - x$. Since $g : L \to L$ we have $h(a) = g(a) - a \geq 0$ and $h(b) = g(b) - b \leq 0$. Moreover, it follows from the contraction property above that g is continuous (think of "$\epsilon\delta$"), therefore so is h. So the Intermediate Value Theorem guarantees the existence of at least one point $x_* \in L$ such that $h(x_*) = 0$, i.e. $g(x_*) = x_*$.
For *uniqueness*, suppose x_* and y_* are both fixed points of g in L. Then

$$|x_* - y_*| = |g(x_*) - g(y_*)| \leq \lambda |x_* - y_*| < |x_* - y_*|,$$

which is a contradiction.
Finally, to show *convergence*, consider

$$|x_* - x_{k+1}| = |g(x_*) - g(x_k)| \leq \lambda |x_* - x_k| \leq \ldots \leq \lambda^{k+1} |x_* - x_0|.$$

Since $\lambda < 1$, we see that $x_k \to x_*$ as $k \to \infty$.

Note

The Contraction Mapping Theorem is also known as the **Banach fixed point theorem**, and was proved by Stefan Banach in his 1920 PhD thesis.

To apply this result in practice, we need to know whether a given function g is a contraction mapping on some interval.

If g is differentiable, then the mean value theorem says that, for any $x, y \in L$, there exists ξ between x and y with

$$g(x) = g(y) + g'(\xi)(x - y) \quad \Longrightarrow \quad |g(x) - g(y)| \leq \left(\max_{\xi \in L} |g'(\xi)| \right) |x - y|.$$

So if (a) $g : L \to L$ and (b) $|g'(x)| \leq M$ for all $x \in L$ with $M < 1$, then g is a contraction mapping on L.

Example 2.17: Iteration (a) from previous example, $g(x) = \sqrt{2x + 3}$.

Here $g' = (2x + 3)^{-1/2}$, so we see that $|g'(x)| < 1$ for all $x > -1$.
For g to be a contraction mapping on an interval L, we also need that g maps L into itself. Since our particular g is continuous and monotonic increasing (for $x > -\frac{3}{2}$), it will map an interval $[a, b]$ to another interval whose end-points are $g(a)$ and $g(b)$.
For example, $g(-\frac{1}{2}) = \sqrt{2}$ and $g(4) = \sqrt{11}$, so the interval $L = [-\frac{1}{2}, 4]$ is mapped into itself. It follows by the Contraction Mapping Theorem that (1) there is a unique fixed point $x_* \in [-\frac{1}{2}, 4]$ (which we know is $x_* = 3$), and (2) the iteration will converge to x_* for any x_0 in this interval (as we saw for $x_0 = 0$).

In practice, it is not always easy to find a suitable interval L. But knowing that $|g'(x_*)| < 1$ is enough to guarantee that the iteration will converge if x_0 is close enough to x_*.

Theorem 2.7: Local Convergence Theorem

Let g and g' be continuous in the neighbourhood of an isolated fixed point $x_* = g(x_*)$. If $|g'(x_*)| < 1$ then there is an interval $L = [x_* - \delta, x_* + \delta]$ such that $x_{k+1} = g(x_k)$ converges to x_* whenever $x_0 \in L$.

i Proof (non-examinable)

By continuity of g', there exists some interval $L = [x_* - \delta, x_* + \delta]$ with $\delta > 0$ such that $|g'(x)| \leq M$ for some $M < 1$, for all $x \in L$. Now let $x \in L$. It follows that

$$|x_* - g(x)| = |g(x_*) - g(x)| \leq M|x_* - x| < |x_* - x| \leq \delta,$$

so $g(x) \in L$. Hence g is a contraction mapping on L and the Contraction Mapping Theorem shows that $x_k \to x_*$.

Example 2.18: Iteration (a) again, $g(x) = \sqrt{2x+3}$.

Here we know that $x_* = 3$, and $|g'(3)| = \frac{1}{3} < 1$, so the Local Convergence Theorem tells us that the iteration will converge to 3 if x_0 is close enough to 3.

Example 2.19: Iteration (e) again, $g(x) = (x^2+3)/(2x-2)$.

Here we have

$$g'(x) = \frac{x^2 - 2x - 3}{2(x-1)^2},$$

so we see that $g'(-1) = g'(3) = 0 < 1$. So the Local Convergence Theorem tells us that the iteration will converge to either root if we start close enough.

i Note

As we will see, the fact that $g'(x_*) = 0$ is related to the fast convergence of iteration (e).

2.2.3 Orders of convergence

To measure the speed of convergence, we compare the error $|x_* - x_{k+1}|$ to the error at the previous step, $|x_* - x_k|$. We can compare a few different iteration schemes that should converge to the same answer to get a sense for how our choice of scheme can impact the speed of

convergence.

Example 2.20: Iteration (a) again, $g(x) = \sqrt{2x+3}$.

Look at the sequence of errors in this case:

x_k	$\lvert 3 - x_k \rvert$	$\lvert 3 - x_k \rvert / \lvert 3 - x_{k-1} \rvert$
0.0000000000	3.0000000000	-
1.7320508076	1.2679491924	0.4226497308
2.5424597568	0.4575402432	0.3608506129
2.8433992885	0.1566007115	0.3422665304
2.9473375404	0.0526624596	0.3362849319
2.9823941860	0.0176058140	0.3343143126
2.9941256440	0.0058743560	0.3336600063

We see that the ratio $|x_* - x_k|/|x_* - x_{k-1}|$ is less than 1, and seems to be converging to a constant $\lambda \approx \frac{1}{3}$.

Example 2.21: Iteration (e) again, $g(x) = (x^2+3)/(2x-2)$.

Now the sequence is:

x_k	$\lvert (-1) - x_k \rvert$	$\lvert (-1) - x_k \rvert / \lvert (-1) - x_{k-1} \rvert$
0.0000000000	1.0000000000	-
-1.5000000000	0.5000000000	0.5000000000
-1.0500000000	0.0500000000	0.1000000000
-1.0006097561	0.0006097561	0.0121951220
-1.0000000929	0.0000000929	0.0001523926

Again the ratio $|x_* - x_k|/|x_* - x_{k-1}|$ is less than 1, but this time it seems to tend to 0 as $k \to \infty$, meaning that the convergence is in some sense "accelerating".

Definition 2.1: Linear and superlinear convergence

Suppose that $x_k \to x_*$ as $k \to \infty$. We say that the sequence $\{x_k\}$ **converges linearly** with **rate** λ if

$$\lim_{k\to\infty} \frac{|x_* - x_{k+1}|}{|x_* - x_k|} = \lambda \quad \text{with} \quad 0 < \lambda < 1.$$

If this limit is $\lambda = 0$, then the convergence is **superlinear**.

So iteration (a) converges linearly with rate $\lambda = \frac{1}{3}$, and iteration (e) converges superlinearly.

Note

Interval bisection does not quite fit this definition, since its error can fluctuate from one step to the next (in our first example, m_0 landed exactly on the root). However, the error *bound* $|b_k - a_k|/2$ halves at every step, so bisection is usually described as converging linearly with rate $\frac{1}{2}$.

The following result establishes conditions for linear and superlinear convergence.

Theorem 2.8

Let g' be continuous in the neighbourhood of a fixed point $x_* = g(x_*)$, and suppose that $x_{k+1} = g(x_k)$ converges to x_* as $k \to \infty$.

1. If $|g'(x_*)| \neq 0$ then the convergence will be linear with rate $\lambda = |g'(x_*)|$.
2. If $|g'(x_*)| = 0$ then the convergence will be superlinear.

Proof: By the mean value theorem, note that

$$x_* - x_{k+1} = g(x_*) - g(x_k) = g(x_*) - \Big[g(x_*) + g'(\xi_k)(x_k - x_*)\Big] = g'(\xi_k)(x_* - x_k)$$

for some ξ_k between x_* and x_k. Since $x_k \to x_*$, we have $\xi_k \to x_*$ as $k \to \infty$, so

$$\lim_{k\to\infty} \frac{|x_* - x_{k+1}|}{|x_* - x_k|} = \lim_{k\to\infty} |g'(\xi_k)| = |g'(x_*)|.$$

This proves the result.

Example 2.22: Iterations (a) and (e) again.

For (a), we saw before that $g'(3) = \frac{1}{3}$, so the theorem above shows that convergence will be linear with $\lambda = |g'(3)| = \frac{1}{3}$, as we found numerically. For (e), we saw that $g'(-1) = 0$, so the theorem shows that convergence will be superlinear, again consistent with our numerical findings.

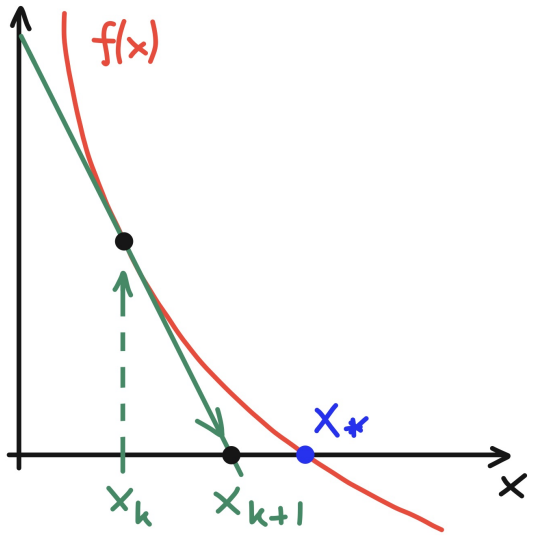
We can further classify convergence by its **order**. If

$$\lim_{k\to\infty} \frac{|x_* - x_{k+1}|}{|x_* - x_k|^\alpha} = C \quad \text{for some constant} \quad 0 < C < \infty,$$

then we say that the sequence converges with **order** α. Linear convergence corresponds to $\alpha = 1$ (with $C = \lambda$), $\alpha = 2$ is called **quadratic** convergence and $\alpha = 3$ is called **cubic** convergence, although for a general sequence α need not be an integer (e.g. the secant method below).

2.2.4 Newton's method

This is a particular fixed point iteration that is very widely used because (as we will see) it usually converges quadratically.



Graphically, the idea of **Newton's method** is simple: given x_k, draw the tangent line to f at $x = x_k$, and let x_{k+1} be the x-intercept of this tangent. So

$$\frac{0 - f(x_k)}{x_{k+1} - x_k} = f'(x_k) \quad \Longrightarrow \quad x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

> **Note**
>
> In fact, Newton only applied the method to polynomial equations, and without using calculus. The general form using derivatives ("fluxions") was first published by Thomas Simpson in 1740. [See "Historical Development of the Newton-Raphson Method" by T.J. Ypma, *SIAM Review* **37**, 531 (1995).]

Another way to derive this iteration is to approximate $f(x)$ by the linear part of its Taylor series centred at x_k:

$$0 \approx f(x_{k+1}) \approx f(x_k) + f'(x_k)(x_{k+1} - x_k).$$

The iteration function for Newton's method is

$$g(x) = x - \frac{f(x)}{f'(x)},$$

so using $f(x_*) = 0$ we see that $g(x_*) = x_*$. To assess the convergence, note that

$$g'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2} \quad \Longrightarrow \; g'(x_*) = 0 \quad \text{if } f'(x_*) \neq 0.$$

So if $f'(x_*) \neq 0$, the Local Convergence Theorem shows that the iteration will converge for x_0 close enough to x_*. Moreover, since $g'(x_*) = 0$, the theorem in Section 2.2.3 shows that this convergence will be superlinear. In fact, expanding $f(x_k)$ and $f'(x_k)$ in Taylor series about x_* shows that

$$x_* - x_{k+1} = -\frac{f''(x_*)}{2f'(x_*)}(x_* - x_k)^2 + \mathcal{O}((x_* - x_k)^3),$$

so the convergence is quadratic (order $\alpha = 2$), provided that $f''(x_*) \neq 0$.

i Big-O notation

Here we used **big-O notation**: we write $g(h) = \mathcal{O}(h^p)$ as $h \to 0$ if there is a constant M such that $|g(h)| \leq M|h|^p$ for all sufficiently small h. In other words, $g(h)$ gets smaller at least as fast as h^p. Above, $h = x_* - x_k$, so the remainder is bounded by a constant times $|x_* - x_k|^3$.

Example 2.23: Calculate a^{-1} using $f(x) = \frac{1}{x} - a$ for $a > 0$.

Newton's method gives the iterative formula

$$x_{k+1} = x_k - \frac{\frac{1}{x_k} - a}{-\frac{1}{x_k^2}} = 2x_k - ax_k^2.$$

Rearranging, we find that $\frac{1}{a} - x_{k+1} = a\left(\frac{1}{a} - x_k\right)^2$ exactly, so the iteration will converge for any $x_0 \in (0, 2/a)$, but will diverge if x_0 is too large. With $a = 0.5$ and $x_0 = 1$, we obtain

x_k	$\lvert 2 - x_k \rvert$	$\lvert 2 - x_k \rvert / \lvert 2 - x_{k-1} \rvert$	$\lvert 2 - x_k \rvert / \lvert 2 - x_{k-1} \rvert^2$
1.0	1.0	-	-
1.5	0.5	0.5	0.5
1.875	0.125	0.25	0.5
1.9921875	0.0078125	0.0625	0.5
1.999969482	3.05×10^{-5}	0.00390625	0.5
2.0	4.65×10^{-10}	1.53×10^{-5}	0.5
2.0	1.08×10^{-19}	2.33×10^{-10}	0.5

In 6 steps, the error is below ϵ_M: pretty rapid convergence! The third column shows that the convergence is superlinear. The fourth column shows that $|x_* - x_{k+1}|/|x_* - x_k|^2$ is constant, indicating that the convergence is quadratic (order $\alpha = 2$), with $C = |f''(x_*)/(2f'(x_*))| = a = 0.5$, as predicted.

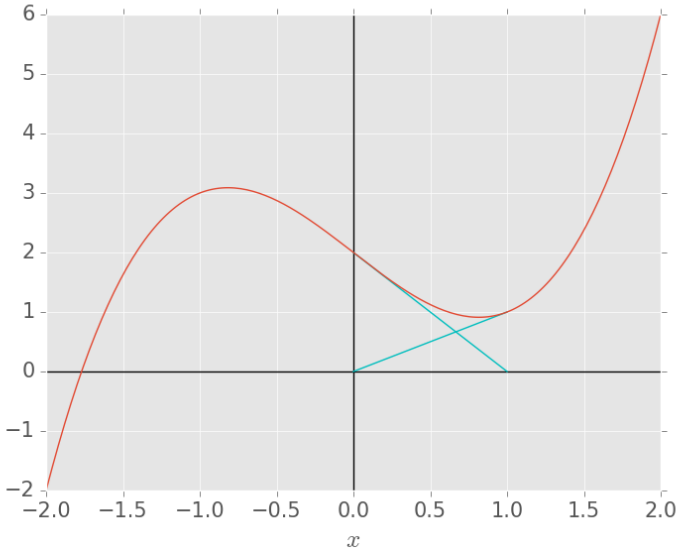
Note

Although the solution $\frac{1}{a}$ is known exactly, this method is so efficient that it is sometimes used in computer hardware to do division!

In practice, it is not usually possible to determine ahead of time whether a given starting value x_0 will converge. A robust computer implementation should catch any attempt to take too large a step, and switch to a less sensitive (but slower) algorithm (e.g. bisection). However, it always makes sense to avoid any points where $f'(x) = 0$.

Example 2.24: $f(x) = x^3 - 2x + 2$.

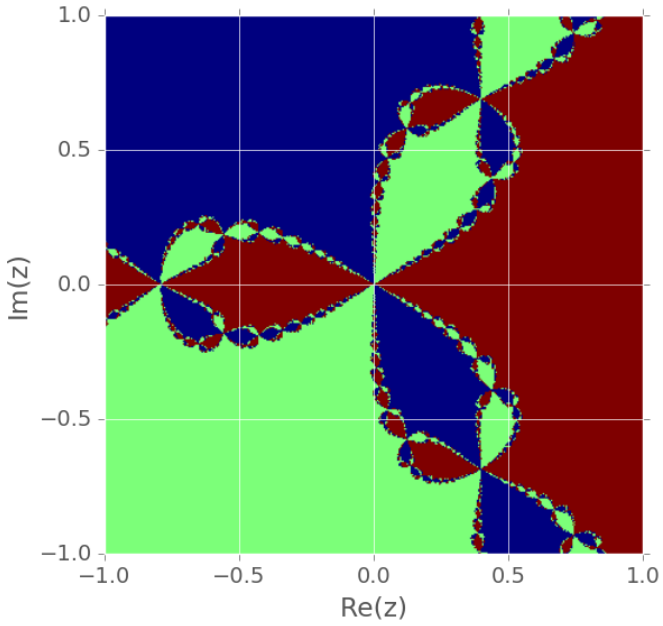
Here $f'(x) = 3x^2 - 2$ so there are turning points at $x = \pm\sqrt{\frac{2}{3}}$ where $f'(x) = 0$, as well as a single real root at $x_* \approx -1.769$. The presence of points where $f'(x) = 0$ means that care is needed in choosing a starting value x_0.
If we take $x_0 = 0$, then $x_1 = 0 - f(0)/f'(0) = 1$, but then $x_2 = 1 - f(1)/f'(1) = 0$, so the iteration gets stuck in an infinite loop:



Other starting values, e.g. $x_0 = -0.5$ can also be sucked into this infinite loop! The correct answer is obtained for $x_0 = -1.0$.

Note

The sensitivity of Newton's method to the choice of x_0 is beautifully illustrated by applying it to a **complex** function such as $f(z) = z^3 - 1$. The following plot colours points z_0 in the complex plane according to which root they converge to (1, $e^{2\pi i/3}$, or $e^{-2\pi i/3}$):



The boundaries of these **basins of attraction** are fractal.

Example 2.25: MATLAB demonstration: comparing convergence rates

```
% Bisection, fixed-point iteration (a) and Newton's method for x^2 - 2x - 3 = 0
f  = @(x) x.^2 - 2*x - 3;
df = @(x) 2*x - 2;
xs = 3;                                  % the root we are aiming for
N = 40;  eb = zeros(N,1);  ef = eb;  en = eb;
a = 2;  b = 5;  xf = 0;  xn = 5;         % starting values
for k = 1:N
    m = (a + b)/2;  eb(k) = abs(m - xs);
    if f(a)*f(m) >= 0, a = m; else, b = m; end
    xf = sqrt(2*xf + 3);       ef(k) = abs(xf - xs);
    xn = xn - f(xn)/df(xn);    en(k) = abs(xn - xs);
end
figure
semilogy(1:N, eb, 'o-', 1:N, ef, 's-', 1:N, en, '^-', 'LineWidth', 2.5, 'MarkerSize', 8)
legend('bisection', 'fixed point (a)', 'Newton', 'Location', 'southwest')
xlabel('iteration k'), ylabel('error |x_k - x_*|'), set(gca, 'FontSize', 18), grid on
```

On a logarithmic scale, linear convergence appears as a straight line, while Newton's method plunges to machine precision within a few steps (errors that are exactly zero are not plotted). MATLAB's built-in rootfinder `fzero(f, [2 5])` combines bisection with faster methods.

2.2.5 Newton's method for systems

Newton's method generalises to higher-dimensional problems where we want to find $\mathbf{x} \in \mathbb{R}^m$ that satisfies $\mathbf{f}(\mathbf{x}) = 0$ for some function $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^m$.

To see how it works, take $m = 2$ so that $\mathbf{x} = (x_1, x_2)^\top$ and $\mathbf{f} = [f_1(\mathbf{x}), f_2(\mathbf{x})]^\top$. Taking the linear terms in Taylor's theorem for two variables gives

$$0 \approx f_1(\mathbf{x}_{k+1}) \approx f_1(\mathbf{x}_k) + \left.\frac{\partial f_1}{\partial x_1}\right|_{\mathbf{x}_k} (x_{1,k+1} - x_{1,k}) + \left.\frac{\partial f_1}{\partial x_2}\right|_{\mathbf{x}_k} (x_{2,k+1} - x_{2,k}),$$
$$0 \approx f_2(\mathbf{x}_{k+1}) \approx f_2(\mathbf{x}_k) + \left.\frac{\partial f_2}{\partial x_1}\right|_{\mathbf{x}_k} (x_{1,k+1} - x_{1,k}) + \left.\frac{\partial f_2}{\partial x_2}\right|_{\mathbf{x}_k} (x_{2,k+1} - x_{2,k}).$$

In matrix form, we can write

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} f_1(\mathbf{x}_k) \\ f_2(\mathbf{x}_k) \end{pmatrix} + \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}_k) & \frac{\partial f_1}{\partial x_2}(\mathbf{x}_k) \\ \frac{\partial f_2}{\partial x_1}(\mathbf{x}_k) & \frac{\partial f_2}{\partial x_2}(\mathbf{x}_k) \end{pmatrix} \begin{pmatrix} x_{1,k+1} - x_{1,k} \\ x_{2,k+1} - x_{2,k} \end{pmatrix}.$$

The matrix of partial derivatives is called the **Jacobian matrix** $J(\mathbf{x}_k)$, so (for any m) we have

$$\mathbf{0} = \mathbf{f}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k).$$

To derive Newton's method, we rearrange this equation for $\mathbf{x}_{k+1}$. At each step, we solve the linear system

$$J(\mathbf{x}_k)\boldsymbol{\delta}_k = -\mathbf{f}(\mathbf{x}_k)$$

for the update $\boldsymbol{\delta}_k$, and then set $\mathbf{x}_{k+1} = \mathbf{x}_k + \boldsymbol{\delta}_k$. Formally, $\mathbf{x}_{k+1} = \mathbf{x}_k - J^{-1}(\mathbf{x}_k)\mathbf{f}(\mathbf{x}_k)$, but in practice we never compute J^{-1} explicitly: solving the linear system is cheaper and more accurate, as we will see in Chapter 3.

> **Note**
>
> If $m = 1$, then $J(x_k) = \frac{\partial f}{\partial x}(x_k)$, and the linear system is just $f'(x_k)\delta_k = -f(x_k)$, so this reduces to the scalar Newton's method.

Example 2.26: Apply Newton's method to the simultaneous equations $xy - y^3 - 1 = 0$ and $x^2y + y - 5 = 0$, with starting values $x_0 = 2$, $y_0 = 3$.

The Jacobian matrix is

$$J(x, y) = \begin{pmatrix} y & x - 3y^2 \\ 2xy & x^2 + 1 \end{pmatrix}.$$

For the first iteration, we have

$$J(2,3) = \begin{pmatrix} 3 & -25 \\ 12 & 5 \end{pmatrix}, \qquad \mathbf{f}(2,3) = \begin{pmatrix} -22 \\ 10 \end{pmatrix},$$

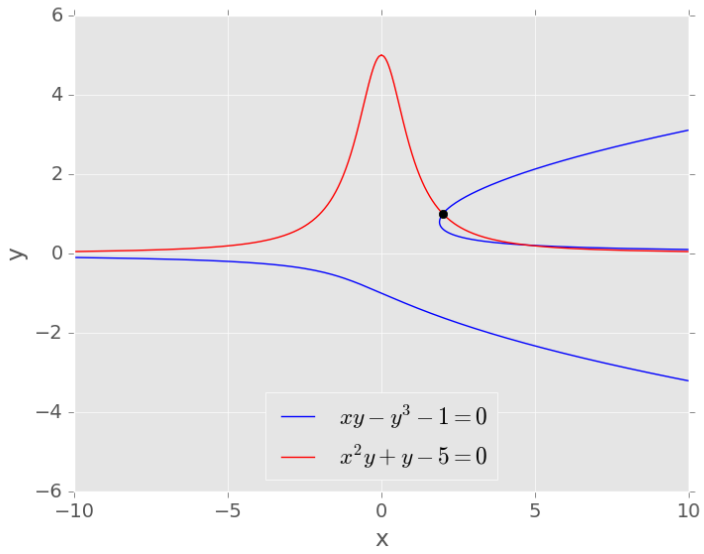
and solving $J(2,3)\boldsymbol{\delta}_0 = -\mathbf{f}(2,3)$ gives $\boldsymbol{\delta}_0 = (-\frac{4}{9}, -\frac{14}{15})^\top$, so

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} -0.44444444 \\ -0.93333333 \end{pmatrix} = \begin{pmatrix} 1.55555556 \\ 2.06666667 \end{pmatrix}.$$

Subsequent iterations give

k	x_k	y_k
2	1.54720541	1.47779333
3	1.78053503	1.15886481
4	1.95284300	1.02844269
5	1.99776297	1.00124041
6	1.99999524	1.00000260

so the method is converging accurately to the root $x_* = 2$, $y_* = 1$, shown in the following plot:



Example 2.27: MATLAB demonstration: Newton's method for systems

```
F = @(v) [v(1)*v(2) - v(2)^3 - 1;  v(1)^2*v(2) + v(2) - 5];
J = @(v) [v(2), v(1) - 3*v(2)^2;  2*v(1)*v(2), v(1)^2 + 1];
v = [2; 3];                                % starting values (x_0, y_0)
for k = 1:8
    v = v - J(v) \ F(v);                   % solve J*delta = -F, no inverse needed
    fprintf('k = %d: x = %.10f, y = %.10f\n', k, v(1), v(2));
end
```

By generalising the scalar analysis (beyond the scope of this course), it can be shown that the convergence is quadratic for $\mathbf{x}_0$ sufficiently close to $\mathbf{x}_*$, provided that $J(\mathbf{x}_*)$ is non-singular (i.e., $\det[J(\mathbf{x}_*)] \neq 0$).

Note

In general, finding a good starting point in more than one dimension is difficult, particularly because interval bisection is not available. Algorithms that try to mimic bisection in

higher dimensions are available, proceeding by a 'grid search' approach.

2.2.6 The secant method

A drawback of Newton's method is that the derivative $f'(x_k)$ must be computed at each iteration. This may be expensive to compute, or may not be available as a formula. For example, the function f might be the right-hand side of some complex partial differential equation, and hence both difficult to differentiate and very high dimensional!

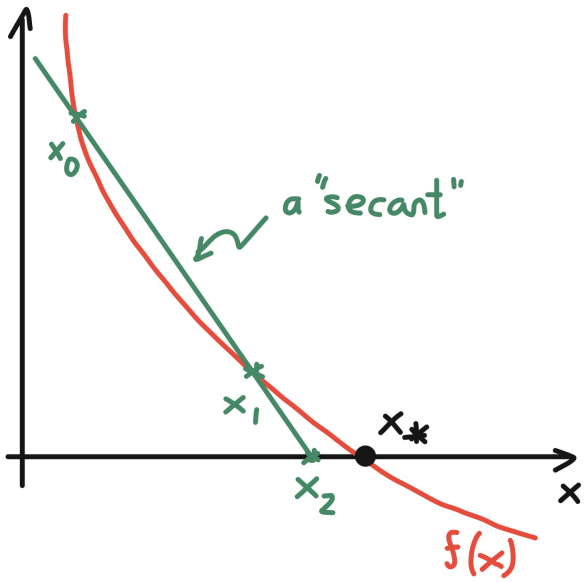
Instead we can use a **quasi-Newton method**

$$x_{k+1} = x_k - \frac{f(x_k)}{g_k},$$

where g_k is some easily-computed approximation to $f'(x_k)$. Once the iteration has started, we already have two nearby points x_{k-1}, x_k, so we can approximate $f'(x_k)$ by a backward difference

$$g_k = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} \qquad \Longrightarrow \qquad x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}.$$

This is called the **secant method**, and requires only one new function evaluation per iteration (once underway). The name comes from its graphical interpretation:



Note

The secant method was introduced by Newton.

Example 2.28: $f(x) = \frac{1}{x} - 0.5$.

Now we need two starting values, so take $x_0 = 0.25$, $x_1 = 0.5$. The secant method gives:

k	x_k	$\lvert x_* - x_k\rvert / \lvert x_* - x_{k-1}\rvert$
2	0.6875	0.875
3	1.01562	0.75
4	1.354	0.65625
5	1.68205	0.492188
6	1.8973	0.322998
7	1.98367	0.158976
8	1.99916	0.0513488

Convergence to ϵ_{M} is achieved in 12 iterations. Notice that the error ratio is decreasing, so the convergence is superlinear.

The secant method is a **two-point method** since $x_{k+1} = g(x_{k-1}, x_k)$, so the theorems above about single-point fixed-point iterations do not apply directly. Nevertheless, a Taylor expansion similar to the one for Newton's method gives the following result, which we state without proof.

Theorem 2.9

If $f'(x_*) \neq 0$ then the secant method converges for x_0, x_1 sufficiently close to x_*, with

$$x_* - x_{k+1} \approx -\frac{f''(x_*)}{2f'(x_*)}(x_* - x_k)(x_* - x_{k-1}),$$

and the order of convergence is $(1 + \sqrt{5})/2 = 1.618\ldots$.

Note

This illustrates that orders of convergence need not be integers, and is also an appearance of the **golden ratio**. Although its order is lower than Newton's method, the secant method needs only one function evaluation per step (and no derivatives), so it is often faster in practice.

Knowledge checklist

Key topics:

1. Taylor's theorem and the Lagrange form of the remainder (truncation error).
2. Polynomial interpolation: existence and uniqueness, the Lagrange form, the interpolation error theorem, the Runge phenomenon, and Chebyshev nodes.
3. Rootfinding methods: interval bisection, fixed-point iteration, Newton's method (including for systems), and the secant method.
4. Convergence of iterative methods: contraction mappings, local convergence, and orders of convergence (linear, superlinear, quadratic).

Key skills:

- Constructing polynomial interpolants for a given data set or function using the Lagrange form, and bounding the error using Taylor's theorem and the interpolation error theorem.
- Explaining the Runge phenomenon and how the choice of interpolation nodes affects convergence.
- Implementing and analysing bisection, fixed-point and Newton iterations, and estimating their order of convergence from numerical data.

3 Linear Algebra

3.1 Systems of Linear Equations

The central goal of this chapter is to answer the following seemingly straightforward question:

How do we solve a linear system numerically?

Linear systems of the form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n &= b_2, \\ \vdots \quad & \quad \vdots \quad \quad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n &= b_n \end{aligned}$$

occur in many applications (often with very large n). It is convenient to express this in matrix form:

$$A\mathbf{x} = \mathbf{b},$$

where A is an $n \times n$ square matrix with elements a_{ij}, and $\mathbf{x}$, $\mathbf{b}$ are $n \times 1$ vectors.

We will need some basic facts from linear algebra:

1. $A^\top$ is the **transpose** of A, so $(A^\top)_{ij} = a_{ji}$.
2. A is **symmetric** if $A = A^\top$.
3. A is **non-singular** iff there exists a solution $\mathbf{x} \in \mathbb{R}^n$ for every $\mathbf{b} \in \mathbb{R}^n$.
4. A is non-singular iff $\det(A) \neq 0$.
5. A is non-singular iff there exists a unique **inverse** A^{-1} such that $AA^{-1} = A^{-1}A = I$.

It follows from fact 5 above that $A\mathbf{x} = \mathbf{b}$ has a unique solution iff A is non-singular, given by $\mathbf{x} = A^{-1}\mathbf{b}$.

In this chapter, we will see how to solve $A\mathbf{x} = \mathbf{b}$ both **efficiently** and **accurately**.

Although this seems like a conceptually easy problem (just use Gaussian elimination!), it is actually a hard one when n gets large. Nowadays, linear systems with $n = 1$ million arise routinely in computational problems. For example, in Chapter 6 we will solve the heat equation

using the backward Euler method, which requires solving a linear system $(I - \Delta t\, \alpha A)\mathbf{u}_{n+1} = \mathbf{u}_n$ at *every* time step, where n is the number of spatial grid points. And even for small n there are some potential pitfalls, as we will see.

> **Note**
>
> If A is instead rectangular ($m \times n$ with $m > n$), then there are more equations than unknowns, and there is usually no exact solution. Instead, we can look for the **least-squares** solution, which minimises $\|A\mathbf{x} - \mathbf{b}\|_2$ (norms are defined later in this chapter). In MATLAB, `x = A\b` returns the least-squares solution when A is rectangular.

Many algorithms are based on the idea of rewriting $A\mathbf{x} = \mathbf{b}$ in a form where the matrix is easier to invert. Easiest to invert are diagonal matrices, followed by orthogonal matrices (where $A^{-1} = A^\top$). However, the most common method for solving $A\mathbf{x} = \mathbf{b}$ transforms the system to *triangular* form.

3.2 Triangular systems

If the matrix A is triangular, then $A\mathbf{x} = \mathbf{b}$ is straightforward to solve.

A matrix L is called **lower triangular** if all entries above the diagonal are zero:

$$L = \begin{pmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \ddots & \vdots \\ \vdots & & \ddots & 0 \\ l_{n1} & \cdots & \cdots & l_{nn} \end{pmatrix}.$$

The determinant is just

$$\det(L) = l_{11} l_{22} \cdots l_{nn},$$

so the matrix will be non-singular iff all of the diagonal elements are non-zero.

Example 3.1: Solve $L\mathbf{x} = \mathbf{b}$ for $n = 4$.

The system is

$$\begin{pmatrix} l_{11} & 0 & 0 & 0 \\ l_{21} & l_{22} & 0 & 0 \\ l_{31} & l_{32} & l_{33} & 0 \\ l_{41} & l_{42} & l_{43} & l_{44} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

which is equivalent to

$$\begin{aligned} l_{11}x_1 &= b_1, \\ l_{21}x_1 + l_{22}x_2 &= b_2, \\ l_{31}x_1 + l_{32}x_2 + l_{33}x_3 &= b_3, \\ l_{41}x_1 + l_{42}x_2 + l_{43}x_3 + l_{44}x_4 &= b_4. \end{aligned}$$

We can just solve step-by-step:

$$x_1 = \frac{b_1}{l_{11}}, \; x_2 = \frac{b_2 - l_{21}x_1}{l_{22}},$$

$$x_3 = \frac{b_3 - l_{31}x_1 - l_{32}x_2}{l_{33}}, \; x_4 = \frac{b_4 - l_{41}x_1 - l_{42}x_2 - l_{43}x_3}{l_{44}}.$$

This is fine since we know that l_{11}, l_{22}, l_{33}, l_{44} are all non-zero when a solution exists.

In general, any lower triangular system $L\mathbf{x} = \mathbf{b}$ can be solved by **forward substitution**

$$x_j = \frac{b_j - \sum_{k=1}^{j-1} l_{jk}x_k}{l_{jj}}, \quad j = 1, \ldots, n.$$

Similarly, an **upper triangular** matrix U has the form

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix},$$

and an upper-triangular system $U\mathbf{x} = \mathbf{b}$ may be solved by **backward substitution**

$$x_j = \frac{b_j - \sum_{k=j+1}^{n} u_{jk}x_k}{u_{jj}}, \quad j = n, \ldots, 1.$$

To estimate the computational cost of forward substitution, we can count the number of floating-point operations ($+$, $-$, $\times$, $\div$).

Example 3.2: Number of operations required for forward substitution.

Consider each x_j. We have - $j = 1$: 1 division - $j = 2$: 1 division + [1 subtraction + 1 multiplication] - $j = 3$: 1 division + 2 $\times$[1 subtraction + 1 multiplication] - $\vdots$ - $j = n$: 1 division + $(n-1)$ $\times$[1 subtraction + 1 multiplication]

So the total number of operations required is

$$\sum_{j=1}^{n} \Big(1 + 2(j-1)\Big) = 2\sum_{j=1}^{n} j - \sum_{j=1}^{n} 1 = n(n+1) - n = n^2.$$

So solving a triangular system by forward (or backward) substitution takes n^2 operations. We may say that the **computational complexity** of the algorithm is n^2.

> **Note**
>
> In practice, this is only a rough estimate of the computational cost, because reading from and writing to the computer's memory also take time. This can be estimated given a "memory model", but this depends on the particular computer.

3.3 Gaussian elimination

If our matrix A is not triangular, we can try to transform it to triangular form. **Gaussian elimination** uses elementary row operations to transform the system to upper triangular form $U\mathbf{x} = \mathbf{y}$.

Elementary row operations include swapping rows and adding multiples of one row to another. They won't change the solution $\mathbf{x}$, but will change the matrix A and the right-hand side $\mathbf{b}$.

Example 3.3: Transform to upper triangular form the system

$$\begin{aligned} x_1 + 2x_2 + x_3 &= 0, \\ x_1 - 2x_2 + 2x_3 &= 4, \\ 2x_1 + 12x_2 - 2x_3 &= 4. \end{aligned}$$

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 1 & -2 & 2 \\ 2 & 12 & -2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ 4 \\ 4 \end{pmatrix}.$$

Stage 1. Subtract 1 times equation 1 from equation 2, and 2 times equation 1 from equation 3, so as to eliminate x_1 from equations 2 and 3:

$$\begin{aligned} x_1 + 2x_2 + x_3 &= 0, \\ -4x_2 + x_3 &= 4, \\ 8x_2 - 4x_3 &= 4. \end{aligned}$$

$$A^{(2)} = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 8 & -4 \end{pmatrix} \quad \mathbf{b}^{(2)} = \begin{pmatrix} 0 \\ 4 \\ 4 \end{pmatrix}, \quad m_{21} = 1, \quad m_{31} = 2.$$

Stage 2. Subtract -2 times equation 2 from equation 3, to eliminate x_2 from equation 3:

$$\begin{aligned} x_1 + 2x_2 + x_3 &= 0, \\ -4x_2 + x_3 &= 4, \\ -2x_3 &= 12. \end{aligned}$$

$$A^{(3)} = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 0 & -2 \end{pmatrix} \quad \mathbf{b}^{(3)} = \begin{pmatrix} 0 \\ 4 \\ 12 \end{pmatrix}, \quad m_{32} = -2.$$

Now the system is upper triangular, and back substitution gives $x_1 = 11$, $x_2 = -\frac{5}{2}$, $x_3 = -6$.

We can write the general algorithm as follows.

Algorithm 3.1: Gaussian elimination

Let $A^{(1)} = A$ and $\mathbf{b}^{(1)} = \mathbf{b}$. Then for each k from 1 to $n-1$, compute a new matrix $A^{(k+1)}$ and right-hand side $\mathbf{b}^{(k+1)}$ by the following procedure:

1. Define the row multipliers

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad i = k+1, \ldots, n.$$

2. Use these to remove the unknown x_k from equations $k+1$ to n, leaving

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)}, \quad b_i^{(k+1)} = b_i^{(k)} - m_{ik} b_k^{(k)}, \quad i, j = k+1, \ldots, n.$$

The final matrix $A^{(n)} = U$ will then be upper triangular.

This procedure works provided that the **pivots** $a_{kk}^{(k)}$ are non-zero for every k. We will see how to remove this restriction (and avoid a more subtle loss of accuracy) using **pivoting** below.

What about the computational cost of Gaussian elimination? At stage k, we compute $n-k$ multipliers, and then update the $(n-k) \times (n-k)$ block of entries below and to the right of the pivot, which costs one multiplication and one subtraction per entry. So stage k costs about

$2(n-k)^2$ operations, and in total

$$N \approx \sum_{k=1}^{n-1} 2(n-k)^2 = 2\sum_{j=1}^{n-1} j^2 \approx \tfrac{2}{3}n^3.$$

(A careful count, including the divisions, gives exactly $N = \frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n$.) So the number of operations required to find U is $\mathcal{O}(n^3)$: doubling n multiplies the cost by about 8. This is much more than the $\mathcal{O}(n^2)$ cost of the substitution step.

> **Note**
>
> This is the same big-O notation as in Chapter 2, but now describing what happens as $n \to \infty$ rather than as $h \to 0$: $N = \mathcal{O}(n^3)$ means that $N \le Mn^3$ for some constant M and all sufficiently large n.

> **Note**
>
> In theory, matrix multiplication (and hence inversion) can be done faster than $\mathcal{O}(n^3)$: the best known bound (as of 2024) is about $\mathcal{O}(n^{2.371})$, and it is an open question whether $\mathcal{O}(n^{2+\epsilon})$ is possible for every $\epsilon > 0$. However, these algorithms are impractical for real-world use due to massive constant factors and implementation overhead.

Example 3.4: MATLAB demonstration: cubic cost of solving a linear system

```
for n = [1000 2000 4000 8000]
    A = rand(n);  b = rand(n, 1);
    tic;  x = A \ b;  t = toc;
    fprintf('n = %4d: time = %.3f s\n', n, t);
end
```

For large enough n, each doubling of n should increase the time by a factor of roughly 8 (on a multi-core computer, the scaling is less clean).

3.4 LU decomposition

In Gaussian elimination, both the final matrix U and the multipliers m_{ik} are determined solely by A, and do not depend on $\mathbf{b}$. In fact, each stage of Gaussian elimination is equivalent to left-multiplying by a lower triangular matrix $F^{(k)}$, which is the identity matrix except for the entries $-m_{ik}$ below the diagonal in column k.

Example 3.5

You can check in the earlier example that

$$F^{(1)}A = \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 1 & -2 & 2 \\ 2 & 12 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 8 & -4 \end{pmatrix} = A^{(2)},$$

and

$$F^{(2)}A^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 8 & -4 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 0 & -2 \end{pmatrix} = A^{(3)} = U.$$

It follows that

$$U = F^{(n-1)}F^{(n-2)} \cdots F^{(1)}A \quad \Longrightarrow \quad A = (F^{(1)})^{-1}(F^{(2)})^{-1} \cdots (F^{(n-1)})^{-1}U.$$

Each $F^{(k)}$ is inverted simply by flipping the signs of the multipliers (adding rows instead of subtracting them). Moreover, multiplying these inverses together just places all of the multipliers into a single lower triangular matrix:

$$(F^{(1)})^{-1}(F^{(2)})^{-1} \cdots (F^{(n-1)})^{-1} = \begin{pmatrix} 1 & 0 & \cdots & \cdots & \cdots & 0 \\ m_{2,1} & 1 & \ddots & & & \vdots \\ m_{3,1} & m_{3,2} & 1 & \ddots & & \vdots \\ m_{4,1} & m_{4,2} & m_{4,3} & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & & 1 & 0 \\ m_{n,1} & m_{n,2} & m_{n,3} & \cdots & m_{n,n-1} & 1 \end{pmatrix} := L.$$

Thus we have established the following result.

Theorem 3.1: LU decomposition

Let U be the upper triangular matrix from Gaussian elimination of A (without pivoting), and let L be the unit lower triangular matrix of multipliers above. Then

$$A = LU.$$

Note

Unit lower triangular means that there are all 1's on the diagonal. With this normalisation, the LU decomposition of a non-singular matrix is unique (when it exists).

The theorem above says that Gaussian elimination is equivalent to factorising A as the product

of a lower triangular and an upper triangular matrix. This is not at all obvious from the algorithm!

The system $A\mathbf{x} = \mathbf{b}$ becomes $LU\mathbf{x} = \mathbf{b}$, which we can readily solve by setting $U\mathbf{x} = \mathbf{y}$. We first solve $L\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$, then $U\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$. Both are triangular systems.

Moreover, if we want to solve several systems $A\mathbf{x} = \mathbf{b}$ with different $\mathbf{b}$ but the same matrix, we just need to compute L and U once. This saves time because, although the initial LU factorisation takes $\mathcal{O}(n^3)$ operations, each subsequent solve takes only $\mathcal{O}(n^2)$. This is exactly the situation in the backward Euler method for the heat equation, where the same matrix appears at every time step.

i Note

This matrix factorisation viewpoint dates only from the 1940s, and LU decomposition was introduced by Alan Turing in a 1948 paper (*Q. J. Mechanics Appl. Mat.* **1**, 287). Other common factorisations used in numerical linear algebra include *QR* and *Cholesky* (for symmetric positive definite matrices), which are beyond the scope of this course.

Example 3.6

Solve our earlier example by LU decomposition.

$$\begin{pmatrix} 1 & 2 & 1 \\ 1 & -2 & 2 \\ 2 & 12 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \\ 4 \end{pmatrix}.$$

We apply Gaussian elimination as before, but ignore $\mathbf{b}$ (for now), leading to

$$U = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 0 & -2 \end{pmatrix}.$$

As we apply the elimination, we record the multipliers so as to construct the matrix

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & -2 & 1 \end{pmatrix}.$$

Thus we have the factorisation/decomposition

$$\begin{pmatrix} 1 & 2 & 1 \\ 1 & -2 & 2 \\ 2 & 12 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & -2 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 0 & -2 \end{pmatrix}.$$

With the matrices L and U, we can readily solve for any right-hand side $\mathbf{b}$. We illustrate for our particular $\mathbf{b}$. Firstly, solve $L\mathbf{y} = \mathbf{b}$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & -2 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \\ 4 \end{pmatrix}$$

$$\implies y_1 = 0,\ y_2 = 4 - y_1 = 4,\ y_3 = 4 - 2y_1 + 2y_2 = 12.$$

Notice that $\mathbf{y}$ is the right-hand side $\mathbf{b}^{(3)}$ constructed earlier. Then, solve $U\mathbf{x} = \mathbf{y}$:

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & -4 & 1 \\ 0 & 0 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \\ 12 \end{pmatrix}$$

$$\implies x_3 = -6,\ x_2 = -\tfrac{1}{4}(4 - x_3) = -\tfrac{5}{2},\ x_1 = -2x_2 - x_3 = 11.$$

Note

Notice that we never need to compute A^{-1}. Computing the inverse costs about $2n^3$ operations (three times as much as the LU decomposition) and is generally less accurate, so in practice we always solve linear systems by factorisation. In MATLAB, use `x = A\b` rather than `x = inv(A)*b`.

Example 3.7: MATLAB demonstration: LU decomposition and reusing the factorisation

```
A = [1 2 1; 1 -2 2; 2 12 -2];
[L, U, P] = lu(A)       % MATLAB pivots (see below), so P is a permutation matrix

% Reusing the factorisation for many right-hand sides
n = 2000;  A = rand(n);  B = rand(n, 100);     % 100 right-hand sides
tic
for j = 1:100
    x = A \ B(:, j);                           % factorises A every time
end
t1 = toc;
tic
[L, U, P] = lu(A);                             % factorise once...
for j = 1:100
    x = U \ (L \ (P*B(:, j)));                 % ...then two triangular solves each time
end
t2 = toc;
fprintf('Backslash each time: %.2f s,  LU once then reuse: %.2f s\n', t1, t2);
```

Because MATLAB swaps rows to choose good pivots, its `L` and `U` for the 3×3 example differ from the ones we computed by hand.

3.5 Pivoting

Gaussian elimination fails if a pivot $a_{kk}^{(k)}$ is zero, as happens immediately for

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix},$$

even though A is non-singular. More subtly, a *small* pivot can destroy the accuracy of the solution in floating-point arithmetic.

Example 3.8: A small pivot.

Let ϵ be a small positive number and consider

$$\begin{aligned} \epsilon x_1 + x_2 &= 1, \\ x_1 + x_2 &= 2, \end{aligned}$$

whose exact solution is $x_1 = \frac{1}{1-\epsilon} \approx 1$, $x_2 = \frac{1-2\epsilon}{1-\epsilon} \approx 1$.

Without pivoting, the multiplier is $m_{21} = 1/\epsilon$, so the second equation becomes

$$\Big(1 - \frac{1}{\epsilon}\Big)x_2 = 2 - \frac{1}{\epsilon}.$$

If $\epsilon = 10^{-20}$ (smaller than ϵ_{M}), then in double precision $\mathrm{fl}(1 - 1/\epsilon) = \mathrm{fl}(2 - 1/\epsilon) = -1/\epsilon$, so we get $x_2 = 1$, which is fine. But back substitution then gives $x_1 = (1 - x_2)/\epsilon = 0$, which is completely wrong!
With the rows swapped, the multiplier is $m_{21} = \epsilon$, and the second equation becomes $(1 - \epsilon)x_2 = 1 - 2\epsilon$. In double precision this gives $x_2 = 1$, and then $x_1 = 2 - x_2 = 1$, which is correct to machine precision.

The problem is the huge multiplier $1/\epsilon$, which swamps the information in the other entries (this is another example of loss of significance from Chapter 1). The fix is to swap rows so that the multipliers stay small.

Algorithm 3.2: Partial pivoting

At each stage k of Gaussian elimination, before computing the multipliers, find the row $p \geq k$ for which $|a_{pk}^{(k)}|$ is largest, and swap rows k and p (in both $A^{(k)}$ and $\mathbf{b}^{(k)}$). Then all of the multipliers satisfy $|m_{ik}| \leq 1$.

The row swaps can be recorded in a **permutation matrix** P (the identity matrix with its rows reordered), and Gaussian elimination with partial pivoting then produces the factorisation

$$PA = LU.$$

To solve $A\mathbf{x} = \mathbf{b}$, we solve $L\mathbf{y} = P\mathbf{b}$ and then $U\mathbf{x} = \mathbf{y}$. Partial pivoting costs very little extra, and in practice it makes Gaussian elimination reliable for almost all matrices. MATLAB's `lu` and backslash use partial pivoting automatically.

Example 3.9: MATLAB demonstration: the danger of small pivots

```
ep = 1e-20;
A = [ep 1; 1 1];  b = [1; 2];

% Gaussian elimination WITHOUT pivoting
m   = A(2,1)/A(1,1);
u22 = A(2,2) - m*A(1,2);
y2  = b(2) - m*b(1);
x2  = y2/u22;
x1  = (b(1) - A(1,2)*x2)/A(1,1);
disp([x1; x2])          % gives [0; 1]: x1 is completely wrong

% Backslash uses partial pivoting
disp(A \ b)             % gives [1; 1]
```

3.6 Vector norms

To measure the error when the solution is a vector, as opposed to a scalar, we usually summarise the error in a single number called a **norm**. A **norm** effectively gives us a way to define a notion of distance in higher dimensions.

A **vector norm** on $\mathbb{R}^n$ is a real-valued function that satisfies: 1. $\|\mathbf{x}+\mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ for every $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ (**triangle inequality**). 2. $\|\alpha\mathbf{x}\| = |\alpha|\,\|\mathbf{x}\|$ for every $\mathbf{x} \in \mathbb{R}^n$ and every $\alpha \in \mathbb{R}$. 3. $\|\mathbf{x}\| \geq 0$ for every $\mathbf{x} \in \mathbb{R}^n$, and $\|\mathbf{x}\| = 0$ implies $\mathbf{x} = 0$.

Example 3.10: There are three common examples:

1. The ℓ_2**-norm**

$$\|\mathbf{x}\|_2 := \sqrt{\sum_{k=1}^{n} x_k^2} = \sqrt{\mathbf{x}^\top \mathbf{x}}.$$

   This is just the usual Euclidean length of $\mathbf{x}$.

2. The ℓ_1**-norm**

$$\|\mathbf{x}\|_1 := \sum_{k=1}^{n} |x_k|.$$

   This is sometimes known as the **taxicab** or **Manhattan** norm, because it corresponds to the distance that a taxi has to drive on a rectangular grid of streets to get to $\mathbf{x} \in \mathbb{R}^2$.

3. The ℓ_∞**-norm**

$$\|\mathbf{x}\|_\infty := \max_{k=1,\ldots,n} |x_k|.$$

This is sometimes known as the **maximum** norm.

The norms in the example above are all special cases of the ℓ_p-norm,

$$\|\mathbf{x}\|_p = \left(\sum_{k=1}^{n} |x_k|^p \right)^{1/p},$$

which is a norm for any real number $p \geq 1$. Increasing p means that more and more emphasis is given to the maximum element $|x_k|$.

Example 3.11: Consider the vectors $\mathbf{a} = (1, -2, 3)^\top$, $\mathbf{b} = (2, 0, -1)^\top$, and $\mathbf{c} = (0, 1, 4)^\top$.

The ℓ_1-, ℓ_2-, and ℓ_∞-norms are:

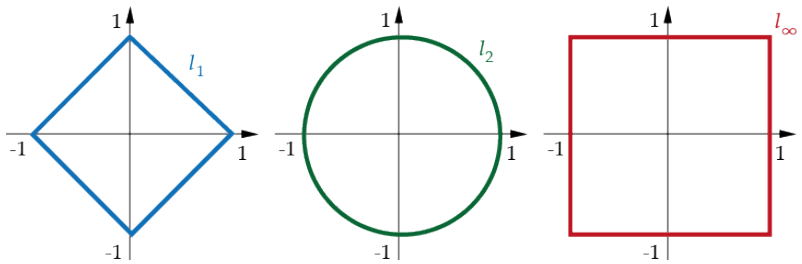
$$\begin{aligned} \|\mathbf{a}\|_1 &= 1 + 2 + 3 = 6 \\ \|\mathbf{b}\|_1 &= 2 + 0 + 1 = 3 \\ \|\mathbf{c}\|_1 &= 0 + 1 + 4 = 5 \end{aligned}$$

$$\begin{aligned} \|\mathbf{a}\|_2 &= \sqrt{1 + 4 + 9} \approx 3.74 \\ \|\mathbf{b}\|_2 &= \sqrt{4 + 0 + 1} \approx 2.24 \\ \|\mathbf{c}\|_2 &= \sqrt{0 + 1 + 16} \approx 4.12 \end{aligned}$$

$$\begin{aligned} \|\mathbf{a}\|_\infty &= \max\{1, 2, 3\} = 3 \\ \|\mathbf{b}\|_\infty &= \max\{2, 0, 1\} = 2 \\ \|\mathbf{c}\|_\infty &= \max\{0, 1, 4\} = 4 \end{aligned}$$

Notice that, for a single vector $\mathbf{x}$, the norms satisfy the ordering $\|\mathbf{x}\|_1 \geq \|\mathbf{x}\|_2 \geq \|\mathbf{x}\|_\infty$, but that vectors may be ordered differently by different norms.

Example 3.12: Sketch the 'unit circles' $\{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\|_p = 1\}$ for $p = 1, 2, \infty$.



3.7 Matrix norms

We also use norms to measure the "size" of matrices. Since the set $\mathbb{R}^{n\times n}$ of $n \times n$ matrices with real entries is a vector space, we could just use a vector norm on this space. But usually we add an additional axiom.

A **matrix norm** is a real-valued function $\|\cdot\|$ on $\mathbb{R}^{n\times n}$ that satisfies:

1. $\|A + B\| \leq \|A\| + \|B\|$ for every $A, B \in \mathbb{R}^{n\times n}$.
2. $\|\alpha A\| = |\alpha| \, \|A\|$ for every $A \in \mathbb{R}^{n\times n}$ and every $\alpha \in \mathbb{R}$.
3. $\|A\| \geq 0$ for every $A \in \mathbb{R}^{n\times n}$ and $\|A\| = 0$ implies $A = 0$.
4. $\|AB\| \leq \|A\|\|B\|$ for every $A, B \in \mathbb{R}^{n\times n}$ (**consistency**).

> **Note**
>
> We usually want this additional axiom because matrices are more than just vectors. Some books call this a **submultiplicative norm** and define a "matrix norm" to satisfy just the first three properties, perhaps because (4) only works for square matrices.
> Treating a matrix as a vector with n^2 components and taking its ℓ_2-norm gives the **Frobenius norm** $\|A\|_F = \left(\sum_{i,j} a_{ij}^2\right)^{1/2}$. This is a matrix norm, but it is not induced by any vector norm in the sense defined below (for example, $\|I\|_F = \sqrt{n}$, whereas every induced norm has $\|I\| = 1$).

The most important matrix norms are so-called **induced** or **operator** norms. Remember that A is a linear map on $\mathbb{R}^n$, meaning that it maps every vector to another vector. So we can measure the size of A by how much it can stretch vectors with respect to a given vector norm. Specifically, if $\|\cdot\|_p$ is a vector norm, then the **induced** norm is defined as

$$\|A\|_p := \sup_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|_p}{\|\mathbf{x}\|_p} = \max_{\|\mathbf{x}\|_p=1} \|A\mathbf{x}\|_p.$$

To see that the two definitions here are equivalent, use the fact that $\|\cdot\|_p$ is a vector norm. So by property (2) we have

$$\sup_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|_p}{\|\mathbf{x}\|_p} = \sup_{\mathbf{x}\neq\mathbf{0}} \left\| A\frac{\mathbf{x}}{\|\mathbf{x}\|_p} \right\|_p = \sup_{\|\mathbf{y}\|_p=1} \|A\mathbf{y}\|_p = \max_{\|\mathbf{y}\|_p=1} \|A\mathbf{y}\|_p.$$

> **Note**
>
> Usually we use the same notation for the induced matrix norm as for the original vector norm. The meaning should be clear from the context.

Example 3.13

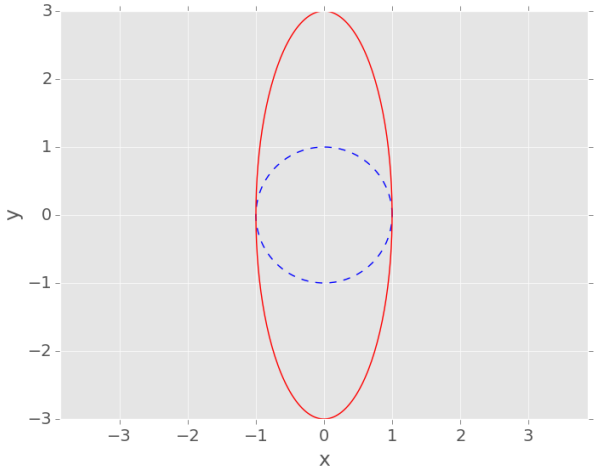
Let

$$A = \begin{pmatrix} 0 & 1 \\ 3 & 0 \end{pmatrix}.$$

In the ℓ_2-norm, a unit vector in $\mathbb{R}^2$ has the form $\mathbf{x} = (\cos\theta, \sin\theta)^\top$, so the image of the unit circle is

$$A\mathbf{x} = \begin{pmatrix} \sin\theta \\ 3\cos\theta \end{pmatrix}.$$

This is illustrated below:



The induced matrix norm is the maximum stretching of this unit circle, which is

$$\|A\|_2 = \max_{\|\mathbf{x}\|_2=1} \|A\mathbf{x}\|_2 = \max_\theta \left(\sin^2\theta + 9\cos^2\theta\right)^{1/2} = \max_\theta \left(1 + 8\cos^2\theta\right)^{1/2} = 3.$$

Theorem 3.2: Induced norms are matrix norms

The induced norm corresponding to any vector norm is a matrix norm, and the two norms satisfy $\|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\|$ for any matrix $A \in \mathbb{R}^{n\times n}$ and any vector $\mathbf{x} \in \mathbb{R}^n$.

Proof (non-examinable)

Properties (1)-(3) follow from the fact that the vector norm satisfies the corresponding properties. To show (4), note that, by the definition above, we have for any vector $\mathbf{y} \in \mathbb{R}^n$ that

$$\|A\| \geq \frac{\|A\mathbf{y}\|}{\|\mathbf{y}\|} \quad \Longrightarrow \quad \|A\mathbf{y}\| \leq \|A\|\|\mathbf{y}\|.$$

Taking $\mathbf{y} = B\mathbf{x}$ for some $\mathbf{x}$ with $\|\mathbf{x}\| = 1$, we get

$$\|AB\mathbf{x}\| \leq \|A\|\|B\mathbf{x}\| \leq \|A\|\|B\|.$$

This holds in particular for the vector $\mathbf{x}$ that maximises $\|AB\mathbf{x}\|$, so

$$\|AB\| = \max_{\|\mathbf{x}\|=1} \|AB\mathbf{x}\| \leq \|A\|\|B\|.$$

It is cumbersome to compute the induced norms from their definition, but fortunately there are some very useful alternative formulae.

Theorem 3.3: Matrix norms induced by ℓ_1 and ℓ_∞

The matrix norms induced by the ℓ_1-norm and ℓ_∞-norm satisfy

$$\|A\|_1 = \max_{j=1,\ldots,n} \sum_{i=1}^{n} |a_{ij}|, \quad \text{(maximum column sum)}$$

$$\|A\|_\infty = \max_{i=1,\ldots,n} \sum_{j=1}^{n} |a_{ij}|. \quad \text{(maximum row sum)}$$

Proof (non-examinable)

We will prove the result for the ℓ_1-norm as an illustration of the method:

$$\|A\mathbf{x}\|_1 = \sum_{i=1}^{n} \left| \sum_{j=1}^{n} a_{ij}x_j \right| \leq \sum_{i=1}^{n} \sum_{j=1}^{n} |a_{ij}|\,|x_j| = \sum_{j=1}^{n} |x_j| \sum_{i=1}^{n} |a_{ij}|.$$

If we let

$$c = \max_{j=1,\dots,n} \sum_{i=1}^{n} |a_{ij}|,$$

then

$$\|A\mathbf{x}\|_1 \leq c\|\mathbf{x}\|_1 \quad \Longrightarrow \quad \|A\|_1 \leq c.$$

Now let m be the column where the maximum sum is attained. If we choose $\mathbf{y}$ to be the vector with components $y_k = \delta_{km}$, then we have $\|A\mathbf{y}\|_1 = c$. Since $\|\mathbf{y}\|_1 = 1$, we must have that

$$\max_{\|\mathbf{x}\|_1=1} \|A\mathbf{x}\|_1 \geq \|A\mathbf{y}\|_1 = c \quad \Longrightarrow \quad \|A\|_1 \geq c.$$

The only way to satisfy both inequalities is if $\|A\|_1 = c$.

Example 3.14

For the matrix

$$A = \begin{pmatrix} -7 & 3 & -1 \\ 2 & 4 & 5 \\ -4 & 6 & 0 \end{pmatrix}$$

we have

$$\|A\|_1 = \max\{13, 13, 6\} = 13, \qquad \|A\|_\infty = \max\{11, 11, 10\} = 11.$$

What about the matrix norm induced by the ℓ_2-norm? This turns out to be related to the eigenvalues of A. Recall that $\lambda \in \mathbb{C}$ is an **eigenvalue** of A with associated **eigenvector** $\mathbf{u}$ if

$$A\mathbf{u} = \lambda\mathbf{u}.$$

We define the **spectral radius** $\rho(A)$ of A to be the maximum $|\lambda|$ over all eigenvalues λ of A.

Theorem 3.4: Spectral norm

The matrix norm induced by the ℓ_2-norm satisfies

$$\|A\|_2 = \sqrt{\rho(A^\top A)}.$$

As a result of the theorem above, this norm is sometimes known as the **spectral norm**.

Example 3.15

For our matrix

$$A = \begin{pmatrix} 0 & 1 \\ 3 & 0 \end{pmatrix},$$

we have

$$A^\top A = \begin{pmatrix} 0 & 3 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 3 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 0 \\ 0 & 1 \end{pmatrix}.$$

We see that the eigenvalues of $A^\top A$ are $\lambda = 1, 9$, so $\|A\|_2 = \sqrt{9} = 3$ (as we calculated earlier).

i Proof (non-examinable)

We want to show that

$$\max_{\|\mathbf{x}\|_2=1} \|A\mathbf{x}\|_2 = \max\{\sqrt{|\lambda|} \,:\, \lambda \text{ eigenvalue of } A^\top A\}.$$

For A real, $A^\top A$ is symmetric, so has real eigenvalues $\lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$ with corresponding orthonormal eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_n$ in $\mathbb{R}^n$. (Orthonormal means that $\mathbf{u}_j^\top \mathbf{u}_k = \delta_{jk}$.) Note also that all of the eigenvalues are non-negative, since

$$A^\top A\mathbf{u}_1 = \lambda_1 \mathbf{u}_1 \quad \implies \lambda_1 = \frac{\mathbf{u}_1^\top A^\top A \mathbf{u}_1}{\mathbf{u}_1^\top \mathbf{u}_1} = \frac{\|A\mathbf{u}_1\|_2^2}{\|\mathbf{u}_1\|_2^2} \geq 0.$$

So we want to show that $\|A\|_2 = \sqrt{\lambda_n}$. The eigenvectors form a basis, so every vector $\mathbf{x} \in \mathbb{R}^n$ can be expressed as a linear combination $\mathbf{x} = \sum_{k=1}^n \alpha_k \mathbf{u}_k$. Therefore

$$\|A\mathbf{x}\|_2^2 = \mathbf{x}^\top A^\top A \mathbf{x} = \mathbf{x}^\top \sum_{k=1}^n \alpha_k \lambda_k \mathbf{u}_k = \sum_{j=1}^n \alpha_j \mathbf{u}_j^\top \sum_{k=1}^n \alpha_k \lambda_k \mathbf{u}_k = \sum_{k=1}^n \alpha_k^2 \lambda_k,$$

where the last step uses orthonormality of the $\mathbf{u}_k$. It follows that

$$\|A\mathbf{x}\|_2^2 \leq \lambda_n \sum_{k=1}^n \alpha_k^2.$$

But if $\|\mathbf{x}\|_2 = 1$, then $\|\mathbf{x}\|_2^2 = \sum_{k=1}^n \alpha_k^2 = 1$, so $\|A\mathbf{x}\|_2^2 \leq \lambda_n$. To show that the maximum of $\|A\mathbf{x}\|_2^2$ is equal to λ_n, we can choose $\mathbf{x}$ to be the corresponding eigenvector $\mathbf{x} = \mathbf{u}_n$. In that case, $\alpha_1 = \ldots = \alpha_{n-1} = 0$ and $\alpha_n = 1$, so $\|A\mathbf{x}\|_2^2 = \lambda_n$.

3.8 Conditioning

Some linear systems are inherently more difficult to solve than others, because the solution is sensitive to small perturbations in the input. We will examine how to quantify this sensitivity,

and what it means for the accuracy we can expect from our methods.

Example 3.16

Consider the linear system

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \implies \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

If we add a small rounding error $0 < \delta \ll 1$ to the data b_1 then

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1+\delta \\ 1 \end{pmatrix} \implies \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \delta \\ 1 \end{pmatrix}.$$

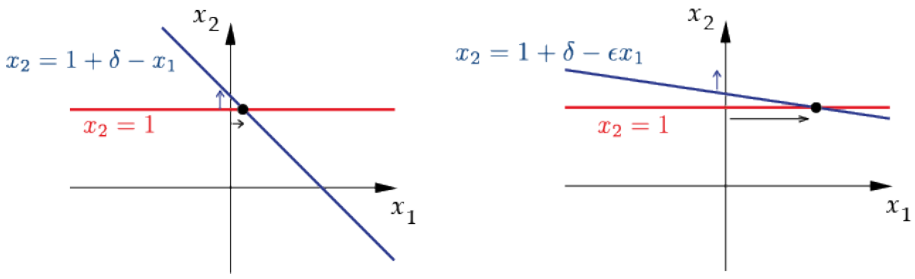
The solution is within rounding error of the true solution, so the system is called **well conditioned**.

Example 3.17

Now let $\epsilon \ll 1$ be a fixed positive number, and consider the linear system

$$\begin{pmatrix} \epsilon & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1+\delta \\ 1 \end{pmatrix} \implies \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \delta/\epsilon \\ 1 \end{pmatrix}.$$

The true solution is still $(0,1)^\top$, but if the error δ is as big as the matrix entry ϵ, then the solution for x_1 will be completely wrong. This system is much more sensitive to errors in $\mathbf{b}$, so is called **ill-conditioned**.
Graphically, this system (right) is more sensitive to δ than the first system (left) because the two lines are closer to parallel:



To measure the **conditioning** of a linear system, suppose that an error $\delta\mathbf{b}$ in the right-hand side causes an error $\delta\mathbf{x}$ in the solution, so that $A(\mathbf{x}+\delta\mathbf{x}) = \mathbf{b}+\delta\mathbf{b}$ and hence $\delta\mathbf{x} = A^{-1}\delta\mathbf{b}$.

Then the ratio of the relative errors in the output ($\mathbf{x}$) and the input ($\mathbf{b}$) satisfies

$$\begin{aligned}\frac{\text{relative error in } \mathbf{x}}{\text{relative error in } \mathbf{b}} &= \frac{\|\delta\mathbf{x}\|/\|\mathbf{x}\|}{\|\delta\mathbf{b}\|/\|\mathbf{b}\|} = \left(\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|}\right)\left(\frac{\|\mathbf{b}\|}{\|\delta\mathbf{b}\|}\right) \\ &= \left(\frac{\|A^{-1}\delta\mathbf{b}\|}{\|\mathbf{x}\|}\right)\left(\frac{\|\mathbf{b}\|}{\|\delta\mathbf{b}\|}\right) \\ &\leq \frac{\|A^{-1}\|\|\delta\mathbf{b}\|}{\|\mathbf{x}\|}\left(\frac{\|\mathbf{b}\|}{\|\delta\mathbf{b}\|}\right) \\ &= \frac{\|A^{-1}\|\|\mathbf{b}\|}{\|\mathbf{x}\|} = \frac{\|A^{-1}\|\|A\mathbf{x}\|}{\|\mathbf{x}\|} \\ &\leq \|A^{-1}\|\|A\|.\end{aligned}$$

We define the **condition number** of a matrix A in some induced norm $\|\cdot\|_*$ to be

$$\kappa_*(A) = \|A^{-1}\|_*\|A\|_*.$$

If $\kappa_*(A)$ is large, then the solution will be sensitive to errors in $\mathbf{b}$, at least for some $\mathbf{b}$. A large condition number means that the matrix is close to being non-invertible (i.e. two rows are close to being linearly dependent).

> **Note**
>
> This is a "worst case" amplification of the error by a given matrix. The actual result will depend on $\delta\mathbf{b}$ (which we usually don't know if it arises from previous rounding error).

Note that $\det(A)$ will tell you whether a matrix is singular or not, but not whether it is ill-conditioned. Since $\det(\alpha A) = \alpha^n \det(A)$, the determinant can be made arbitrarily large or small by scaling (which does not change the condition number). For instance, the matrix

$$\begin{pmatrix} 10^{-50} & 0 \\ 0 & 10^{-50} \end{pmatrix}$$

has tiny determinant but is well-conditioned.

Example 3.18

Return to our earlier examples and consider the condition numbers in the 1-norm. We have (assuming $0 < \epsilon \ll 1$) that

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \implies A^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \implies \|A\|_1 = \|A^{-1}\|_1 = 2 \implies \kappa_1(A) = 4,$$

$$B = \begin{pmatrix} \epsilon & 1 \\ 0 & 1 \end{pmatrix} \implies B^{-1} = \frac{1}{\epsilon}\begin{pmatrix} 1 & -1 \\ 0 & \epsilon \end{pmatrix}$$

$$\implies \|B\|_1 = 2,\ \|B^{-1}\|_1 = \frac{1+\epsilon}{\epsilon} \implies \kappa_1(B) = \frac{2(1+\epsilon)}{\epsilon}.$$

For matrix B, $\kappa_1(B) \to \infty$ as $\epsilon \to 0$, showing that the matrix B is ill-conditioned.

Example 3.19

The **Hilbert matrix** H_n is the $n \times n$ symmetric matrix with entries

$$(h_n)_{ij} = \frac{1}{i+j-1}.$$

These matrices are notoriously ill-conditioned. For example, $\kappa_2(H_5) \approx 4.8 \times 10^5$, and $\kappa_2(H_{20}) \approx 2.5 \times 10^{28}$. Solving an associated linear system in floating-point arithmetic would be hopeless.

What does this mean in practice? Even a perfectly implemented solver (such as Gaussian elimination with partial pivoting) introduces rounding errors of relative size about ϵ_{M} in the data. So, as a rule of thumb, the computed solution $\hat{\mathbf{x}}$ satisfies

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim \kappa(A)\,\epsilon_{\mathrm{M}}.$$

In other words, we can expect to lose about $\log_{10} \kappa(A)$ significant digits. For the Hilbert matrix H_{20}, $\kappa_2(H_{20}) \approx 10^{28}$ is far larger than $1/\epsilon_{\mathrm{M}} \approx 10^{16}$, so none of the computed digits can be trusted.

i Note

The Vandermonde matrices from Chapter 2 are another example: for equally spaced nodes, their condition number grows exponentially with n. This is why we do not compute high-degree interpolating polynomials by solving the Vandermonde system.

A practical limitation of the condition number is that computing it exactly requires A^{-1}. However, we can cheaply bound $\|A^{-1}\|$ from below: for any $\mathbf{b}$, solving $A\mathbf{x} = \mathbf{b}$ (using the LU decomposition we already have) gives

$$\|A^{-1}\| \geq \frac{\|A^{-1}\mathbf{b}\|}{\|\mathbf{b}\|} = \frac{\|\mathbf{x}\|}{\|\mathbf{b}\|}.$$

Choosing $\mathbf{b}$ cleverly gives good estimates of $\kappa(A)$; this is what MATLAB's `condest` does, while `cond` computes $\kappa_2(A)$ exactly (at greater cost).

Example 3.20: MATLAB demonstration: conditioning and accuracy

```
ns = 2:13;  kappa = zeros(size(ns));  err = kappa;
for j = 1:length(ns)
    n = ns(j);  H = hilb(n);
    x = ones(n, 1);  b = H*x;                  % so the exact solution is all ones
    kappa(j) = cond(H);
    err(j) = norm(H\b - x)/norm(x);
end
figure
semilogy(ns, kappa, 'o-', ns, err, 's-', ns, kappa*eps/2, '--', ...
    'LineWidth', 2.5, 'MarkerSize', 8)
legend('condition number \kappa_2(H_n)', 'relative error in x', ...
    '\kappa_2(H_n) \epsilon_M', 'Location', 'northwest')
xlabel('n'), set(gca, 'FontSize', 18), grid on
```

The relative error grows roughly in line with (and usually stays below) $\kappa_2(H_n)\,\epsilon_{\mathrm{M}}$. For the largest n, MATLAB warns that the matrix is close to singular.

3.9 Iterative methods

For large systems, the $\mathcal{O}(n^3)$ cost of Gaussian elimination is prohibitive. Fortunately, many such systems that arise in practice are **sparse**, meaning that most of the entries of the matrix A are zero. For example, the matrix in the backward Euler method for the heat equation (Chapter 6) is tridiagonal: each row has at most three non-zero entries. With $n = 10^6$ grid points, storing this matrix in full would require 10^{12} numbers (8 terabytes), but it has only about 3×10^6 non-zero entries.

For a tridiagonal matrix, Gaussian elimination can be carried out cheaply. But for the sparse matrices arising from two- or three-dimensional problems, elimination "fills in" many of the zero entries and becomes very expensive. In this case, we can often do much better than $\mathcal{O}(n^3)$ using **iterative** algorithms, whose main cost is computing matrix-vector products $A\mathbf{d}$, which only involve the non-zero entries.

In this course, we will only study algorithms for symmetric positive definite matrices. A matrix A is called **symmetric positive definite** (or **SPD**) if it is symmetric and $\mathbf{x}^\top A\mathbf{x} > 0$ for every vector $\mathbf{x} \neq 0$.

Note

Recall that a symmetric matrix has real eigenvalues. It is positive definite iff all of its eigenvalues are positive.

Example 3.21

Show that the following matrix is SPD:

$$A = \begin{pmatrix} 3 & 1 & -1 \\ 1 & 4 & 2 \\ -1 & 2 & 5 \end{pmatrix}.$$

The matrix is clearly symmetric, and its eigenvalues are approximately 1.29, 4.14 and 6.57 (in MATLAB, `eig(A)`), which are all positive, so A is SPD.
Alternatively, with $\mathbf{x} = (x_1, x_2, x_3)^\top$, we can complete the square:

$$\begin{aligned} \mathbf{x}^\top A\mathbf{x} &= 3x_1^2 + 4x_2^2 + 5x_3^2 + 2x_1x_2 + 4x_2x_3 - 2x_1x_3 \\ &= x_1^2 + x_2^2 + 2x_3^2 + (x_1 + x_2)^2 + (x_1 - x_3)^2 + 2(x_2 + x_3)^2, \end{aligned}$$

which is positive for any non-zero vector $\mathbf{x} \in \mathbb{R}^3$.

If A is SPD, then solving $A\mathbf{x} = \mathbf{b}$ is equivalent to minimising the quadratic functional

$$f : \mathbb{R}^n \to \mathbb{R}, \qquad f(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top \mathbf{x}.$$

When A is SPD, this functional behaves like a U-shaped parabola, and has a unique finite global minimiser $\mathbf{x}_*$ such that $f(\mathbf{x}_*) < f(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x} \neq \mathbf{x}_*$.

To find $\mathbf{x}_*$, we set $\nabla f = \mathbf{0}$. Writing $f(\mathbf{x}) = \frac{1}{2}\sum_{i,j} x_i a_{ij} x_j - \sum_j b_j x_j$ and differentiating with respect to each x_k (using the symmetry $a_{ik} = a_{ki}$ to combine the two terms from the double sum) gives

$$\nabla f = A\mathbf{x} - \mathbf{b},$$

so locating the minimum of $f(\mathbf{x})$ is indeed equivalent to solving $A\mathbf{x} = \mathbf{b}$.

Note

Minimising functions is a vast sub-field of numerical analysis known as **optimisation**. We will only cover this specific case.

A popular class of methods for optimisation are **line search** methods, where at each iteration the search is restricted to a single **search direction** $\mathbf{d}_k$. The iteration takes the form

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k.$$

The **step size** α_k is chosen by minimising $f(\mathbf{x})$ along the line $\mathbf{x} = \mathbf{x}_k + \alpha\mathbf{d}_k$. For our functional above, we have

$$f(\mathbf{x}_k + \alpha\mathbf{d}_k) = (\tfrac{1}{2}\mathbf{d}_k^\top A\mathbf{d}_k)\alpha^2 + \mathbf{d}_k^\top(A\mathbf{x}_k - \mathbf{b})\alpha + \tfrac{1}{2}\mathbf{x}_k^\top A\mathbf{x}_k - \mathbf{b}^\top\mathbf{x}_k.$$

This is a quadratic in α, and the coefficient of α^2 is positive because A is positive definite. It is therefore a U-shaped parabola and achieves its minimum when

$$\frac{\partial f}{\partial \alpha} = \mathbf{d}_k^\top A\mathbf{d}_k\alpha + \mathbf{d}_k^\top(A\mathbf{x}_k - \mathbf{b}) = 0.$$

Defining the **residual** $\mathbf{r}_k := A\mathbf{x}_k - \mathbf{b}$, we see that the desired choice of step size is

$$\alpha_k = -\frac{\mathbf{d}_k^\top \mathbf{r}_k}{\mathbf{d}_k^\top A\mathbf{d}_k}.$$

> **Note**
>
> Many textbooks (and MATLAB) define the residual with the opposite sign, $\mathbf{b} - A\mathbf{x}_k$. This changes some signs in the formulae, but not the algorithms themselves.

Different line search methods differ in how the search direction $\mathbf{d}_k$ is chosen at each iteration. For example, the **method of steepest descent** sets

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k) = -\mathbf{r}_k,$$

where we have remembered the gradient formula above.

Example 3.22

Use the method of steepest descent to solve the system

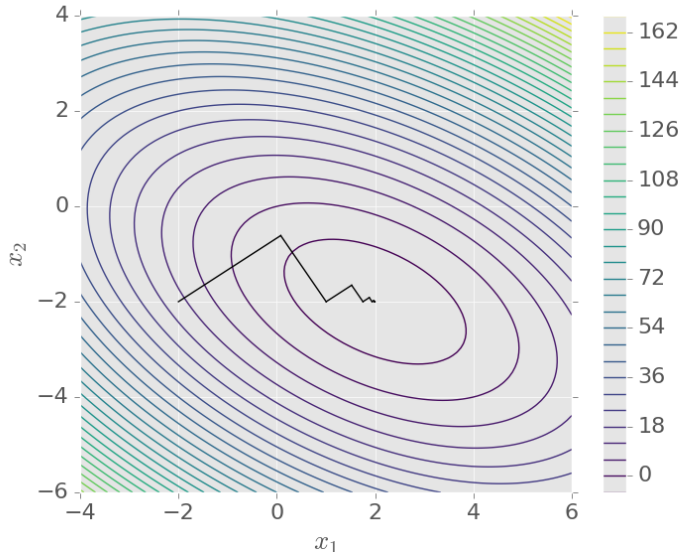
$$\begin{pmatrix} 3 & 2 \\ 2 & 6 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ -8 \end{pmatrix}.$$

Starting from $\mathbf{x}_0 = (-2, -2)^\top$, we get

$$\mathbf{d}_0 = \mathbf{b} - A\mathbf{x}_0 = \begin{pmatrix} 12 \\ 8 \end{pmatrix} \implies \alpha_0 = \frac{\mathbf{d}_0^\top \mathbf{d}_0}{\mathbf{d}_0^\top A\mathbf{d}_0} = \frac{208}{1200}$$

$$\implies \mathbf{x}_1 = \mathbf{x}_0 + \alpha_0\mathbf{d}_0 \approx \begin{pmatrix} 0.08 \\ -0.613 \end{pmatrix}.$$

Continuing the iteration, $\mathbf{x}_k$ proceeds towards the solution $(2, -2)^\top$ as illustrated below. The coloured contours show the value of $f(x_1, x_2)$.



Unfortunately, the method of steepest descent can be slow to converge, because the iterates zig-zag across the valley of f. Its speed depends on the conditioning of A: it can be shown that the error decreases by a factor of about

$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}},$$

at each iteration (measured in a suitable norm). For an ill-conditioned matrix, this factor is very close to 1.

In the **conjugate gradient method**, we still take $\mathbf{d}_0 = -\mathbf{r}_0$, but subsequent search directions $\mathbf{d}_k$ are chosen to be **A-conjugate**, meaning that

$$\mathbf{d}_{k+1}^\top A \mathbf{d}_k = 0.$$

This means that minimisation in one direction does not undo the previous minimisations.

In particular, we construct $\mathbf{d}_{k+1}$ by writing

$$\mathbf{d}_{k+1} = -\mathbf{r}_{k+1} + \beta_k \mathbf{d}_k,$$

then choosing the scalar β_k such that $\mathbf{d}_{k+1}^\top A \mathbf{d}_k = 0$. This gives

$$0 = \big(-\mathbf{r}_{k+1} + \beta_k \mathbf{d}_k \big)^\top A \mathbf{d}_k = -\mathbf{r}_{k+1}^\top A \mathbf{d}_k + \beta_k \mathbf{d}_k^\top A \mathbf{d}_k$$

and hence

$$\beta_k = \frac{\mathbf{r}_{k+1}^\top A \mathbf{d}_k}{\mathbf{d}_k^\top A \mathbf{d}_k}.$$

Thus we get the basic conjugate gradient algorithm.

Algorithm 3.3: Conjugate gradient method

Start with an initial guess $\mathbf{x}_0$ and initial search direction $\mathbf{d}_0 = -\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$. For each $k = 0, 1, \ldots$, do the following:

1. Compute step size
$$\alpha_k = -\frac{\mathbf{d}_k^\top \mathbf{r}_k}{\mathbf{d}_k^\top A\mathbf{d}_k}.$$
2. Compute $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$.
3. Compute residual $\mathbf{r}_{k+1} = A\mathbf{x}_{k+1} - \mathbf{b}$.
4. If $\|\mathbf{r}_{k+1}\| <$ tolerance, output $\mathbf{x}_{k+1}$ and stop.
5. Determine new search direction
$$\mathbf{d}_{k+1} = -\mathbf{r}_{k+1} + \beta_k \mathbf{d}_k \quad \text{where} \quad \beta_k = \frac{\mathbf{r}_{k+1}^\top A\mathbf{d}_k}{\mathbf{d}_k^\top A\mathbf{d}_k}.$$

Example 3.23

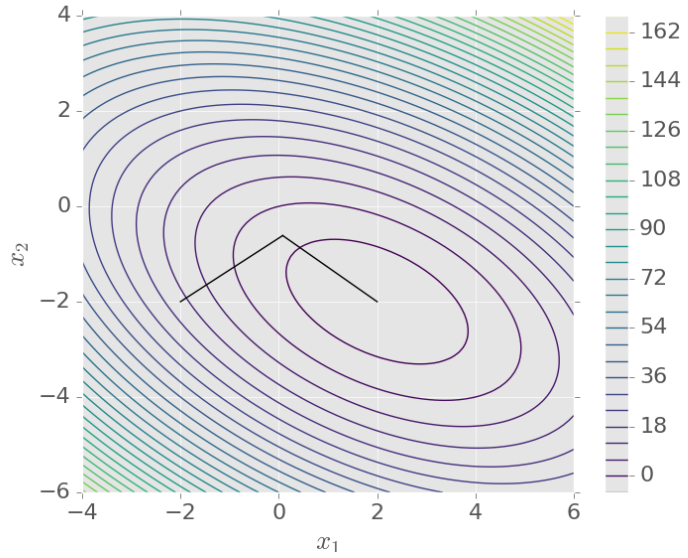
Solve our previous example with the conjugate gradient method.
Starting with $\mathbf{x}_0 = (-2, -2)^\top$, the first step is the same as in steepest descent, giving $\mathbf{x}_1 = (0.08, -0.613)^\top$. But then we take

$$\mathbf{r}_1 = A\mathbf{x}_1 - \mathbf{b} = \begin{pmatrix} -2.99 \\ 4.48 \end{pmatrix}, \quad \beta_0 = \frac{\mathbf{r}_1^\top A\mathbf{d}_0}{\mathbf{d}_0^\top A\mathbf{d}_0} = 0.139, \quad \mathbf{d}_1 = -\mathbf{r}_1 + \beta_0 \mathbf{d}_0 = \begin{pmatrix} 4.66 \\ -3.36 \end{pmatrix}.$$

The second iteration then gives

$$\alpha_1 = -\frac{\mathbf{d}_1^\top \mathbf{r}_1}{\mathbf{d}_1^\top A\mathbf{d}_1} = 0.412 \implies \mathbf{x}_2 = \mathbf{x}_1 + \alpha_1 \mathbf{d}_1 = \begin{pmatrix} 2 \\ -2 \end{pmatrix}.$$

This time there is no zig-zagging and the solution is reached in just two iterations:



In exact arithmetic, the conjugate gradient method will always give the exact answer in n iterations – one way to see this is to use the following.

Theorem 3.5

The residuals $\mathbf{r}_k := A\mathbf{x}_k - \mathbf{b}$ at each stage of the conjugate gradient method are mutually orthogonal, meaning $\mathbf{r}_j^\top \mathbf{r}_k = 0$ for $j = 0, \ldots, k-1$.

After n iterations, the only residual vector that can be orthogonal to all of the previous ones is $\mathbf{r}_n = \mathbf{0}$, so $\mathbf{x}_n$ must be the exact solution.

In practice, conjugate gradients is not competitive as a direct method for small dense systems, and rounding errors can destroy the orthogonality, meaning that more than n iterations may be required. Instead, its main use is for large sparse systems, where we stop long before n iterations. With some algebraic rearrangement, each iteration requires only *one* matrix-vector product, and the error decreases by a factor of about

$$\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$$

per iteration, which is much faster than steepest descent when κ is large. This is another reason why conditioning matters: **preconditioning** replaces $A\mathbf{x} = \mathbf{b}$ by an equivalent system with a smaller condition number, so that conjugate gradients converges in fewer iterations.

Example 3.24: MATLAB demonstration: steepest descent vs conjugate gradients

```
A = [3 2; 2 6];  b = [2; -8];
[X, Y] = meshgrid(linspace(-4, 6, 200), linspace(-6, 4, 200));
F = 0.5*(A(1,1)*X.^2 + 2*A(1,2)*X.*Y + A(2,2)*Y.^2) - b(1)*X - b(2)*Y;

xs = zeros(2, 11);  xs(:,1) = [-2; -2];        % steepest descent
for k = 1:10
    d = b - A*xs(:,k);                          % d = -r
    xs(:,k+1) = xs(:,k) + (d'*d)/(d'*A*d)*d;
end

xc = zeros(2, 3);  xc(:,1) = [-2; -2];         % conjugate gradients
r = A*xc(:,1) - b;  d = -r;
for k = 1:2
    alpha = -(d'*r)/(d'*A*d);
    xc(:,k+1) = xc(:,k) + alpha*d;
    rnew = A*xc(:,k+1) - b;
    beta = (rnew'*A*d)/(d'*A*d);
    d = -rnew + beta*d;  r = rnew;
end

figure
contour(X, Y, F, 30), hold on
plot(xs(1,:), xs(2,:), 'o-', 'LineWidth', 2.5, 'MarkerSize', 8)
plot(xc(1,:), xc(2,:), 's-', 'LineWidth', 2.5, 'MarkerSize', 8), hold off
legend('contours of f', 'steepest descent', 'conjugate gradients', 'Location', 'northwest')
xlabel('x_1'), ylabel('x_2'), axis equal, set(gca, 'FontSize', 18)
```

Example 3.25: MATLAB demonstration: conjugate gradients for a large sparse system

```
A = gallery('poisson', 100);       % sparse SPD matrix, 10^4-by-10^4, at most 5 non-zeros p
figure
spy(A(1:500, 1:500))
title('Sparsity pattern (first 500 rows and columns)'), set(gca, 'FontSize', 18)

b = ones(size(A, 1), 1);
[x, flag, relres, iter, resvec] = pcg(A, b, 1e-8, 1000);
figure
semilogy(0:length(resvec)-1, resvec/norm(b), 'LineWidth', 2.5)
legend('conjugate gradients', 'Location', 'northeast')
xlabel('iteration k'), ylabel('relative residual'), set(gca, 'FontSize', 18), grid on
fprintf('CG converged in %d iterations for n = %d unknowns\n', iter, size(A, 1));
```

This matrix comes from the two-dimensional heat (or Poisson) equation, and conjugate gradients converges in far fewer than $n = 10^4$ iterations.

Knowledge checklist

Key topics:

1. Direct methods for linear systems: triangular systems, Gaussian elimination, LU decomposition and partial pivoting, and their computational cost.
2. Vector norms and induced matrix norms.
3. Conditioning: the condition number and its effect on the accuracy of computed solutions.
4. Iterative methods for sparse symmetric positive definite systems: steepest descent and the conjugate gradient method.

Key skills:

- Solving linear systems by forward and backward substitution, Gaussian elimination and LU decomposition (with partial pivoting), and counting the operations required.
- Computing vector and matrix norms and condition numbers, and using them to estimate the accuracy of a computed solution.
- Applying steepest descent and conjugate gradients to SPD systems, and explaining how their convergence depends on the condition number.

4 Calculus

4.1 Differentiation

How do we differentiate functions numerically?

4.1.1 Basics

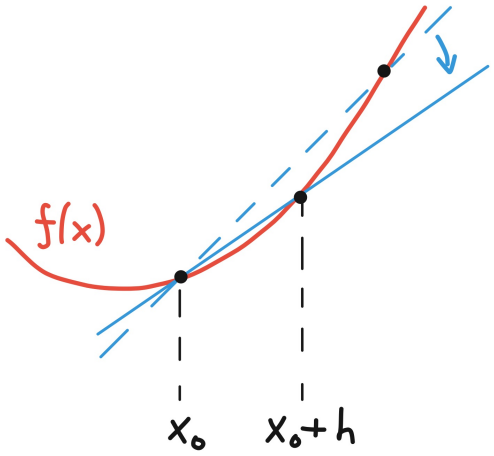
The definition of the derivative as

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h},$$

suggests an obvious approximation: just pick some small finite h to give the estimate

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}.$$

For $h > 0$ this is called a **forward difference** (and, for $h < 0$, a **backward difference**). It is an example of a **finite-difference formula**.



Geometrically, the forward difference is the slope of the straight line that interpolates f at the nodes x_0 and $x_0 + h$. To find its error, we use Taylor's theorem, which tells us that

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2} f''(\xi)$$

for some ξ between x_0 and $x_0 + h$. Rearranging gives

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi).$$

This shows that the **truncation error** for our forward difference approximation is $-f''(\xi)h/2$, for some $\xi \in [x_0, x_0 + h]$. In other words, a smaller interval or a less "wiggly" function will lead to a better estimate, as you would expect.

i Note

We could also have derived the forward difference (and its error) by differentiating the linear interpolating polynomial and its error formula from Chapter 2. In general, finite-difference formulae are derivatives of interpolating polynomials, but Taylor's theorem is usually the quickest way to find their errors.

Example 4.1: Derivative of $f(x) = \log(x)$ at $x_0 = 2$.

Using a forward-difference, we get the following sequence of approximations:

h	Forward difference	Error $\lvert f'(2) - D \rvert$
1	0.405465	0.0945349
0.1	0.487902	0.0120984
0.01	0.498754	0.00124585
0.001	0.499875	0.000124958

Indeed the error is linear in h, and we estimate that it is approximately $0.125h$ when h is small. This agrees with the error formula above, since $f''(x) = -x^{-2}$, so we expect $|f''(\xi)|/2 \approx \frac{1}{8}$.

Since the error is proportional to h, the approximation is called **linear**, or **first order**, and we write the error as $\mathcal{O}(h)$.

i Reminder: big-O notation

We write $g(h) = \mathcal{O}(h^p)$ as $h \to 0$ if there is a constant M such that $|g(h)| \leq M|h|^p$ for all sufficiently small h. In other words, $g(h)$ gets smaller at least as fast as h^p. A method

whose error is $\mathcal{O}(h^p)$ is said to be of **order** p.

4.1.2 Higher-order finite differences

To get a higher-order approximation, we can use more nodes. The most important example uses the two points on either side of x.

Example 4.2: Central difference

Using Taylor's theorem to expand f about x in both directions gives

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(\xi_\pm),$$

for some $\xi_+ \in (x, x+h)$ and $\xi_- \in (x-h, x)$. Subtracting the two expansions cancels the $f(x)$ and $f''(x)$ terms, and (using the Intermediate Value Theorem to combine the two f''' terms) we get

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6}f'''(\xi)$$

for some $\xi \in (x-h, x+h)$. This is called a **central difference** approximation for $f'(x)$, and is frequently used in practice. It is the slope of the quadratic that interpolates f at $x-h$, x and $x+h$, evaluated at the middle point.

To see the quadratic behaviour of the truncation error, go back to our earlier example.

Example 4.3: Derivative of $f(x) = \log(x)$ at $x = 2$

h	Forward difference	Error	Central difference	Error
1	0.405465	0.0945349	0.549306	0.0493061
0.1	0.487902	0.0120984	0.500417	0.000417293
0.01	0.498754	0.00124585	0.500004	4.16673e-06
0.001	0.499875	0.000124958	0.500000	4.16666e-08

The error for the central difference is about $0.04h^2$, which agrees with the formula since $f'''(\xi) \approx 2/2^3 = \frac{1}{4}$ when h is small.

The same idea gives a formula for the **second derivative**. Adding (rather than subtracting) the Taylor expansions of $f(x \pm h)$, carried to one more term, cancels all of the odd derivatives

and gives

$$f''(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} - \frac{h^2}{12} f^{(4)}(\xi)$$

for some $\xi \in (x-h, x+h)$. This **second-order central difference** for f'' is the basis of the finite-difference methods for partial differential equations in Chapter 6.

Example 4.4: Second derivative of $f(x) = \log(x)$ at $x = 2$

Here $f''(2) = -\frac{1}{4}$ and $f^{(4)}(2) = -6/2^4 = -\frac{3}{8}$, so we expect an error of about $\frac{3}{8} \cdot \frac{h^2}{12} = 0.03125h^2$:

h	Central difference for $f''(2)$	Error
1	-0.2876820725	0.0376821
0.1	-0.2503130218	0.000313022
0.01	-0.2500031251	3.12505e-06
0.001	-0.2500000311	3.11212e-08

4.1.3 Rounding error

The problem with numerical differentiation is that it involves subtraction of nearly-equal numbers. As h gets smaller, the problem gets worse.

To quantify this for the central difference, suppose that we have the correctly rounded values of $f(x \pm h)$, so that

$$\mathrm{fl}[f(x+h)] = (1+\delta_1) f(x+h), \qquad \mathrm{fl}[f(x-h)] = (1+\delta_2) f(x-h),$$

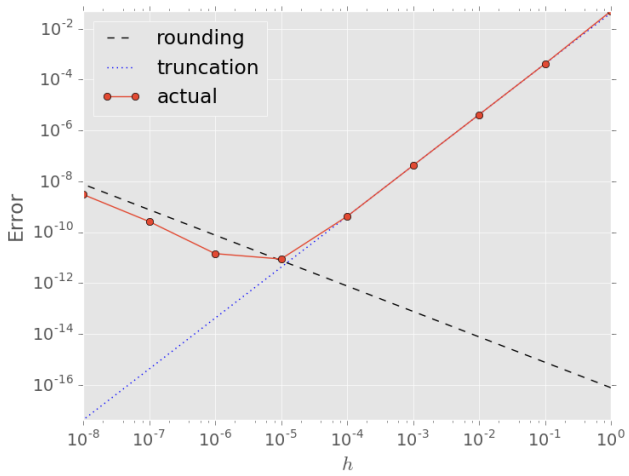
where $|\delta_1|, |\delta_2| \le \epsilon_{\mathrm{M}}$. Ignoring the rounding error in dividing by $2h$, we then have that

$$\begin{aligned}
&\left| f'(x) - \frac{\mathrm{fl}[f(x+h)] - \mathrm{fl}[f(x-h)]}{2h} \right| \\
=&\left| -\frac{f'''(\xi)}{6} h^2 - \frac{\delta_1 f(x+h) - \delta_2 f(x-h)}{2h} \right| \\
\le&\frac{|f'''(\xi)|}{6} h^2 + \epsilon_{\mathrm{M}} \frac{|f(x+h)| + |f(x-h)|}{2h} \\
\le&\frac{h^2}{6} M_3 + \frac{\epsilon_{\mathrm{M}}}{h} M_0,
\end{aligned}$$

where M_0 and M_3 are the maximum values of $|f|$ and $|f'''|$ on $[x-h, x+h]$. The first term is the truncation error, which tends to zero as $h \to 0$. But the second term is the rounding error, which tends to infinity as $h \to 0$.

Example 4.5: Derivative of $f(x) = \log(x)$ at $x = 2$ again

Here is a comparison of the terms in the inequality above (the red points are the left-hand side), shown on logarithmic scales, using $\xi = 2$ to estimate the maxima.



You see that once h is small enough, rounding error takes over and the error in the computed derivative starts to increase again.

So there is an **optimal step size**. Minimising the bound $E(h) = \frac{1}{6}h^2 M_3 + \epsilon_{\mathrm{M}} M_0/h$ by setting $E'(h) = \frac{1}{3}hM_3 - \epsilon_{\mathrm{M}} M_0/h^2 = 0$ gives

$$h_* = \left(\frac{3\epsilon_{\mathrm{M}} M_0}{M_3}\right)^{1/3} \sim \epsilon_{\mathrm{M}}^{1/3},$$

and the smallest possible error is of order $\epsilon_{\mathrm{M}}^{2/3}$. For our example, $h_* \approx 10^{-5}$, giving an error of about 10^{-11}: we can never get a central-difference derivative accurate to the full 16 digits. The same argument for the (first-order) forward difference gives $h_* \sim \epsilon_{\mathrm{M}}^{1/2} \approx 10^{-8}$, with an error of about 10^{-8}.

4.1.4 Differentiating noisy data

Exactly the same trade-off arises when f is only known through **measurements**, except that the errors in the data are usually far larger than ϵ_{M}. If each measured value has an error of size up to δ, then the error in the central difference is bounded by about

$$\frac{h^2}{6} M_3 + \frac{\delta}{h},$$

so the error from the data is amplified by a factor $1/h$. For the second derivative, the amplification factor is about $4/h^2$, which is much worse. With noisy data, it is often better to first **fit** a smooth model to the data (for example by least squares, as mentioned in Chapter 3) and then differentiate the model.

Example 4.6: MATLAB demonstration: differentiating noisy measurements

A ball rolls down a ramp with constant acceleration $a = 2\,\mathrm{m/s^2}$, so its position is $x(t) = t^2$. A sensor records the position every $0.02\,\mathrm{s}$, with random errors of about $2\,\mathrm{mm}$.

```
rng(1)                                          % reproducible "measurements"
dt = 0.02;  t = (0:dt:2)';
x = t.^2 + 0.002*randn(size(t));                % noisy positions [m]

% Central differences for velocity and acceleration at interior points
v = (x(3:end) - x(1:end-2)) / (2*dt);
a = (x(3:end) - 2*x(2:end-1) + x(1:end-2)) / dt^2;

% Least-squares quadratic fit, then differentiate the fitted model
p = polyfit(t, x, 2);
aFit = 2*p(1);

figure
subplot(1, 2, 1)
plot(t(2:end-1), v, 'o', t, 2*t, 'LineWidth', 2.5, 'MarkerSize', 5)
legend('finite differences', 'exact v = 2t', 'Location', 'northwest')
xlabel('t [s]'), ylabel('velocity [m/s]'), grid on, set(gca, 'FontSize', 16)
subplot(1, 2, 2)
plot(t(2:end-1), a, 'o', t, 2*ones(size(t)), t, aFit*ones(size(t)), '--', ...
    'LineWidth', 2.5, 'MarkerSize', 5)
legend('finite differences', 'exact a = 2', 'from least-squares fit', 'Location', 'southwes
xlabel('t [s]'), ylabel('acceleration [m/s^2]'), grid on, set(gca, 'FontSize', 16)
```

The velocities are slightly noisy, but the accelerations are useless: errors of $2\,\mathrm{mm}$ become errors of about $10\,\mathrm{m/s^2}$ after dividing by $h^2 = 4 \times 10^{-4}$. The acceleration from the fitted model is very close to the true value.

4.1.5 Richardson extrapolation

Finding higher-order formulae by differentiating interpolating polynomials is tedious, and there is a simpler trick to obtain higher-order formulae, called **Richardson extrapolation**.

We begin from the central-difference formula. Since we will use formulae with different h, let us define the notation

$$D_h := \frac{f(x+h) - f(x-h)}{2h}.$$

Now use Taylor's theorem to expand more terms in the truncation error:

$$\begin{aligned} f(x \pm h) = f(x) \pm f'(x)h + \frac{f''(x)}{2}h^2 \pm \frac{f'''(x)}{3!}h^3 \\ + \frac{f^{(4)}(x)}{4!}h^4 \pm \frac{f^{(5)}(x)}{5!}h^5 + \mathcal{O}(h^6). \end{aligned}$$

Substituting into the formula for D_h, the even powers of h cancel and we get

$$\begin{aligned} D_h &= \frac{1}{2h}\left(2f'(x)h + 2f'''(x)\frac{h^3}{6} + 2f^{(5)}(x)\frac{h^5}{120} + \mathcal{O}(h^7)\right) \\ &= f'(x) + f'''(x)\frac{h^2}{6} + f^{(5)}(x)\frac{h^4}{120} + \mathcal{O}(h^6). \end{aligned}$$

> **Note**
>
> The remainder here is $\mathcal{O}(h^6)$ because it gets smaller at least as fast as h^6 as $h \to 0$ (essentially, it contains no powers of h less than 6).

The leading term in the error here has the same coefficient $h^2/6$ as the truncation error we derived earlier, although we have now expanded the error to higher powers of h.

The trick is to apply the same formula with different step-sizes, typically h and $h/2$:

$$\begin{aligned} D_h &= f'(x) + f'''(x)\frac{h^2}{6} + f^{(5)}(x)\frac{h^4}{120} + \mathcal{O}(h^6), \\ D_{h/2} &= f'(x) + f'''(x)\frac{h^2}{2^2(6)} + f^{(5)}(x)\frac{h^4}{2^4(120)} + \mathcal{O}(h^6). \end{aligned}$$

We can then eliminate the h^2 term by simple algebra:

$$\begin{aligned} D_h - 2^2 D_{h/2} &= -3f'(x) + \left(1 - \frac{2^2}{2^4}\right) f^{(5)}(x)\frac{h^4}{120} + \mathcal{O}(h^6), \\ \implies \quad D_h^{(1)} &:= \frac{2^2 D_{h/2} - D_h}{3} = f'(x) - f^{(5)}(x)\frac{h^4}{480} + \mathcal{O}(h^6). \end{aligned}$$

The new formula $D_h^{(1)}$ is 4th-order accurate.

Example 4.7: Derivative of $f(x) = \log(x)$ at $x = 2$ (central difference).

h	D_h	|Error|	$D_h^{(1)}$	|Error|
1.0	0.5493061443	0.04930614433	0.4979987836	0.00200121642
0.1	0.5004172928	0.00041729278	0.4999998434	1.56599487e-07
0.01	0.5000041667	4.16672916e-06	0.5000000000	1.56388791e-11
0.001	0.5000000417	4.16666151e-08	0.5000000000	9.29256672e-14

The error in $D_h^{(1)}$ decreases by a factor of about 10^4 each time h is divided by 10, as expected for a 4th-order method, until the last row. There, the truncation error would be about 2×10^{-15}, but rounding error (of size about ϵ_{M}/h) has taken over.

In fact, we could have applied this **Richardson extrapolation** procedure without knowing the coefficients of the error series. If we have some general order-n approximation

$$D_h = f'(x) + Ch^n + \mathcal{O}(h^{n+1}),$$

then we can always evaluate it with $h/2$ to get

$$D_{h/2} = f'(x) + C\frac{h^n}{2^n} + \mathcal{O}(h^{n+1})$$

and then eliminate the h^n term to get a new approximation

$$D_h^{(1)} := \frac{2^n D_{h/2} - D_h}{2^n - 1} = f'(x) + \mathcal{O}(h^{n+1}).$$

Furthermore, Richardson extrapolation can be applied iteratively. For example, for central differences, $D_h^{(1)}$ has error $C_1 h^4 + \mathcal{O}(h^6)$, so combining $D_h^{(1)}$ and $D_{h/2}^{(1)}$ in the same way gives the 6th-order approximation $D_h^{(2)} := (2^4 D_{h/2}^{(1)} - D_h^{(1)})/(2^4 - 1)$, and so on.

i Note

The technique is used not only in differentiation but also in **Romberg integration** and the **Bulirsch-Stoer method** for solving ODEs. There is also nothing special about taking $h/2$; we could have taken $h/3$ or even $2h$, and modified the formula accordingly. But $h/2$ is usually convenient.

Note

Lewis Fry Richardson (1881–1953) was from Newcastle and an undergraduate there (when it was still a College of Durham). He was the first person to apply mathematics (finite differences) to weather prediction, and was ahead of his time: in the absence of electronic computers, he estimated that 60,000 people would be needed to predict the next day's weather!

4.2 Numerical integration

How do we calculate integrals numerically?

The definite integral

$$I(f) := \int_a^b f(x)\,\mathrm{d}x$$

can usually not be evaluated in closed form. To approximate it numerically, we can use a **quadrature formula**

$$I_n(f) := \sum_{k=0}^{n} \sigma_k f(x_k),$$

where $x_0, \ldots, x_n$ are a set of **nodes** and $\sigma_0, \ldots, \sigma_n$ are a set of corresponding **weights**.

Note

The nodes are also known as **quadrature points** or **abscissas**, and the weights as **coefficients**.

Example 4.8: The trapezium rule

$$I_1(f) = \frac{b-a}{2}\Big(f(a) + f(b)\Big).$$

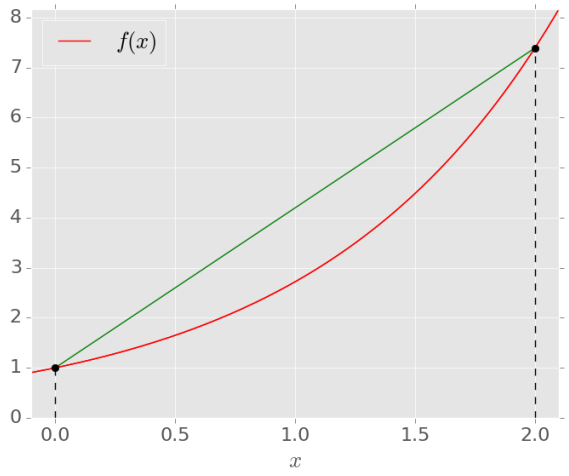
This is the quadrature formula above with $x_0 = a$, $x_1 = b$, $\sigma_0 = \sigma_1 = \frac{1}{2}(b-a)$.
For example, with $a = 0$, $b = 2$, $f(x) = \mathrm{e}^x$, we get

$$I_1(f) = \frac{2-0}{2}(\mathrm{e}^0 + \mathrm{e}^2) = 8.389 \quad \text{to 4 s.f.}$$

The exact answer is

$$I(f) = \int_0^2 \mathrm{e}^x\,\mathrm{d}x = \mathrm{e}^2 - \mathrm{e}^0 = 6.389 \quad \text{to 4 s.f.}$$

Graphically, $I_1(f)$ measures the area under the straight line that interpolates f at the ends:



4.2.1 Newton-Cotes formulae

We can derive a family of "interpolatory" quadrature formulae by integrating interpolating polynomials of different degrees. We will also get error estimates using the interpolation error theorem.

Let $x_0, \ldots, x_n \in [a, b]$, where $x_0 < x_1 < \cdots < x_n$, be a set of $n+1$ nodes, and let $p_n \in \mathcal{P}_n$ be the polynomial that interpolates f at these nodes. This may be written in Lagrange form as

$$p_n(x) = \sum_{k=0}^{n} f(x_k)\ell_k(x), \quad \text{where} \quad \ell_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}.$$

To approximate $I(f)$, we integrate $p_n(x)$ to define the quadrature formula

$$I_n(f) := \int_a^b \sum_{k=0}^{n} f(x_k)\ell_k(x)\,\mathrm{d}x = \sum_{k=0}^{n} f(x_k) \int_a^b \ell_k(x)\,\mathrm{d}x.$$

In other words,

$$I_n(f) := \sum_{k=0}^{n} \sigma_k f(x_k), \quad \text{where} \quad \sigma_k = \int_a^b \ell_k(x)\,\mathrm{d}x.$$

When the nodes are equidistant, this is called a **Newton-Cotes formula**. If $x_0 = a$ and $x_n = b$, it is called a **closed Newton-Cotes formula**.

Example 4.9: Trapezium rule

This is the closed Newton-Cotes formula with $n = 1$. To see this, let $x_0 = a$, $x_1 = b$. Then

$$\begin{aligned}\ell_0(x) = \frac{x-b}{a-b} \implies \sigma_0 &= \int_a^b \ell_0(x)\,\mathrm{d}x \\ &= \frac{1}{a-b}\int_a^b (x-b)\,\mathrm{d}x \\ &= \frac{1}{2(a-b)}(x-b)^2\Big|_a^b \\ &= \frac{b-a}{2},\end{aligned}$$

and

$$\begin{aligned}\ell_1(x) = \frac{x-a}{b-a} \implies \sigma_1 &= \int_a^b \ell_1(x)\,\mathrm{d}x \\ &= \frac{1}{b-a}\int_a^b (x-a)\,\mathrm{d}x \\ &= \frac{1}{2(b-a)}(x-a)^2\Big|_a^b = \frac{b-a}{2}.\end{aligned}$$

This gives

$$I_1(f) = \sigma_0 f(a) + \sigma_1 f(b) = \frac{b-a}{2}(f(a)+f(b)).$$

Theorem 4.1

Let f be continuous on $[a,b]$ with $n+1$ continuous derivatives on (a,b). Then the Newton-Cotes formula above satisfies the error bound

$$|I(f) - I_n(f)| \leq \frac{\max_{\xi\in[a,b]} |f^{(n+1)}(\xi)|}{(n+1)!}\int_a^b |(x-x_0)(x-x_1)\cdots(x-x_n)|\,\mathrm{d}x.$$

Proof: First note that the error in the Newton-Cotes formula may be written

$$\begin{aligned}|I(f) - I_n(f)| &= \left|\int_a^b f(x)\,\mathrm{d}x - \int_a^b p_n(x)\,\mathrm{d}x\right| \\ &= \left|\int_a^b [f(x) - p_n(x)]\,\mathrm{d}x\right| \\ &\leq \int_a^b |f(x) - p_n(x)|\,\mathrm{d}x.\end{aligned}$$

Now recall the interpolation error theorem, which says that, for each $x \in [a, b]$, we can write

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)(x - x_1) \cdots (x - x_n)$$

for some $\xi \in (a, b)$. The theorem simply follows by inserting this into the inequality above. □

Example 4.10: Trapezium rule error

Let $M_2 = \max_{\xi \in [a,b]} |f''(\xi)|$. Here the theorem reduces to

$$\begin{aligned} |I(f) - I_1(f)| &\leq \frac{M_2}{(1+1)!} \int_a^b |(x-a)(x-b)| \, \mathrm{d}x \\ &= \frac{M_2}{2!} \int_a^b (x-a)(b-x) \, \mathrm{d}x \\ &= \frac{(b-a)^3}{12} M_2. \end{aligned}$$

For our earlier example with $a = 0$, $b = 2$, $f(x) = \mathrm{e}^x$, the estimate gives

$$|I(f) - I_1(f)| \leq \tfrac{1}{12}(2^3)\mathrm{e}^2 \approx 4.926.$$

This is an overestimate of the actual error which was ≈ 2.000.

The theorem suggests that the accuracy of I_n is limited both by the smoothness of f (outside our control) and by the location of the nodes x_k. If the nodes are free to be chosen, then we can use **Gaussian quadrature** (see below).

Note

As with interpolation, taking a high n is not usually a good idea. One can prove for the closed Newton-Cotes formula that

$$\sum_{k=0}^{n} |\sigma_k| \to \infty \quad \text{as} \quad n \to \infty.$$

This makes the quadrature vulnerable to rounding errors for large n.

4.2.2 Composite Newton-Cotes formulae

Since the Newton-Cotes formulae are based on polynomial interpolation at equally-spaced points, the results need not converge as the number of nodes increases (recall the Runge phenomenon from Chapter 2). A better way to improve accuracy is to divide the interval $[a, b]$

into m subintervals $[x_{i-1}, x_i]$ of equal length

$$h := \frac{b-a}{m},$$

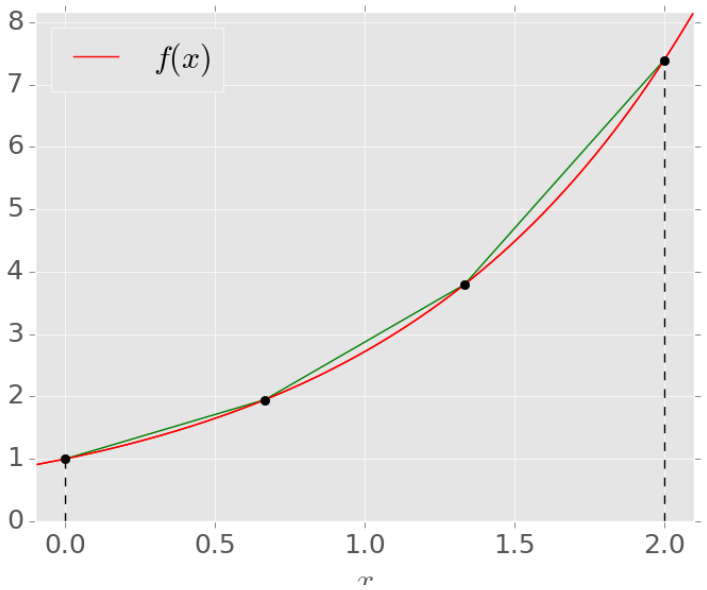
and use a Newton-Cotes formula of small degree n on each subinterval.

Example 4.11: Composite trapezium rule

Applying the trapezium rule $I_1(f)$ on each subinterval gives

$$\begin{aligned} C_{1,m}(f) &= \frac{h}{2}\left[f(x_0) + f(x_1) + f(x_1) + f(x_2) + \ldots + f(x_{m-1}) + f(x_m)\right], \\ &= h\left[\tfrac{1}{2}f(x_0) + f(x_1) + f(x_2) + \ldots + f(x_{m-1}) + \tfrac{1}{2}f(x_m)\right]. \end{aligned}$$

We are effectively integrating a piecewise-linear approximation of $f(x)$; here we show $m = 3$ for our test problem $f(x) = \mathrm{e}^x$ on $[0, 2]$:



When we halve the sub-interval width h, the error goes down by a factor 4 (see the table below), suggesting that we have quadratic convergence, i.e., $\mathcal{O}(h^2)$.
To show this theoretically, we can apply the Newton-Cotes error theorem on each subinterval. On $[x_{i-1}, x_i]$, the error of the trapezium rule is at most

$$\frac{\max_{\xi\in[x_{i-1},x_i]} |f''(\xi)|}{2!} \int_{x_{i-1}}^{x_i} |(x - x_{i-1})(x - x_i)| \, \mathrm{d}x$$

Note that

$$\begin{aligned}\int_{x_{i-1}}^{x_i} \left|(x-x_{i-1})(x-x_i)\right| \mathrm{d}x &= \int_{x_{i-1}}^{x_i} (x-x_{i-1})(x_i-x)\,\mathrm{d}x \\ &= \int_{x_{i-1}}^{x_i} \left[-x^2+(x_{i-1}+x_i)x - x_{i-1}x_i\right] \mathrm{d}x \\ &= \left[-\tfrac{1}{3}x^3 + \tfrac{1}{2}(x_{i-1}+x_i)x^2 - x_{i-1}x_i x\right]_{x_{i-1}}^{x_i} \\ &= \tfrac{1}{6}x_i^3 - \tfrac{1}{2}x_{i-1}x_i^2 + \tfrac{1}{2}x_{i-1}^2 x_i - \tfrac{1}{6}x_{i-1}^3 \\ &= \tfrac{1}{6}(x_i - x_{i-1})^3 = \tfrac{1}{6}h^3.\end{aligned}$$

Adding up the errors on all m subintervals gives

$$\begin{aligned}|I(f) - C_{1,m}(f)| &\leq \tfrac{1}{2}\max_i \left(\max_{\xi\in[x_{i-1},x_i]} |f''(\xi)|\right) m\frac{h^3}{6} \\ &= \frac{mh^3}{12}\max_{\xi\in[a,b]}|f''(\xi)| = \frac{b-a}{12}h^2 \max_{\xi\in[a,b]}|f''(\xi)|.\end{aligned}$$

As long as f is sufficiently smooth, this shows that the composite trapezium rule will converge as $m\to\infty$. Moreover, the convergence will be $\mathcal{O}(h^2)$.

4.2.3 Exactness

From the Newton-Cotes error theorem, we see that the Newton-Cotes formula $I_n(f)$ will give the exact answer if $f^{(n+1)} = 0$. In other words, it will be exact if $f \in \mathcal{P}_n$.

Example 4.12

The trapezium rule $I_1(f)$ is exact for all linear polynomials $f \in \mathcal{P}_1$.

The **degree of exactness** of a quadrature formula is the largest integer n for which the formula is exact for all polynomials in $\mathcal{P}_n$.

To check whether a quadrature formula has degree of exactness n, it suffices to check whether it is exact for the basis 1, x, $x^2, \ldots, x^n$.

Example 4.13: Simpson's rule

This is the $n = 2$ closed Newton-Cotes formula

$$I_2(f) = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right],$$

derived by integrating a quadratic interpolating polynomial. Let us find its degree of exactness:

$$\begin{aligned}
I(1) &= \int_a^b \mathrm{d}x = (b-a),\\
I_2(1) &= \frac{b-a}{6}[1+4+1] = b-a = I(1),\\
I(x) &= \int_a^b x\,\mathrm{d}x = \frac{b^2-a^2}{2},\\
I_2(x) &= \frac{b-a}{6}\left[a + 2(a+b) + b\right] = \frac{(b-a)(b+a)}{2} = I(x),\\
I(x^2) &= \int_a^b x^2\,\mathrm{d}x = \frac{b^3-a^3}{3},\\
I_2(x^2) &= \frac{b-a}{6}\left[a^2 + (a+b)^2 + b^2\right] = \frac{2(b^3-a^3)}{6} = I(x^2),\\
I(x^3) &= \int_a^b x^3\,\mathrm{d}x = \frac{b^4-a^4}{4},\\
I_2(x^3) &= \frac{b-a}{6}\left[a^3 + \tfrac{1}{2}(a+b)^3 + b^3\right] = \frac{b^4-a^4}{4} = I(x^3).
\end{aligned}$$

This shows that the degree of exactness is at least 3 (contrary to what might be expected from the interpolation picture). You can verify that $I_2(x^4) \neq I(x^4)$, so the degree of exactness is exactly 3.

This shows that the term $f'''(\xi)$ in the Newton-Cotes error bound for Simpson's rule is misleading. In fact, a more careful analysis gives the error bound

$$|I(f) - I_2(f)| \leq \frac{(b-a)^5}{2880} \max_{\xi\in[a,b]} |f^{(4)}(\xi)|.$$

In terms of degree of exactness, Simpson's formula does better than expected. In general, Newton-Cotes formulae with even n have degree of exactness $n+1$. But this is by no means the highest possible (see the next section).

Simpson's rule can also be used in composite form. Divide $[a,b]$ into an *even* number m of subintervals of width $h = (b-a)/m$, and apply Simpson's rule on each consecutive pair of subintervals. This gives the **composite Simpson's rule**

$$C_{2,m}(f) = \frac{h}{3}\Big[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \ldots + 2f(x_{m-2}) + 4f(x_{m-1}) + f(x_m)\Big],$$

which satisfies

$$|I(f) - C_{2,m}(f)| \le \frac{b-a}{180} h^4 \max_{\xi \in [a,b]} |f^{(4)}(\xi)|.$$

So the composite Simpson's rule converges with order $\mathcal{O}(h^4)$, using the same function values as the composite trapezium rule.

The midpoint rule

Another simple and widely used rule is the **midpoint rule**

$$M(f) = (b-a)\, f\Big(\frac{a+b}{2}\Big),$$

which approximates f by a constant (its value at the midpoint). Despite using only one node, it is exact for all linear polynomials, because the errors on either side of the midpoint cancel; in fact, it is the one-node Gaussian quadrature rule (see below). Its error satisfies $|I(f) - M(f)| \le \frac{(b-a)^3}{24} \max_{\xi \in [a,b]} |f''(\xi)|$, and the **composite midpoint rule** $h \sum_{i=1}^{m} f(\frac{x_{i-1}+x_i}{2})$ converges with order $\mathcal{O}(h^2)$, like the composite trapezium rule but with half the error constant.

Example 4.14: Composite rules for $\int_0^2 \mathrm{e}^x \,\mathrm{d}x = 6.389056\ldots$

m	h	$C_{1,m}(f)$	$\lvert I(f) - C_{1,m}(f)\rvert$	$C_{2,m}(f)$	$\lvert I(f) - C_{2,m}(f)\rvert$
1	2	8.389056	2.000	–	–
2	1	6.912810	0.524	6.420728	3.17e-02
4	0.5	6.521610	0.133	6.391210	2.15e-03
8	0.25	6.422298	0.033	6.389194	1.38e-04
16	0.125	6.397373	0.008	6.389065	8.65e-06
32	0.0625	6.391136	0.002	6.389057	5.41e-07

Halving h reduces the error by a factor of about 4 for the trapezium rule, and about 16 for Simpson's rule.

Example 4.15: MATLAB demonstration: integrating sampled data

In applications, we often only know the integrand at the points where it was measured, so Newton-Cotes formulae are the natural choice. Suppose a car accelerates from rest with speed $v(t) = 20(1 - \mathrm{e}^{-t/20})$ m/s, but we only have speedometer readings every 5 s for one minute. The distance travelled is the integral of the speed.

```
t = 0:5:60;                                   % times of the readings [s]
v = 20*(1 - exp(-t/20));                      % speedometer readings [m/s]
dTrap = trapz(t, v);                          % composite trapezium rule
h = t(2) - t(1);                              % composite Simpson (12 subintervals)
dSimp = h/3*(v(1) + 4*sum(v(2:2:end-1)) + 2*sum(v(3:2:end-2)) + v(end));
dExact = 20*(60 + 20*exp(-3) - 20);
fprintf('Trapezium: %.3f m,  Simpson: %.3f m,  exact: %.3f m\n', dTrap, dSimp, dExact);

figure
area(t, v, 'FaceColor', [0.85 0.9 1]), hold on
plot(t, v, 'o-', 'LineWidth', 2.5, 'MarkerSize', 8), hold off
legend('distance = area under the curve', 'speedometer readings', 'Location', 'southeast')
xlabel('time [s]'), ylabel('speed [m/s]'), grid on, set(gca, 'FontSize', 18)
```

The trapezium rule underestimates the exact distance of 819.9 m by about 2 m, because the speed curve is concave; Simpson's rule is much more accurate using the same readings.

4.2.4 Gaussian quadrature

The idea of **Gaussian quadrature** is to choose not only the weights σ_k but also the nodes x_k, in order to achieve the highest possible degree of exactness.

Firstly, we will illustrate the brute force **method of undetermined coefficients**.

Example 4.16: Gaussian quadrature formula $G_1(f) = \sum_{k=0}^{1} \sigma_k f(x_k)$ on the interval $[-1, 1]$

Here we have four unknowns x_0, x_1, σ_0 and σ_1, so we can impose four conditions:

$$G_1(1) = I(1) \implies \sigma_0 + \sigma_1 = \int_{-1}^{1} \mathrm{d}x = 2,$$

$$G_1(x) = I(x) \implies \sigma_0 x_0 + \sigma_1 x_1 = \int_{-1}^{1} x \,\mathrm{d}x = 0,$$

$$G_1(x^2) = I(x^2) \implies \sigma_0 x_0^2 + \sigma_1 x_1^2 = \int_{-1}^{1} x^2 \,\mathrm{d}x = \tfrac{2}{3},$$

$$G_1(x^3) = I(x^3) \implies \sigma_0 x_0^3 + \sigma_1 x_1^3 = \int_{-1}^{1} x^3 \,\mathrm{d}x = 0.$$

To solve this system, the symmetry suggests that $x_1 = -x_0$ and $\sigma_0 = \sigma_1$. This will

automatically satisfy the equations for x and x^3, leaving the two equations

$$2\sigma_0 = 2, \qquad 2\sigma_0 x_0^2 = \tfrac{2}{3},$$

so that $\sigma_0 = \sigma_1 = 1$ and $x_1 = -x_0 = 1/\sqrt{3}$. The resulting Gaussian quadrature formula is

$$G_1(f) = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

This formula has degree of exactness 3.

In general, the Gaussian quadrature formula with $n+1$ nodes has degree of exactness $2n+1$.

The method of undetermined coefficients becomes unworkable for larger numbers of nodes, because of the nonlinearity of the equations. A much more elegant method uses orthogonal polynomials.

i Orthogonal polynomials

Given a **weight function** $w(x) \geq 0$ on $[a, b]$, we define the inner product

$$(f, g) := \int_a^b f(x)g(x)w(x)\,\mathrm{d}x.$$

Polynomials $\phi_0, \phi_1, \phi_2, \ldots$, with ϕ_k of degree k, are **orthogonal** if $(\phi_j, \phi_k) = 0$ whenever $j \neq k$. They can be constructed by applying the Gram-Schmidt process to $1, x, x^2, \ldots$; for example, taking each ϕ_k to be monic,

$$\phi_0 = 1, \qquad \phi_1 = x - \frac{(x, \phi_0)}{(\phi_0, \phi_0)}\phi_0, \qquad \phi_2 = x^2 - \frac{(x^2, \phi_0)}{(\phi_0, \phi_0)}\phi_0 - \frac{(x^2, \phi_1)}{(\phi_1, \phi_1)}\phi_1.$$

For $w(x) = 1$ on $[-1, 1]$, this gives the (monic) **Legendre polynomials** $\phi_0 = 1$, $\phi_1 = x$, $\phi_2 = x^2 - \frac{1}{3}$, … The key property we need is that ϕ_{n+1} is orthogonal to *every* polynomial of degree at most n, since any such polynomial is a combination of $\phi_0, \ldots, \phi_n$.
For more details, see Burden & Faires, *Numerical Analysis*, Chapter 8 (Approximation Theory), on orthogonal polynomials and least-squares approximation.

Theorem 4.2

If $\{\phi_0, \phi_1, \ldots, \phi_n\}$ is a set of orthogonal polynomials on $[a, b]$ under the inner product $(f, g) = \int_a^b f(x)g(x)w(x)\,\mathrm{d}x$ and ϕ_k is of degree k for each $k = 0, 1, \ldots, n$, then ϕ_k has k

distinct real roots, and these roots lie in the interval (a, b).

i Proof (non-examinable)

Let $x_1, \ldots, x_j$ be the points where $\phi_k(x)$ changes sign in (a, b). If $j = k$ then we are done. Otherwise, suppose $j < k$, and consider the polynomial

$$q_j(x) = (x - x_1)(x - x_2) \cdots (x - x_j).$$

Since q_j has lower degree than ϕ_k, they must be orthogonal, meaning

$$(q_j, \phi_k) = 0 \implies \int_a^b q_j(x)\phi_k(x)w(x)\,\mathrm{d}x = 0.$$

On the other hand, notice that the product $q_j(x)\phi_k(x)$ cannot change sign in $[a, b]$, because each sign change in $\phi_k(x)$ is cancelled out by one in $q_j(x)$. This means that

$$\int_a^b q_j(x)\phi_k(x)w(x)\,\mathrm{d}x \neq 0,$$

which is a contradiction. □

Remarkably, these roots are precisely the optimum choice of nodes for a quadrature formula to approximate the (weighted) integral

$$I_w(f) = \int_a^b f(x)w(x)\,\mathrm{d}x.$$

Theorem 4.3: Gaussian quadrature

Let ϕ_{n+1} be a polynomial in $\mathcal{P}_{n+1}$ that is orthogonal on $[a, b]$ to all polynomials in $\mathcal{P}_n$, with respect to the weight function $w(x)$. If $x_0, x_1, \ldots, x_n$ are the roots of ϕ_{n+1}, then the quadrature formula

$$G_{n,w}(f) := \sum_{k=0}^{n} \sigma_k f(x_k), \qquad \sigma_k = \int_a^b \ell_k(x)w(x)\,\mathrm{d}x$$

approximates $I_w(f)$ with degree of exactness $2n + 1$ (the largest possible).

Proof: First, recall that any interpolatory quadrature formula based on $n + 1$ nodes will be exact for all polynomials in $\mathcal{P}_n$ (this follows from the Newton-Cotes theorem, which can be modified to include the weight function $w(x)$). So in particular, $G_{n,w}$ is exact for $p_n \in \mathcal{P}_n$.

Now let $p_{2n+1} \in \mathcal{P}_{2n+1}$. The trick is to divide this by the orthogonal polynomial ϕ_{n+1} whose roots are the nodes. This gives

$$p_{2n+1}(x) = \phi_{n+1}(x)q_n(x) + r_n(x) \quad \text{for some} \quad q_n, r_n \in \mathcal{P}_n.$$

Then

$$\begin{aligned} G_{n,w}(p_{2n+1}) &= \sum_{k=0}^{n} \sigma_k p_{2n+1}(x_k) = \sum_{k=0}^{n} \sigma_k \Big[\phi_{n+1}(x_k)q_n(x_k) + r_n(x_k)\Big] \\ &= \sum_{k=0}^{n} \sigma_k r_n(x_k) = I_w(r_n), \end{aligned}$$

where we have used the fact that $G_{n,w}$ is exact for $r_n \in \mathcal{P}_n$. Now, since q_n has lower degree than ϕ_{n+1}, it must be orthogonal to ϕ_{n+1}, so

$$I_w(\phi_{n+1}q_n) = \int_a^b \phi_{n+1}(x)q_n(x)w(x)\,\mathrm{d}x = 0$$

and hence

$$\begin{aligned} G_{n,w}(p_{2n+1}) &= I_w(r_n) + 0 = I_w(r_n) + I_w(\phi_{n+1}q_n) \\ &= I_w(\phi_{n+1}q_n + r_n) = I_w(p_{2n+1}). \qquad \square \end{aligned}$$

Like Newton-Cotes, we see that Gaussian quadrature is based on integrating an interpolating polynomial, but now the nodes are the roots of an orthogonal polynomial, rather than equally spaced points.

Example 4.17: Gaussian quadrature with $n = 1$ on $[-1, 1]$ and $w(x) = 1$ (again)

To find the nodes x_0, x_1, we need to find the roots of the orthogonal polynomial $\phi_2(x)$. For this inner product, these are the Legendre polynomials, so

$$\phi_2(x) = x^2 - \tfrac{1}{3}.$$

Thus the nodes are $x_0 = -1/\sqrt{3}, x_1 = 1/\sqrt{3}$. Integrating the Lagrange polynomials gives the corresponding weights

$$\sigma_0 = \int_{-1}^{1} \ell_0(x)\,\mathrm{d}x = \int_{-1}^{1} \frac{x - \frac{1}{\sqrt{3}}}{-\frac{2}{\sqrt{3}}}\,\mathrm{d}x = -\tfrac{\sqrt{3}}{2}\left[\tfrac{1}{2}x^2 - \tfrac{1}{\sqrt{3}}x\right]_{-1}^{1} = 1,$$

$$\sigma_1 = \int_{-1}^{1} \ell_1(x)\,\mathrm{d}x = \int_{-1}^{1} \frac{x + \frac{1}{\sqrt{3}}}{\frac{2}{\sqrt{3}}}\,\mathrm{d}x = \tfrac{\sqrt{3}}{2}\left[\tfrac{1}{2}x^2 + \tfrac{1}{\sqrt{3}}x\right]_{-1}^{1} = 1,$$

as before.

Gaussian quadrature formulae are usually tabulated on $[-1,1]$. To use them on a general interval $[a,b]$, we change variables to $x = \frac{a+b}{2} + \frac{b-a}{2}t$, so that

$$\int_a^b f(x)\,\mathrm{d}x = \frac{b-a}{2}\int_{-1}^{1} f\Big(\tfrac{a+b}{2} + \tfrac{b-a}{2}t\Big)\,\mathrm{d}t \approx \frac{b-a}{2}\sum_{k=0}^{n} \sigma_k f\Big(\tfrac{a+b}{2} + \tfrac{b-a}{2}t_k\Big),$$

where t_k and σ_k are the nodes and weights on $[-1,1]$.

Example 4.18: Two-point Gaussian quadrature for $\int_0^2 \mathrm{e}^x\,\mathrm{d}x$

With $a=0$, $b=2$, the nodes $t_k = \pm 1/\sqrt{3}$ map to $x = 1 \pm 1/\sqrt{3}$, and the weights are $\frac{b-a}{2}\sigma_k = 1$. So

$$G_1(\mathrm{e}^x) = \mathrm{e}^{1-1/\sqrt{3}} + \mathrm{e}^{1+1/\sqrt{3}} = 6.368108\ldots,$$

with error 0.021. Using only two function evaluations, this is far more accurate than the trapezium rule (error 2.000), and even more accurate than Simpson's rule (error 0.032), which uses three.

Note

Using an appropriate weight function $w(x)$ can be useful for integrands with a singularity, since we can incorporate this in $w(x)$ and still approximate the integral with $G_{n,w}$.

Example 4.19: Gaussian quadrature for $\int_0^1 \cos(x)x^{-1/2}\,\mathrm{d}x$, with $n=0$

This is a Fresnel integral, with exact value $1.80905\ldots$ Let us compare the effect of using an appropriate weight function.

1. *Unweighted quadrature ($w(x) \equiv 1$).* The orthogonal polynomial of degree 1 is

$$\phi_1(x) = x - \frac{\int_0^1 x\,\mathrm{d}x}{\int_0^1 \mathrm{d}x} = x - \tfrac{1}{2} \implies x_0 = \tfrac{1}{2}.$$

   The corresponding weight may be found by imposing $G_0(1) = I(1)$, which gives $\sigma_0 = \int_0^1 \mathrm{d}x = 1$. Then our estimate is

$$G_0\left(\frac{\cos(x)}{\sqrt{x}}\right) = \frac{\cos\left(\frac{1}{2}\right)}{\sqrt{\frac{1}{2}}} = 1.2411\ldots$$

2. *Weighted quadrature with $w(x) = x^{-1/2}$.* This time we get

$$\phi_1(x) = x - \frac{\int_0^1 x^{1/2}\,\mathrm{d}x}{\int_0^1 x^{-1/2}\,\mathrm{d}x} = x - \frac{2/3}{2} \implies x_0 = \tfrac{1}{3}.$$

The corresponding weight is $\sigma_0 = \int_0^1 x^{-1/2}\,\mathrm{d}x = 2$, so the new estimate is the more accurate

$$G_{0,w}(\cos(x)) = 2\cos\left(\tfrac{1}{3}\right) = 1.8899\ldots$$

Unlike Newton-Cotes formulae with equally-spaced points, it can be shown that $G_{n,w}(f) \to I_w(f)$ as $n \to \infty$, for any continuous function f. This follows (with a bit of analysis) from the fact that all of the weights σ_k are positive, along with the fact that they sum to a fixed number $\int_a^b w(x)\,\mathrm{d}x$. For Newton-Cotes, the signed weights still sum to a fixed number, but $\sum_{k=0}^n |\sigma_k| \to \infty$, which destroys convergence.

Not surprisingly, there is also an error formula, which depends on $f^{(2n+2)}$.

Theorem 4.4: Error estimate for Gaussian quadrature

Let $\phi_{n+1} \in \mathcal{P}_{n+1}$ be monic and orthogonal on $[a,b]$ to all polynomials in $\mathcal{P}_n$, with respect to the weight function $w(x)$. Let $x_0, x_1, \ldots, x_n$ be the roots of ϕ_{n+1}, and let $G_{n,w}(f)$ be the Gaussian quadrature formula defined above. If f has $2n+2$ continuous derivatives on (a,b), then there exists $\xi \in (a,b)$ such that

$$I_w(f) - G_{n,w}(f) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!}\int_a^b \phi_{n+1}^2(x)w(x)\,\mathrm{d}x.$$

Proof (non-examinable)

We will need the following result from calculus.
Mean value theorem for integrals. If f, g are continuous on $[a,b]$ and $g(x) \geq 0$ for all $x \in [a,b]$, then there exists $\xi \in [a,b]$ such that

$$\int_a^b f(x)g(x)\,\mathrm{d}x = f(\xi)\int_a^b g(x)\,\mathrm{d}x.$$

To prove this, let m and M be the minimum and maximum values of f on $[a,b]$, respectively. Since $g(x) \geq 0$, we have that

$$m\int_a^b g(x)\,\mathrm{d}x \leq \int_a^b f(x)g(x)\,\mathrm{d}x \leq M\int_a^b g(x)\,\mathrm{d}x.$$

Now let $I = \int_a^b g(x)\,\mathrm{d}x$. If $I = 0$ then $g(x) \equiv 0$, so $\int_a^b f(x)g(x)\,\mathrm{d}x = 0$ and the result holds for every ξ. Otherwise, we have

$$m \leq \frac{1}{I}\int_a^b f(x)g(x)\,\mathrm{d}x \leq M,$$

and by the Intermediate Value Theorem, f attains every value between m and M somewhere in $[a, b]$.

Proof of the error estimate. We use **Hermite interpolation**. Since the x_k are distinct, there exists a unique polynomial p_{2n+1} such that

$$p_{2n+1}(x_k) = f(x_k), \quad p'_{2n+1}(x_k) = f'(x_k) \quad \text{for } k = 0, \ldots, n,$$

and (by an argument similar to the proof of the interpolation error theorem) for each x there exists $\lambda \in (a, b)$, depending on x, such that

$$f(x) - p_{2n+1}(x) = \frac{f^{(2n+2)}(\lambda)}{(2n+2)!} \prod_{i=0}^{n} (x - x_i)^2.$$

Now we know that $(x - x_0)(x - x_1) \cdots (x - x_n) = \phi_{n+1}(x)$, since we fixed ϕ_{n+1} to be monic. Hence

$$\int_a^b f(x)w(x)\,\mathrm{d}x - \int_a^b p_{2n+1}(x)w(x)\,\mathrm{d}x = \int_a^b \frac{f^{(2n+2)}(\lambda)}{(2n+2)!}\phi_{n+1}^2(x)w(x)\,\mathrm{d}x.$$

Now we know that $G_{n,w}$ must be exact for p_{2n+1}, so

$$\int_a^b p_{2n+1}(x)w(x)\,\mathrm{d}x = G_{n,w}(p_{2n+1}) = \sum_{k=0}^{n} \sigma_k p_{2n+1}(x_k) = \sum_{k=0}^{n} \sigma_k f(x_k) = G_{n,w}(f).$$

For the right-hand side, we can't take $f^{(2n+2)}(\lambda)$ outside the integral since λ depends on x. But $\phi_{n+1}^2(x)w(x) \geq 0$ on $[a, b]$, so we can apply the mean value theorem for integrals and get

$$I_w(f) - G_{n,w}(f) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \int_a^b \phi_{n+1}^2(x)w(x)\,\mathrm{d}x$$

for some ξ that does not depend on x.

Knowledge checklist

Key topics:

1. Finite-difference approximations of first and second derivatives (forward, backward and central differences) and their truncation errors.
2. The trade-off between truncation and rounding (or measurement) error in numerical differentiation, and the optimal step size.
3. Richardson extrapolation.

4. Quadrature: Newton-Cotes formulae (trapezium and Simpson's rules), their composite versions, error bounds and degree of exactness.
5. Gaussian quadrature: orthogonal polynomials, optimal nodes, degree of exactness and weighted integrals.

Key skills:

- Deriving finite-difference formulae and their errors using Taylor's theorem, and estimating their order of accuracy from numerical results.
- Choosing a step size that balances truncation and rounding error, and recognising when data are too noisy to differentiate directly.
- Applying Richardson extrapolation to improve the accuracy of an approximation.
- Deriving and applying quadrature formulae (including weights, nodes and degree of exactness), and bounding their errors.
- Using composite rules and Gaussian quadrature to compute integrals efficiently, including on general intervals.

5 Differential Equations

How can computers solve differential equations?

Almost all differential equations which arise in mathematics and its applications do not have exact analytical solutions. This is a central motivation for numerical analysis, as well as the historical development of increasingly powerful computers. Most scientific computing languages have pre-built packages for solving differential equations quickly and accurately using numerical approximations, and there are entire classes of software packages designed to do this for specialised industries, such as aerospace or finance. The goal of this Chapter is to understand the fundamentals of numerical timestepping, so that you can understand properties of these numerical methods, and hence so you can choose which to use for a given problem.

> **Note**
>
> This topic is vast, with many textbooks detailing aspects of numerical differential equations from a variety of theoretical or applied perspectives. We will focus on building up the mathematical terminology of different classes of solvers, such as those available in MATLAB and described in detail here, as well as the basics of convergence theory and error analysis.

> **Note**
>
> Symbolic tools can solve many of the analytically-tractable classes of differential equations using a variety of algorithms and heuristics, but these are such a limited set of equations that they are not as frequently used outside of theoretical areas. The MATLAB function `desolve` can be used to solve a reasonably large class of ODEs symbolically.

5.1 Basic Concepts

We will focus on general systems of n first-order differential equations of the form

$$\dot{\mathbf{u}} = \frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = \mathbf{f}(t, \mathbf{u}), \quad \mathbf{u}(t_0) = \mathbf{u}_0 \in \mathbb{R}^n.$$

Many general classes of equations can be written in this form, and such systems also arise when discretising partial differential equations, as well as other kinds of models involving derivatives. When $\mathbf{f}(t, \mathbf{u}) = \mathbf{f}(\mathbf{u})$ (i.e. does not depend on time) we call the system **autonomous**.

It is reasonable to then ask: Why do we only focus on first order differential equations? This ie because any ordinary differential equation of order n can be written as a system of n first-order ordinary differential equations by introducing new variables to represent each derivative up to order $n-1$. This is a standard step in both numerical and analytical solution methods as most modern solvers only handle systems of first-order ODEs.

Suppose we have an nth order ODE:

$$u^{(n)} = f\big(t, u, u', u'', \ldots, u^{(n-1)}\big),$$

where $u^{(n)} := d^n u(t)/du^n$. Now introduce the auxiliary variables:

$$y_0 = u, \quad y_1 = u', \quad \ldots, \quad y_{n-1} = u^{(n-1)}$$

Then, the new system is:

$$\begin{aligned} y_0' &= y_1 \\ y_1' &= y_2 \\ &\vdots \\ y_{n-2}' &= y_{n-1} \\ y_{n-1}' &= f\big(t, y_0, y_1, \ldots, y_{n-1}\big) \end{aligned}$$

This approach allows the use of vectorized notation and standard solution methods for systems of first-order ODEs.

Example 5.1

Convert the third-order ODE

$$u''' = t + 2u + 3u''$$

with initial conditions $u(0) = 1$, $u'(0) = 2$, $u''(0) = 3$, into a system of three first-order ODEs.
Let

$$y_0 = u \qquad y_1 = u' \qquad y_2 = u'',$$

and then deduce that

$$\begin{aligned} y_0' &= y_1 \\ y_1' &= y_2 \\ y_2' &= t + 2y_0 + 3y_2 \end{aligned}$$

with initial conditions

$$y_0(0) = 1, \quad y_1(0) = 2, \quad y_2(0) = 3.$$

We can also represent this system in matrix form as follows:

$$\frac{d}{dt}\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 2 & 0 & 3 \end{bmatrix}\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ t \end{bmatrix}.$$

5.1.1 Preliminary ODE theory

Before discussing practical aspects of solving such equations, recall the basic existence and uniqueness theory. For ODEs this theory is not too complicated, provided the right-hand side is sufficiently well behaved.

Theorem 5.1

Picard–Lindelöf theorem.
Let $\mathcal{D} \subset \mathbb{R} \times \mathbb{R}^n$ be an open rectangle with interior point $(t_0, \mathbf{u}_0)$. Suppose $\mathbf{f} : \mathcal{D} \to \mathbb{R}^n$ is continuous in t and Lipschitz continuous in $\mathbf{u}$ for all $(t, \mathbf{u}) \in \mathcal{D}$. Then there exists $\varepsilon > 0$ such that the initial-value problem above has a unique solution $\mathbf{u}(t)$ for $t \in [t_0 - \varepsilon, t_0 + \varepsilon]$.

Essentially, this says that if $\mathbf{f}$ is sufficiently nice then the ODE has a unique solution for each initial condition. The result follows from the contraction-mapping theorem or via an iterative scheme. If $\mathbf{f}$ is globally Lipschitz (same Lipschitz constant for all $\mathbf{u} \in \mathbb{R}^n$) and continuous for all $t \in \mathbb{R}$, then the solution exists for all t. For autonomous systems, solution curves $\mathbf{u}(t) \in \mathbb{R}^n$ cannot cross.

While this is theoretical, the theorem is important: there are ODEs without solutions or with non-unique solutions, and for PDEs existence and uniqueness are often much harder (and can fail).

Example 5.2

The ODE given by

$$\frac{\mathrm{d}^2 u}{\mathrm{d}t^2} = \sqrt{u}, \quad u(0) = \frac{\mathrm{d}u}{\mathrm{d}t}(0) = 0,$$

which is equivalent to the first-order system

$$\frac{\mathrm{d}u}{\mathrm{d}t} = v, \quad \frac{\mathrm{d}v}{\mathrm{d}t} = \sqrt{u}, \quad u(0) = v(0) = 0,$$

is a famous example of a system with non-unique solutions. Namely, for any $T > 0$, there are infinitely many solutions to this equation given by

$$u(t) = \begin{cases} 0 & t < T, \\ \frac{1}{144}(t - T)^4 & t \geq T, \end{cases}$$

in addition to the solution $u(t) = 0$. This is also known as Norton's Dome, which has an amusing interpretation as a model in Newtonian mechanics that breaks the notion of causality, as it represents a situation where a particle can spontaneously start moving after an arbitrary amount of time.

5.2 Finite Difference Methods

Moving to practical questions, we now consider how to approximate solutions to the general ODE system using a computer. We have already seen in the previous chapter how to approximate the derivative using what are known as **finite differences.** These approximations are the main ingredients we will use to develop our numerical approaches.

The most well-known numerical method is commonly referred to as the **forward Euler method**, which is given by taking the forward difference operator on the left-hand side of the ODE system and rearranging the equation to obtain:

$$\mathbf{u}(t + \Delta t) = \mathbf{u}(t) + \Delta t \mathbf{f}(\mathbf{u}(t)),$$

where we are now using Δt to denote a small time step, rather than h. This is essentially the same approximation for the gradient of a function illustrated in Chapter 4, except now for the solution of an ODE. We can then iterate this formula starting at the initial condition to find an approximate solution at an arbitrary time t.

We can use a more natural notation for this approximation by taking $t \approx n\Delta t$ and letting $\mathbf{u}_n \approx \mathbf{u}(n\Delta t)$. The forward Euler method can then be written as the iteration:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{f}(\mathbf{u}_n).$$

How accurate is this method for approximating the true solution? How can we know that this method is **convergent**; that is, if we take $\Delta t \to 0$, can we ensure that $\mathbf{u}_n \to \mathbf{u}(t)$? These are more subtle issues compared to numerical differentiation, as here we are using previous approximations for each subsequent timestep, and the dynamics of these schemes can play important roles in determining convergence. Let's consider a simple example to illustrate these ideas.

5.2.1 The van der Pol Oscillator

The van der Pol oscillator is given by

$$\frac{\mathrm{d}^2 u}{\mathrm{d}t^2} - C\left(1 - u^2\right)\frac{\mathrm{d}u}{\mathrm{d}t} + u = 0,$$

which can be converted to the first-order system:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = v, \quad \frac{\mathrm{d}v}{\mathrm{d}t} = C\left(1 - u^2\right)\frac{\mathrm{d}u}{\mathrm{d}t} - u.$$

For $C = 0$, this is the simple harmonic oscillator. For $C > 0$, the extra term means that for $u > 1$, the oscillations are damped, but for $u < 1$, there is an extra force due to "negative damping" driving the system away from $u = 0$.

We can first solve the simple case of $C = 0$ using the forward Euler scheme. The code for this can be compared with a high-order scheme that MATLAB has built-in called `ode45`.

```
% Forward Euler implementation for the simple harmonic oscillator
% Solving the van der Pol equation using Matlab's ode45 command

C = 0; % Parameter C in the model.
% The ODE rhs function as an "anonymous function".
f = @(t,u)  [u(2); -C*(u(1)^2 - 1).*u(2) - u(1)];

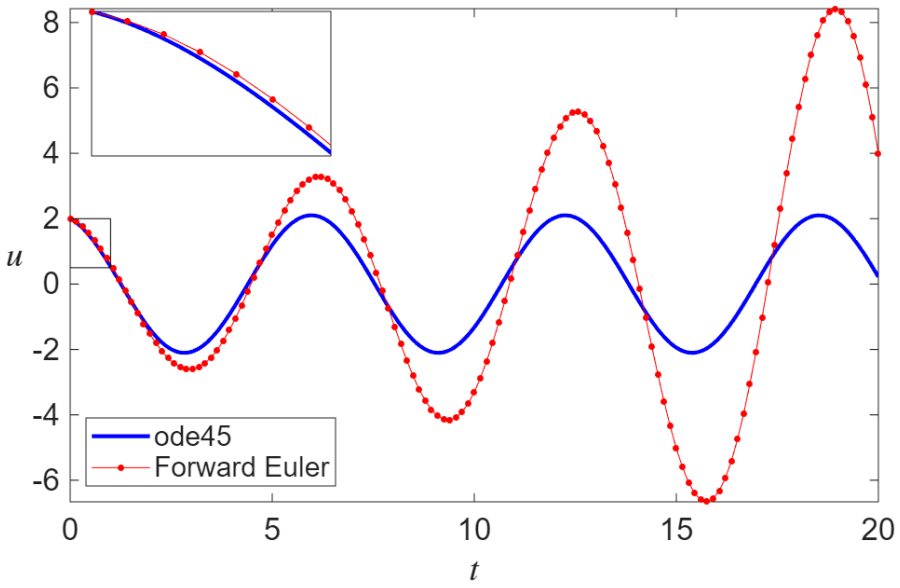
U0 = [2; -0.65];  % Initial condition
tspan = linspace(0,20,1e3); % Time span

% Set low tolerances to ensure an accurate solution
options = odeset('RelTol',1e-11,'AbsTol',1e-11);
[~,u_ode45] = ode45(f, tspan, U0);

% Forward Euler setup (timestep, vector of solution points etc)
dt = 0.15; n = round(tspan(end)/dt); u_Euler = NaN(2,n);

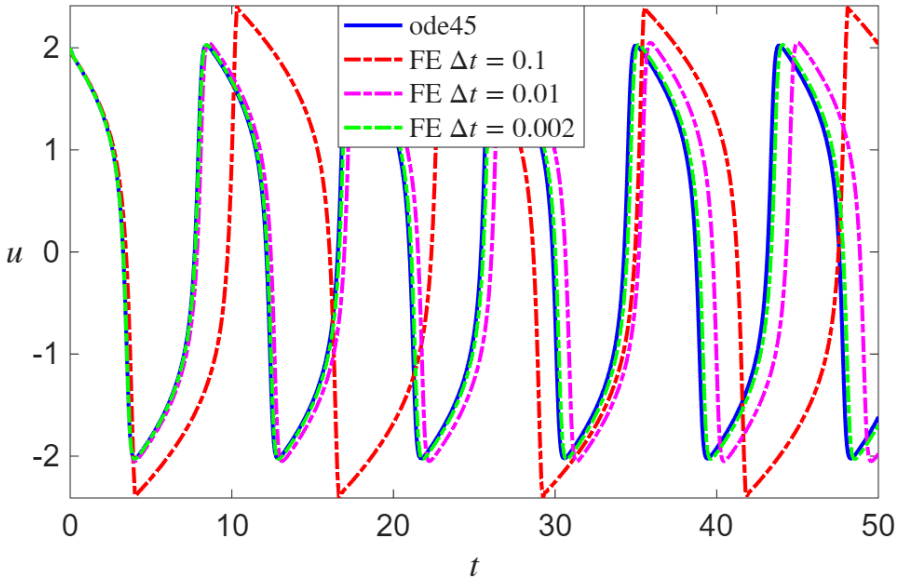
u_Euler(:,1) = U0; % Initial condition for Euler method
for i = 1:n-1
    t = (i-1) * dt; % Current time (NB: Unused currently!)
    u_Euler(:,i+1) = u_Euler(:,i) + dt * f(t, u_Euler(:,i));
end
```

Running this code and plotting the outputs, we obtain the following graph:



The inset shows that the first few steps of the method match the solution reasonably well. However, over time it appears that the error builds up, and the amplitudes grow (despite the correct solution having the same amplitude for all time). Reducing the timestep will improve this, but eventually the amplitude will always begin to grow, at a rate which will become approximately exponential.

We can see how this scheme behaves with the timestep more clearly by considering the nonlinear van der Pol oscillator with $C = 3$, shown below for four different choices of timestep Δt:



We observe convergence towards a similar solution profile as Δt decreases. However, there is still a buildup of error as t increases, meaning that we will need to consider local errors over one timestep, as well as global errors over iterative schemes for many timesteps.

We can formalize these ideas in terms of the **local truncation error (LTE),** which is essentially how much the approximation given in the forward Euler method fails to exactly satisfy the ODE. We can compute this by substituting in the true solution, $\mathbf{u}(t)$, and using Taylor series expansions to determine the error.

Example 5.3

LTE of forward Euler
Expanding $\mathbf{u}(t)$ in a Taylor series, we find:

$$\mathbf{u}_{n+1} = \mathbf{u}(t + \Delta t) = \mathbf{u}(t) + \Delta t \frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} + O(\Delta t^2) = \mathbf{u}_n + \Delta t \mathbf{f}(\mathbf{u}_n),$$

which implies that this method has an LTE of $O(\Delta t^2)$. Note, however, that we can do precisely the same calculation before rearranging (via the approximation of the derivative directly) as:

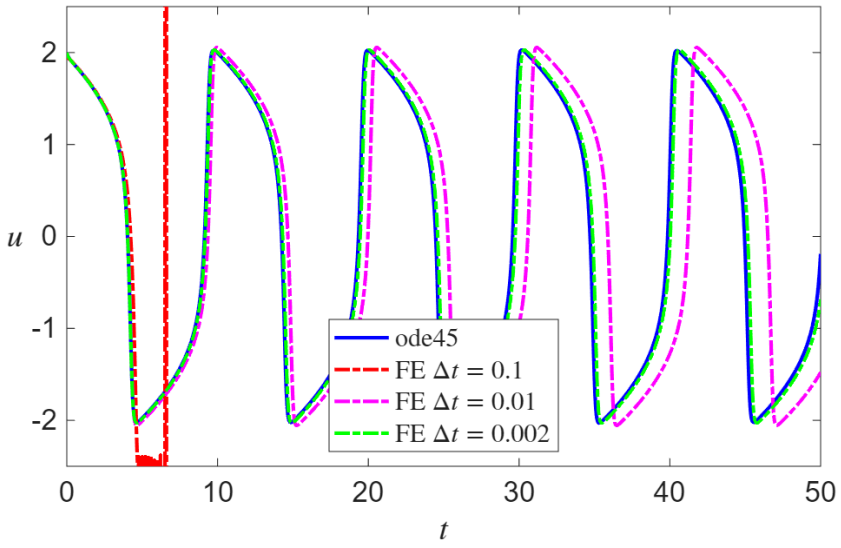
$$\frac{\mathbf{u}_{n+1} - \mathbf{u}_n}{\Delta t} = \frac{\mathbf{u}(t + \Delta t) - \mathbf{u}(t)}{\Delta t} = \frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} + O(\Delta t) = \mathbf{f}(\mathbf{u}(t)),$$

which implies an LTE of $O(\Delta t)$. These two definitions are both used, despite being somewhat inconsistent. We will adopt the former definition, which is sometimes called the **single-step error.**

An integration scheme which has an LTE of the form $O(\Delta t)$ or smaller is called **consistent.** Essentially, a consistent scheme is one where the approximations are equivalent to a collection of Taylor series approximations, and hence, subject to various smoothness assumptions, we expect to be able to make the error tend to 0 for small enough time steps. However, consistency is not enough to ensure that a numerical scheme converges to the analytical solution of the original ODE.

5.3 Stability of Finite Difference Schemes

A consistent scheme may fail to converge to the true solution if the method is not stable. To illustrate stability, we can explore exactly the plot above of the van der Pol oscillator, but for $\$C = 4$ instead. The code gives us this output:



The solutions for $\Delta t \leq 0.01$ appear very much as they did before, but the solution for $\Delta t = 0.1$ now rapidly increases. MATLAB says that it has reached $u_{70} \approx 10^{172}$, meaning that after 70 timesteps (i.e., $t \approx 7$), it has blown up in such a way that numerical calculations on these values are no longer meaningful. This illustrates a general property of numerical methods for differential equations known as **stability**, which is essentially the idea that we want numerical methods not to make successive iterations worse by compounding small errors. To be precise, let's focus on a particular equation which serves as a model for this phenomenon.

5.3.1 The Dahlquist Problem

We consider the **Dahlquist test problem**, which is the following simple linear equation:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = \lambda u,$$

where $\lambda \in \mathbb{C}$ is a given complex parameter. While this equation is simple, it gives insight into any solution of our original problem if we "linearize" the equations around a single point in time. Importantly, this problem also allows us to make analytical progress in determining the stability of a general numerical scheme. Analytically, we expect the ODE to have decaying solutions for $\Re(\lambda) < 0$, but the iterations themselves can sometimes cause the opposite behavior of solutions growing without bound. It is easiest to see this by working with an example.

Example 5.4

Stability of forward Euler

Let's apply the forward Euler method to the Dahlquist problem. We have the iterations:

$$u_{n+1} = u_n + \Delta t \lambda u_n = (1 + \Delta t \lambda) u_n = (1 + \Delta t \lambda)^n u_0,$$

where we have explicitly "solved" the difference equation by iterating it back to the initial condition. Importantly, this will now blow up if $|1 + \Delta t \lambda| \geq 1$, meaning that for real λ, stability requires

$$-2 < \Delta t \lambda < 0.$$

Example 5.5

A consistent but unstable method
Consider the following numerical method for the test problem:

$$u_{n+1} = -4u_n + 5u_{n-1} + \Delta t(4\lambda u_n + 2\lambda u_{n-1}),$$

which can be rewritten as:

$$u_{n+1} = 4(\Delta t \lambda - 1)u_n + (5 + 2\Delta t \lambda)u_{n-1}.$$

We can solve this difference equation by rewriting it as a system, or by noting that it must have two solutions of the form $u_n = C\mu^n$ for some μ (analogous to how linear ODEs have solutions of the form $e^{\mu t}$). Substituting this in, and dividing by u_n, we find that μ satisfies:

$$\mu^2 = 4(\Delta t \lambda - 1)\mu + (5 + 2\Delta t \lambda),$$

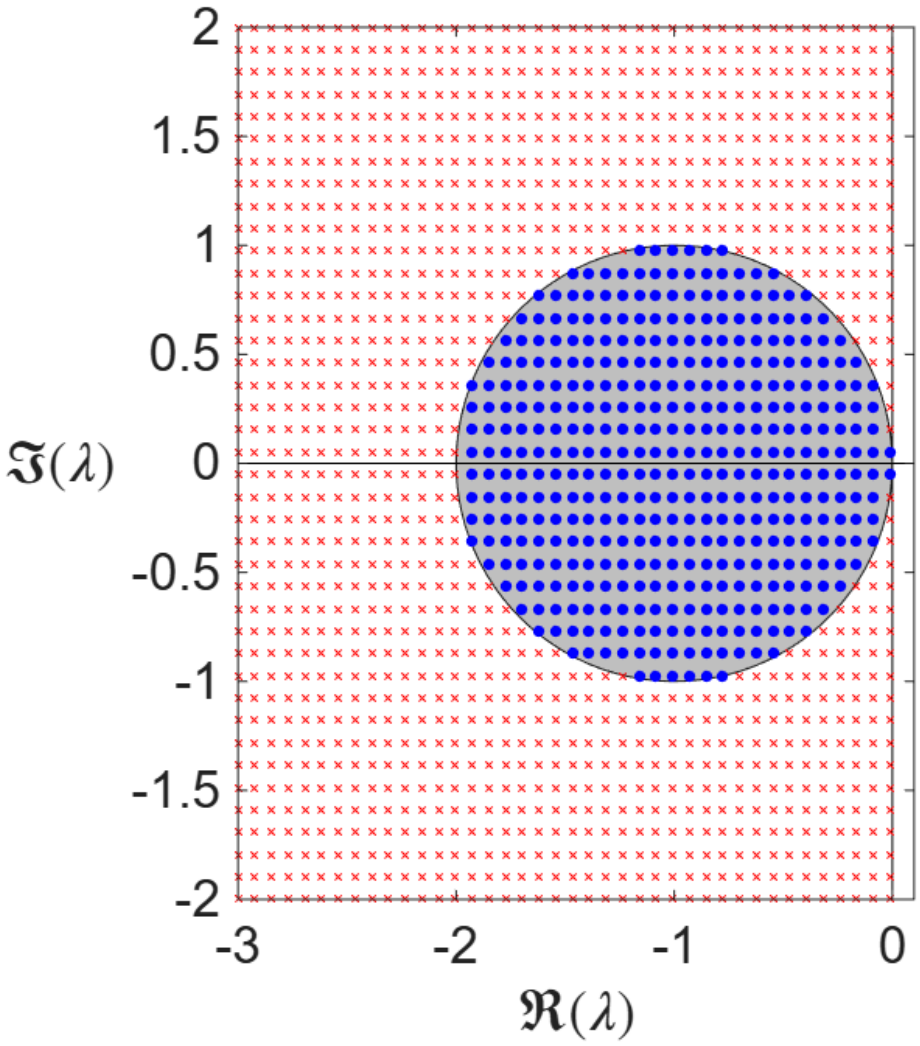
which simplifies to:

$$\mu = 2(\Delta t \lambda - 1) \pm \sqrt{4(\Delta t \lambda)^2 - 6\Delta t \lambda + 9}.$$

For $|\Delta t| \ll 1$, we see that one of these solutions approaches $\mu \approx -5$, so that $u_n \sim (-5)^n$, implying that we expect this scheme to be unstable for all λ as long as Δt is sufficiently small. With more work, we can in fact show that at least one of the solutions μ will always have $|\mu| \geq \sqrt{19} > 1$ for all $\lambda \in \mathbb{C}$ and $\Delta t \in \mathbb{R}$, so this scheme is *always* unstable.

5.3.2 Stability Regions and A-Stability

For a given numerical method, we can use the Dahlquist problem to define **stability regions** for complex λ. These essentially tell us for what values of λ we expect the numerical method to behave well. Importantly, this also gives us insight into how they impact the growth of truncation errors, which is why these stability regions apply to the method more generally.



We see that, as long as $\Re(\lambda) < 0$, then for sufficiently small Δt, this scheme is stable. In practice, this criterion means that we have to take extremely small time steps. A numerical method that is stable for all $\Re(\lambda) < 0$ is called **A-Stable**, which is a very desirable property. For such methods, we can (usually) mostly ignore aspects of stability and decide on suitable timesteps based solely on accuracy considerations.

The simplest example of an A-Stable alternative is to consider a slight modification of the Euler scheme, where instead of taking a forward derivative, we take a backward derivative. This results in the **backward Euler** scheme:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{f}(\mathbf{u}_{n+1}).$$

While this method looks very similar, in practice it is harder to implement as it is an **implicit** scheme, where the unknown $\mathbf{u}_{n+1}$ appears inside of the function on the right-hand side. This

means that to actually take a step, we have to solve the problem:

$$\mathbf{G}(\mathbf{u}_{n+1}) := \mathbf{u}_{n+1} - \Delta t \mathbf{f}(\mathbf{u}_{n+1}) = \mathbf{u}_n \implies \mathbf{u}_{n+1} = \mathbf{G}^{-1}(\mathbf{u}_n),$$

where $\mathbf{G}^{-1}$ is the inverse of the function $\mathbf{G}$. In practice, this becomes a (possibly high-dimensional) rootfinding problem, though for some special functions $\mathbf{f}$, it may be possible to evaluate it directly (e.g., when $\mathbf{f}$ is a linear function).

You can show (try this yourself) that this scheme has the same LTE as the forward Euler method. But its real advantage is in its stability.

Example 5.6

Stability of backward Euler
As before, we have the iterations:

$$u_{n+1} - \Delta t \lambda u_{n+1} = u_n \implies u_{n+1} = \frac{1}{1 - \Delta t \lambda} u_n = \left| \frac{1}{1 - \Delta t \lambda} \right|^n u_0.$$

Now, for any complex number z with $\Re(z) < 0$, we have that $|1 - z|^2 = 1 - 2\Re(z) + z^2 > 0$. Hence, as $\Delta t > 0$, if $\Re(\lambda) < 0$, then we have that this scheme is A-Stable.

5.3.3 Convergence

In general, it is difficult to prove that a numerical scheme converges in a suitable sense to the actual solution of an ODE system, as this requires some heavy machinery from analysis involving how the solutions of difference equations can embed into spaces of functions properly. One way to short-circuit these deep issues, however, is the use of the following theorem.

Theorem 5.2

Lax Equivalence Theorem
Assume that the system satisfies the conditions of the Picard–Lindelöf theorem. A finite difference method converges (in a suitable sense) to the unique solution of the system

$$\dot{\mathbf{u}} = \frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = \mathbf{f}(t, \mathbf{u}), \quad \mathbf{u}(t_0) = \mathbf{u}_0 \in \mathbb{R}^N,$$

as $\Delta t \to 0$ if the method is both consistent and stable.

One important caveat here is that this theorem only holds in the theoretical case of infinite-precision arithmetic; rounding error can build up if we need to simulate the equation for a long time interval, or to use very small time steps (as we have to for methods that are not A-stable). There is a related concept known as **global truncation error**, which is one way to keep track of how the LTE can build up over many timesteps, but we will not discuss this further except to say that the LTE being sufficiently small is usually the most desirable measure of accuracy.

5.4 Higher-Order Methods

So far we have discussed just the Euler stepping methods, which both have an LTE of $O(\Delta t)$. There are a variety of higher-order methods commonly used, which broadly fall into classes of **explicit** or **implicit** as described before, as well as **single-step** and **multi-step**. Here we will focus on explicit methods and give an example from each class. For simplicity of notation, let's consider a single ODE (i.e., $N = 1$), and let's only consider autonomous problems where $\dot{u} = f(u)$. While the basic ideas will be essentially identical for larger systems, adding in explicit time-dependence will make the formulae messier (but conceptually the same).

5.4.1 Single-step (Runge-Kutta) methods

One can take a single integration step broken up over different points within an interval. Let $t_n = n\Delta t$ be the current time corresponding to the approximate solution u_n. Then these methods can be written as:

$$u_{n+1} = u_n + \Delta t \sum_{i=1}^{s} a_i k_i, \quad k_1 = f(u_n), \quad k_i = f\left(u_n + \Delta t \sum_{j=1}^{i-1} a_{ij} k_j\right).$$

These schemes are typically referred to as explicit Runge-Kutta methods (specifically having order s). They involve taking a single timestep using multiple function evaluations, essentially in order to get a better approximation of the derivative. They are derived in a similar way to how higher-order differentiation schemes can be derived; namely by taking different points within the interval of one timestep, and considering function evaluations and Taylor series on those points. It is easiest to see this with an example.

Example 5.7

Derivation of RK2 Methods
Let's consider three time points $t_n, t_n + \Delta t/2$, and $t_{n+1} = t_n + \Delta t$. In this case, we can expand the Taylor series for $u(t_n + \Delta t)$ as:

$$\begin{aligned} u(t_n + \Delta t) &= u(t_n) + \Delta t \dot{u}(t_n) + \frac{\Delta t^2}{2}\ddot{u}(t_n) + O(\Delta t^3) \\ &= u(t_n) + \Delta t f(u(t_n)) + \frac{\Delta t^2}{2} f_u(u(t_n))\dot{u}(t_n) + O(\Delta t^3), \end{aligned}$$

where we have used the chain rule to evaluate $\ddot{u}(t_n)$. The goal will be to derive a formula which includes the $O(\Delta t^2)$ term explicitly, so that the method is exact up to this order.

We perform a Taylor expansion of f evaluated at the midpoint as:

$$\begin{aligned} &f(u(t_n) + (\Delta t/2) f(u(t_n))) \\ =&f(u(t_n)) + \frac{\Delta t}{2} f_u(u(t_n))\dot{u}(t_n) + O(\Delta t^2). \end{aligned}$$

Now, if we multiply this expansion by Δt, we see that it exactly matches the expansion of $u(t + \Delta t)$ up to $O(\Delta t^2)$. So if we consider the numerical scheme:

$$u_{n+1} = u_n + \Delta t f(u_n + (\Delta t/2) f(u_n)),$$

we see that this scheme has an LTE of $O(\Delta t^2)$, which is an improvement over the Euler methods at the cost of one additional function evaluation.

One can work out that this method also very slightly improves the region of numerical stability compared to forward Euler, though it is not an A-stable method and hence will require sufficiently small timesteps to be stable.

5.4.2 Linear multistep methods

One disadvantage of the Runge-Kutta methods is that they require multiple function evaluations to arrive at a single timestep with a desired accuracy. A different approach is to use information from multiple timesteps to predict the next one in such a way that higher-order accuracy is maintained. Such methods are known as linear multistep methods, which have the general form:

$$u_{n+s} = \Delta t \sum_{j=0}^{s} \beta_j f(u_{n+j}),$$

where β_j are constants chosen to ensure the scheme is consistent, has a good LTE, etc. If $\beta_k = 0$, then the method is explicit, as everything on the right-hand side involves earlier function values. An example of such a method is the two-step **Adams-Bashforth** scheme given by:

$$u_{n+2} = u_{n+1} + \frac{\Delta t}{2} \left(3f(u_{n+1}) - f(u_n)\right).$$

One can work out that this method is second-order accurate, just like the RK2 method above, but only requires one new function evaluation at each timestep as opposed to two.

One disadvantage to these methods is that the initial-value problem given in the ODE system is not enough data to start them, as it only contains the solution at one time point u_0. So a typical method is to use a high-order Runge-Kutta scheme to start a multistep method.

5.5 Beyond the Basics

What we have covered here is a very brief sketch of the core ideas which were all mostly well-understood before 1960 or so. Since then, enormous progress has been made on a number of topics to develop increasingly sophisticated algorithms for simulating differential equations. A key idea from a "user" perspective is to be aware of how the stability of a method is important, but often difficult to ensure. There are no explicit methods of the forms described which are A-stable, and all implicit methods require some form of (often difficult) rootfinding.

> **Note**
>
> A **stiff** differential equation is one which is difficult to integrate from a stability perspective; often these correspond to equations which have large (in magnitude) values of λ when linearized, though making the idea precise to cover all known cases is not so easy.

6 Further Topics

6.1 Partial Differential Equations

How can we solve Partial Differential Equations numerically?

We have seen how to approximate solutions to systems of Ordinary Differential Equations (ODEs) using various timestepping schemes. Many Partial Differential Equations (PDEs) can also be reduced to large systems of ODEs using the **method of lines**. In this section, we will illustrate this idea for one of the simplest PDEs: the one-dimensional heat equation.

> **Note**
>
> As in the last Chapter, this topic is also vast. Entire fields of science, such as numerical General Relativity, Computational Fluid Mechanics, and Finite Element Analysis, are essentially subfields within the broad topic of Numerical PDEs. Each of those topics has dozens or hundreds of textbooks written on particular aspects of it. We will therefore be extremely cursory, giving a flavour of some ideas that underlie VisualPDE.com and many standard numerical PDE tools, but leaving a "deep" exploration for future study.

6.1.1 The heat equation and boundary conditions

The one-dimensional heat equation describes the diffusion of temperature $u(x,t)$ in a homogeneous medium:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, L], \quad t > 0,$$

where $\alpha > 0$ is the diffusivity constant. To fully specify the problem, we must also prescribe initial and boundary conditions.

We will use an initial condition:

$$u(x, 0) = u_0(x),$$

and homogeneous **Neumann boundary conditions**:

$$\frac{\partial}{\partial x} u(0, t) = \frac{\partial}{\partial x} u(L, t) = 0,$$

which correspond physically to an insulated domain where no heat flows in or out through the boundaries.

6.1.2 Spatial discretisation: finite differences

To apply the method of lines, we first discretise the spatial variable x using a uniform grid of N nodes. Let:

$$x_j = (j-1)\Delta x, \quad j = 1, 2, \ldots, N, \quad \Delta x = L/(N-1),$$

so that $x_1 = 0$ and $x_N = L$, and define:

$$u_j(t) \approx u(x_j, t).$$

The spatial derivatives in the heat equation can be approximated using finite differences. For the second derivative, a centred difference gives:

$$\frac{\partial^2}{\partial x^2} u(x_j, t) \approx \frac{u_{j-1}(t) - 2u_j(t) + u_{j+1}(t)}{(\Delta x)^2}.$$

For the Neumann boundary conditions, we use the concept of **ghost nodes**, introducing extra nodes $x_0 = -\Delta x$ and $x_{N+1} = L + \Delta x$ just outside the domain, with values u_0 and u_{N+1}. To determine these, we approximate the first derivative at the endpoints using forward and backward differences:

$$\frac{u_1 - u_0}{\Delta x} = 0 \quad \Rightarrow \quad u_1 = u_0,$$

$$\frac{u_{N+1} - u_N}{\Delta x} = 0 \quad \Rightarrow \quad u_{N+1} = u_N.$$

6.1.3 The method of lines formulation

Plugging the finite difference approximation into the heat equation leads to the large system of ODEs for the approximate solution at each gridpoint, $u_j(t)$:

$$\frac{d}{dt} u_j(t) = \alpha \frac{u_{j-1}(t) - 2u_j(t) + u_{j+1}(t)}{(\Delta x)^2}, \quad j = 1, 2, \ldots N,$$

where we recall that $u_0 = u_1$ and $u_{N+1} = u_N$, so that two of these equations have a slightly different form.

We can also write this system for the vector:

$$\mathbf{u}(t) = (u_1(t), u_2(t), \ldots, u_N(t))^\top.$$

We obtain:

$$\frac{d}{dt}\mathbf{u}(t) = \alpha \mathbf{A}\mathbf{u}(t),$$

where $\mathbf{A}$ is the $N \times N$ matrix defined by:

$$\mathbf{A} = \frac{1}{(\Delta x)^2} \begin{pmatrix} -1 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & \cdots & 0 \\ 0 & 1 & -2 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & -2 & 1 \\ 0 & \cdots & 0 & 0 & 1 & -1 \end{pmatrix}.$$

This matrix represents the discrete Laplacian operator with Neumann boundary conditions.

6.1.4 Time discretisation and stability

For example, applying the forward Euler method gives:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\, \alpha \mathbf{A}\mathbf{u}_n.$$

This scheme is simple to implement, but its stability depends strongly on the size of Δt. For the one-dimensional heat equation, it can be shown that stability requires:

$$\Delta t \leq \frac{(\Delta x)^2}{2\alpha}.$$

Violating this condition leads to rapidly growing (unphysical) numerical oscillations.

Alternatively, we can use the backward Euler scheme, which gives:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\, \alpha \mathbf{A}\mathbf{u}_{n+1}.$$

This is an implicit system, requiring the solution of a linear system at each timestep:

$$(\mathbf{I} - \Delta t \alpha \mathbf{A})\mathbf{u}_{n+1} = \mathbf{u}_n.$$

However, the backward Euler method is A-stable, and therefore unconditionally stable for the heat equation. This means that we can take arbitrarily large timesteps without introducing instability, though accuracy may still require small Δt.

6.1.5 Summary

The method of lines converts a PDE into a large system of ODEs by discretising only in space. The resulting ODE system can then be integrated using any of the timestepping methods from Chapter 5. The stability and accuracy of the full PDE scheme are determined by both the spatial discretisation and the ODE integrator used in time.

The same approach applies to **reaction–diffusion systems**, in which several chemical or biological species $u, v, \ldots$ diffuse and react, for example

$$\frac{\partial u}{\partial t} = D_u \frac{\partial^2 u}{\partial x^2} + f(u, v), \qquad \frac{\partial v}{\partial t} = D_v \frac{\partial^2 v}{\partial x^2} + g(u, v).$$

Discretising in space gives $\mathbf{u}' = D_u \mathbf{A}\mathbf{u} + f(\mathbf{u}, \mathbf{v})$ and $\mathbf{v}' = D_v \mathbf{A}\mathbf{v} + g(\mathbf{u}, \mathbf{v})$, where f and g act componentwise. The eigenvalues of $\mathbf{A}$ are as large as $4/(\Delta x)^2$ in magnitude, so the diffusion terms force an explicit method to take $\Delta t = O((\Delta x)^2)$, even when the solution itself changes slowly. Such systems are therefore **stiff** (Chapter 5), and are usually integrated with implicit, adaptive solvers designed for stiff problems, such as MATLAB's `ode15s`. Despite their simple form, reaction–diffusion systems can produce remarkably rich behaviour, including spatial patterns, travelling waves and spatiotemporal chaos, which you can explore on VisualPDE.com.

On the Blackboard page are several code demos for the Heat equation, using both the matrix approach and a for-loop approach to discretising this equation. A demo is also provided for the Schnakenberg system of reaction-diffusion equations.

6.2 Monte Carlo Methods

How can random numbers compute things that are not random?

Many quantities in applied mathematics are averages: the expected payoff of a financial contract, the mean energy of a magnet in statistical physics, the probability that a structure fails under random loads. Each of these is an integral, and often an integral over a space of very high dimension. The **Monte Carlo method** estimates such integrals by averaging the integrand at randomly chosen points. It was developed by Stan Ulam, John von Neumann and Nicholas Metropolis at Los Alamos in the 1940s to simulate neutron transport, and is named after the casinos of Monte Carlo.

In this section we will see how computers produce random numbers, how to turn an average of random samples into an estimate with an error bar, and, most importantly, why this apparently crude method is the method of choice for integrals in high dimensions.

6.2.1 Pseudorandom numbers

A computer executes deterministic instructions, so it cannot compute truly random numbers. Instead it produces **pseudorandom numbers**: a deterministic sequence $u_1, u_2, \ldots \in (0,1)$ designed to be statistically indistinguishable from independent samples of the uniform distribution on $[0,1]$. Most generators have the form

$$s_{i+1} = g(s_i), \qquad u_i = h(s_i),$$

where s_i is the internal **state** of the generator, g updates the state, and h converts the state into a number in $(0,1)$. The initial state s_0 is called the **seed**. Since the state can take only finitely many values, the sequence must eventually repeat, and the length of the cycle is called the **period**.

Determinism is a feature rather than a bug: running the same code with the same seed reproduces exactly the same "random" numbers, which makes simulations repeatable and much easier to debug. This is why the noisy-data demonstration in Chapter 4 begins with `rng(1)`.

What should we ask of the sequence? Each u_i should look uniformly distributed, but this is the easy part: the sequence $0.01, 0.02, \ldots, 0.99$ has a perfect histogram and is not random at all. The essential property is **independence**: knowing $u_1, \ldots, u_{i-1}$ should tell us nothing about u_i. In particular, pairs (u_i, u_{i+1}) should fill the unit square uniformly, triples (u_i, u_{i+1}, u_{i+2}) should fill the unit cube uniformly, and so on.

Definition 6.1

A **linear congruential generator (LCG)** is an iteration of the form

$$x_{i+1} = (a x_i + c) \bmod m, \qquad u_{i+1} = \frac{x_{i+1}}{m},$$

where the multiplier a, the increment c and the modulus m are integers, and $y \bmod m$ denotes the remainder when y is divided by m.

Example 6.1

With $a = 6$, $c = 0$, $m = 11$ and seed $x_0 = 1$ we obtain

$$1, 6, 3, 7, 9, 10, 5, 8, 4, 2, 1, 6, \ldots,$$

which visits every nonzero remainder before repeating (period 10). With $a = 3$ instead we obtain $1, 3, 9, 5, 4, 1, \ldots$, which has period only 5.

The period of an LCG is at most m, and number theory tells us how to choose a, c and m to achieve this maximum (see Knuth, *The Art of Computer Programming*, Volume 2). But a long period is not enough, as the following cautionary tale shows.

Example 6.2: RANDU

The LCG with $a = 65539 = 2^{16} + 3$, $c = 0$ and $m = 2^{31}$, known as RANDU, was supplied with IBM mainframes in the 1960s and was very widely used for scientific simulation. Its output passes a casual inspection: the histogram of the u_i looks uniform, and so does a 3D scatter plot of consecutive triples (u_i, u_{i+1}, u_{i+2}). But if we rotate the cube, all of the triples turn out to lie on just 15 parallel planes!

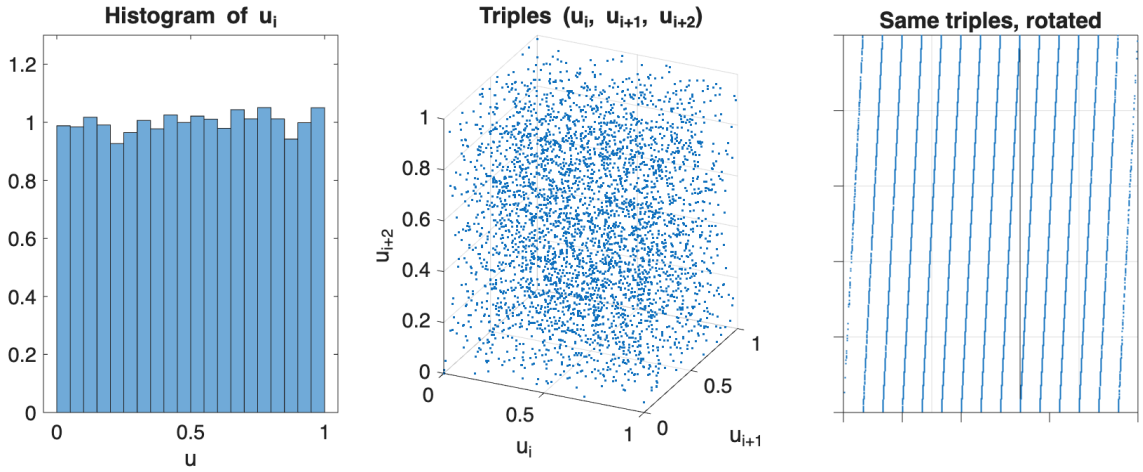


Figure 6.1: RANDU output: histogram of 3×10^4 values (left), consecutive triples (middle), and the same triples viewed from a different angle (right).

To see why, note that $a^2 = 2^{32} + 6 \cdot 2^{16} + 9 = 2^{32} + 6a - 9$. Since 2^{32} is a multiple of 2^{31},

$$x_{i+2} = a^2 x_i \bmod 2^{31} = (6a x_i - 9x_i) \bmod 2^{31} = (6x_{i+1} - 9x_i) \bmod 2^{31}.$$

Hence $9x_i - 6x_{i+1} + x_{i+2}$ is an integer multiple of 2^{31}, and dividing by 2^{31} gives

$$9u_i - 6u_{i+1} + u_{i+2} = k \in \mathbb{Z}.$$

Since each $u_i \in (0, 1)$, the left-hand side lies in $(-6, 10)$, so $k \in \{-5, -4, \ldots, 9\}$. Every triple lies on one of these 15 planes, and any simulation that uses three consecutive numbers (for example, to place a random point in 3D) only ever explores those planes.

```
n = 3e4;
x = zeros(n, 1);  x(1) = 1;                % seed
for i = 1:n-1
    x(i+1) = mod(65539*x(i), 2^31);        % exact: 65539*x(i) < 2^53 (Chapter 1)
end
u = x/2^31;
plot3(u(1:end-2), u(2:end-1), u(3:end), '.', 'MarkerSize', 2)
view([2 3 0])                              % look along the planes
```

RANDU is a particularly bad choice of parameters, but the defect is shared by every LCG: Marsaglia showed in 1968 that consecutive d-tuples from an LCG always lie on at most $(d!\,m)^{1/d}$ parallel hyperplanes.

6.2.2 Testing generators: the chi-squared test

How can we tell whether a generator is any good? We use statistical hypothesis tests of the null hypothesis

> H_0: the numbers $u_1, \dots, u_n$ are independent samples from the uniform distribution on $[0, 1]$.

A test can never show that H_0 is true (we know that the numbers are not random!), but a generator that fails a test has a detectable non-random pattern. The simplest such test is the chi-squared goodness-of-fit test.

Definition 6.2

Let $Z_1, \dots, Z_\nu$ be independent standard normal random variables. The distribution of $Z_1^2 + \cdots + Z_\nu^2$ is called the **chi-squared distribution** with ν degrees of freedom, written χ^2_ν.

To test for uniformity, partition $[0, 1]$ into k equal bins $[0, \frac{1}{k}), [\frac{1}{k}, \frac{2}{k}), \dots, [\frac{k-1}{k}, 1]$ and let n_j be the number of samples in bin j. Under H_0, each bin has expected count n/k, and the **chi-squared test statistic** measures how far the observed counts are from this:

$$T = \sum_{j=1}^{k} \frac{(n_j - n/k)^2}{n/k} = \frac{k}{n} \sum_{j=1}^{k} \left(n_j - \frac{n}{k} \right)^2 .$$

Theorem 6.1

Under H_0, T converges in distribution to χ^2_{k-1} as $n \to \infty$.

We will not prove this, but here is the intuition. For large n, each count n_j is approximately normal with mean n/k and variance approximately n/k, so each term in T is approximately the square of a standard normal. The terms are not independent, since $n_1 + \cdots + n_k = n$, and this single constraint removes one degree of freedom.

The test is carried out at a chosen **significance level** α, typically $\alpha = 0.05$. Let c be the $(1 - \alpha)$-quantile of χ^2_{k-1}, computed in MATLAB as `chi2inv(1 - alpha, k - 1)`. If $T > c$ we reject H_0 (the sample **fails** the test), and otherwise the sample **passes**. If H_0 is true, the sample wrongly fails with probability approximately α. The limiting distribution is a good approximation when every expected count n/k is at least about 5.

Example 6.3: Chi-squared test with ten bins

A generator produces $n = 1000$ numbers, which we sort into $k = 10$ bins, each with expected count 100. Suppose the observed counts are:

Bin j	1	2	3	4	5	6	7	8	9	10
n_j	98	104	112	107	94	95	108	97	96	89

Then

$$T = \frac{10}{1000}\left[(-2)^2 + 4^2 + 12^2 + 7^2 + (-6)^2 + (-5)^2 + 8^2 + (-3)^2 + (-4)^2 + (-11)^2\right] = \frac{484}{100} = 4.84.$$

The critical value at significance level $\alpha = 0.05$ is `chi2inv(0.95, 9)` ≈ 16.92. Since $4.84 < 16.92$, the sample passes the test.

Example 6.4: Catching RANDU

Take $n = 3 \times 10^5$ numbers $u_1, \ldots, u_n$ from RANDU with seed $x_0 = 1$. With $k = 10$ bins, the chi-squared test gives $T \approx 4.3$, well below the critical value 16.92, so RANDU passes. The individual numbers really do look uniform.

The same idea applies to triples. Group the numbers into 10^5 non-overlapping triples $(u_1, u_2, u_3), (u_4, u_5, u_6), \ldots$, divide the unit cube into $k^3 = 1000$ small cubes of side 0.1, and count the triples in each. Under H_0, each small cube has expected count 100, and T is approximately χ^2_{999}, with critical value `chi2inv(0.95, 999)` ≈ 1074. For RANDU the counts range from 68 to 145 and $T \approx 1593$, so RANDU fails decisively. The Mersenne Twister gives $T \approx 1043$ on the same test and passes.

The lesson is that tests must probe *independence*, not just the histogram of single values.

i Modern generators

MATLAB's default generator is the **Mersenne Twister** (Matsumoto and Nishimura, 1998), which has period $2^{19937} - 1$ and is equidistributed in up to 623 dimensions. Newer families include PCG (the default in Python's NumPy) and xoshiro (the default in Julia). Large simulations now run on thousands of GPU cores at once, which favours **counter-based** generators such as Philox (`rng(seed, 'philox')` in MATLAB). In these generators $u_i = h(\text{key}, i)$ is a scrambled version of the counter i, so each core can jump directly to its own part of the sequence.

Generators are vetted by large batteries of statistical tests, such as the TestU01 suite, which apply hypothesis tests like the one above (and many others, such as the Kolmogorov–Smirnov test) to hundreds of different features of the output. The practical lessons are:

never write your own generator, always record your seed, and make sure that parallel computations use independent streams of random numbers.

6.2.3 Sampling from other distributions

Recall that a real-valued random variable X has **cumulative distribution function (CDF)** $F(x) = \mathbb{P}[X \le x]$ and, if X is continuous, **probability density function (PDF)** $f = F'$. Generators produce uniform samples, and samples from other distributions are obtained by transforming these.

Theorem 6.2: Inverse transform method

Let U be uniformly distributed on $[0,1]$ and let F be a continuous, strictly increasing CDF. Then $X = F^{-1}(U)$ has CDF F.

Proof. Since F is increasing, $F^{-1}(U) \le x$ if and only if $U \le F(x)$. Hence

$$\mathbb{P}[X \le x] = \mathbb{P}[U \le F(x)] = F(x),$$

because $\mathbb{P}[U \le y] = y$ for $y \in [0,1]$. □

Example 6.5: Exponential distribution

If $X \sim \mathrm{Exp}(\lambda)$, then $F(x) = 1 - e^{-\lambda x}$ for $x \ge 0$. Solving $F(x) = u$ gives $F^{-1}(u) = -\log(1-u)/\lambda$. Since $1 - U$ is also uniform on $[0,1]$, the command `X = -log(rand(n, 1))/lambda` generates n independent samples of X.

Exercise 6.1

The Pareto distribution with parameters $x_m, \alpha > 0$ has PDF $f(x) = \alpha x_m^{\alpha} / x^{\alpha+1}$ for $x \ge x_m$. Find its CDF and show that $x_m(1-U)^{-1/\alpha}$ is Pareto distributed. Generate 10^5 samples in MATLAB and compare a normalised histogram (`histogram(X, 'Normalization', 'pdf')`) with f.

When F^{-1} has no closed form, other transformations are used. For example, the **Box–Muller transform** turns two independent uniform samples U_1, U_2 into two independent standard normal samples $\sqrt{-2\log U_1}\cos(2\pi U_2)$ and $\sqrt{-2\log U_1}\sin(2\pi U_2)$. In practice, built-in functions such as `randn` should be used.

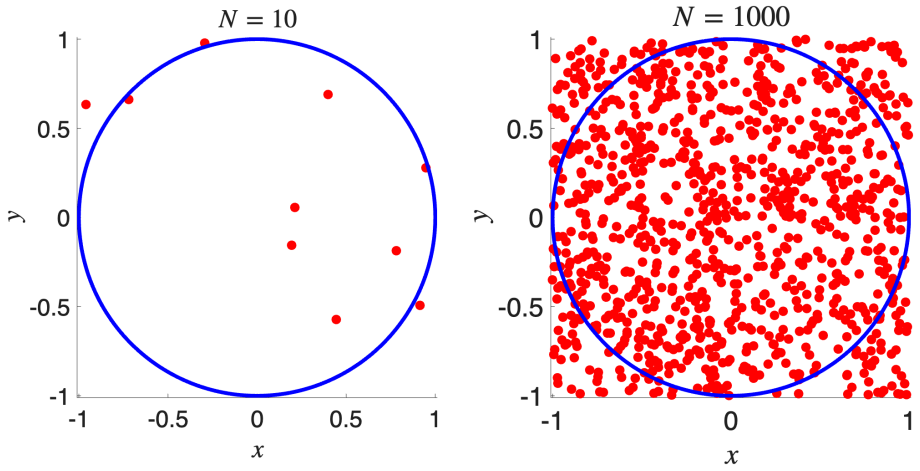
6.2.4 The Monte Carlo estimator

Example 6.6: Estimating π

Suppose (X_i, Y_i) are independent and uniformly distributed on the square $[-1,1]^2$. The probability that a point lands in the unit disc is the ratio of the areas, $\pi/4$. So $4\times$ (fraction of points in the disc) estimates π. In the left panel below, 8 of 10 points fall in the disc, giving the estimate 3.2. With 1000 points (right panel) the estimate improves to about 3.15. In other words, we are estimating the integral

$$\pi = \int_{-1}^{1}\int_{-1}^{1} \mathbb{1}\{x^2 + y^2 \le 1\}\, dy\, dx$$

by averaging the integrand at random points.



More generally, suppose that we want to compute

$$\mu = \mathbb{E}[A(X)] = \int_{\mathbb{R}^d} A(x) f(x)\, dx,$$

where X is a random variable in $\mathbb{R}^d$ with PDF f. In particular, any integral over the unit cube is an expectation: if U is uniformly distributed on $[0,1]^d$ (so $f \equiv 1$ there), then

$$\int_{[0,1]^d} g(x)\, dx = \mathbb{E}[g(U)].$$

Similarly, if U is uniformly distributed on an interval $[a,b]$, it has density $1/(b-a)$ there, so

$$\frac{1}{b-a}\int_a^b g(x)\, dx = \mathbb{E}[g(U)],$$

and samples of U are given by $a + (b-a)$ `rand`.

Definition 6.3

Let $X_1, \ldots, X_n$ be independent random variables with the same distribution as X. The **Monte Carlo estimator** of $\mu = \mathbb{E}[A(X)]$ is the sample average

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} A(X_i).$$

Theorem 6.3: Properties of the Monte Carlo estimator

Suppose $\mathbb{E}|A(X)| < \infty$. Then:

1. $\hat{\mu}_n$ is **unbiased**: $\mathbb{E}[\hat{\mu}_n] = \mu$.
2. $\hat{\mu}_n$ is **consistent**: $\hat{\mu}_n \to \mu$ almost surely as $n \to \infty$ (the strong law of large numbers).
3. If also $\sigma^2 = \mathrm{Var}[A(X)] < \infty$, then $\mathrm{Var}[\hat{\mu}_n] = \sigma^2/n$, and by the central limit theorem

$$\frac{\sqrt{n}}{\sigma}(\hat{\mu}_n - \mu) \to \mathcal{N}(0,1) \quad \text{in distribution as } n \to \infty.$$

Part 1 follows from linearity of expectation, and the variance formula in part 3 holds because the variance of a sum of independent random variables is the sum of their variances. Part 3 is what gives us an **error bar**. For $0 < \alpha < 1$, let $z_{1-\alpha/2}$ be the $(1-\alpha/2)$-quantile of the standard normal distribution, so that $\mathbb{P}[|Z| \le z_{1-\alpha/2}] = 1 - \alpha$ for $Z \sim \mathcal{N}(0,1)$. Then

$$\mathbb{P}\left[|\hat{\mu}_n - \mu| \le z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}}\right] \approx 1 - \alpha \quad \text{for large } n.$$

The most common choices are $z_{0.975} \approx 1.96$ (for $\alpha = 0.05$) and $z_{0.995} \approx 2.576$ (for $\alpha = 0.01$). In MATLAB, $z_{1-\alpha/2}$ is `norminv(1 - alpha/2)`.

We don't usually know σ (if we can't compute μ, we probably can't compute σ either), so in practice we replace it by the sample standard deviation $\hat{\sigma}_n$ of the same samples, where

$$\hat{\sigma}_n^2 = \frac{1}{n-1} \sum_{i=1}^{n} \left(A(X_i) - \hat{\mu}_n\right)^2.$$

The quantity $\hat{\sigma}_n/\sqrt{n}$, which estimates the standard deviation of $\hat{\mu}_n$, is called the **standard error**, and

$$\hat{\mu}_n \pm z_{1-\alpha/2} \frac{\hat{\sigma}_n}{\sqrt{n}}$$

is an approximate $100(1-\alpha)\%$ **confidence interval** for μ. For example, $\hat{\mu}_n \pm 1.96\, \hat{\sigma}_n/\sqrt{n}$ is an approximate 95% confidence interval. Unlike the quadrature rules of Chapter 4, Monte Carlo provides an estimate of its own error at no extra cost.

> **Note**
>
> **Almost sure** convergence means that, with probability one, the sequence of estimates $\hat{\mu}_1, \hat{\mu}_2, \ldots$ produced by a single run converges to μ. It is one of several modes of convergence for random variables that you will meet in probability courses.

The error therefore decreases like $n^{-1/2}$. This is **slow**: to gain one more correct digit we need 100 times as many samples.

Example 6.7: A one-dimensional integral

Let U be uniform on $[0,1]$ and $\mu = \mathbb{E}[U^2] = \int_0^1 x^2\,dx = \tfrac{1}{3}$. Here

$$\sigma^2 = \mathbb{E}[U^4] - \mu^2 = \frac{1}{5} - \frac{1}{9} = \frac{4}{45},$$

so the 95% confidence interval has half-width $1.96\sqrt{4/45}/\sqrt{n} \approx 0.58/\sqrt{n}$.

```
n = 1e4;
gU = rand(n, 1).^2;                    % samples of A(U) = U^2
muHat = mean(gU);
SE = std(gU)/sqrt(n);                  % standard error
fprintf('%.4f +/- %.4f\n', muHat, 1.96*SE)
```

With $n = 10^4$ samples the error is around 6×10^{-3}. By comparison, the composite trapezium rule with 10^4 points has error $h^2/6 \approx 2\times10^{-9}$ for this integral. In one dimension, Monte Carlo is a terrible method!

6.2.5 Why Monte Carlo? The curse of dimensionality

So why bother with Monte Carlo at all? Consider an integral over the d-dimensional unit cube,

$$I = \int_{[0,1]^d} g(x)\,dx.$$

The quadrature rules of Chapter 4 extend to d dimensions by applying them in each coordinate direction in turn (a **tensor-product** rule). With m points in each direction, the composite trapezium rule uses a grid of $n = m^d$ points and, for smooth g, has error $O(h^2) = O(m^{-2})$. In terms of the total number of function evaluations, this is

$$\text{trapezium: } O(n^{-2/d}), \qquad \text{Simpson: } O(n^{-4/d}), \qquad \text{Monte Carlo: } O(n^{-1/2}).$$

The Monte Carlo rate **does not depend on the dimension** (although the constant σ does depend on g). It overtakes the trapezium rule when $d > 4$ and Simpson's rule when $d > 8$.

Worse still for grids, even the crudest grid with 2 points per direction needs 2^d evaluations. For $d = 100$, this is about 10^{30} evaluations, which would take more than 10^{13} years at a billion evaluations per second. This exponential growth of cost with dimension is called the **curse of dimensionality**.

Example 6.8: Monte Carlo versus the trapezium rule in higher dimensions

The integrand $g(x) = e^{-|x|^2}$ on $[0,1]^d$ has exact integral $I_d = \left(\int_0^1 e^{-s^2}\,ds\right)^d = \left(\frac{\sqrt{\pi}}{2}\,\mathrm{erf}(1)\right)^d$. The Monte Carlo estimate takes a few lines in any dimension:

```
d = 10;  n = 1e6;
gX = exp(-sum(rand(n, d).^2, 2));        % g at n uniform points in [0,1]^d
muHat = mean(gX);  SE = std(gX)/sqrt(n);
Iexact = (sqrt(pi)/2*erf(1))^d;
fprintf('%.6f +/- %.6f (exact %.6f)\n', muHat, 1.96*SE, Iexact)
```

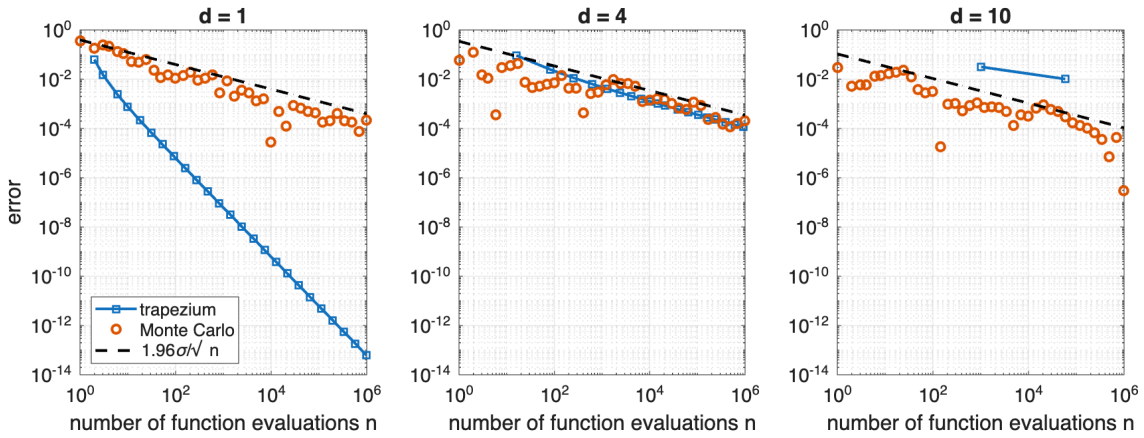


Figure 6.2: Error against the number of evaluations of g for the tensor-product trapezium rule and for a single Monte Carlo run, in dimensions $d = 1, 4, 10$. The dashed line is the 95% error bar $1.96\sigma/\sqrt{n}$.

In one dimension the trapezium rule wins by many orders of magnitude. In four dimensions the two methods are comparable. In ten dimensions, the trapezium rule with only 3 points per direction already needs $3^{10} = 59049$ evaluations and is accurate to just 10^{-2}, while a million Monte Carlo samples (fewer than a grid with 4 points per direction) give an error of order 10^{-4}, as predicted by the dashed line, together with an error bar.

Summary: Monte Carlo is a poor method in one dimension and often the only practical method in a hundred.

Where high-dimensional integrals arise

- **Statistical physics:** averages over all configurations of a system of N particles or spins, such as the Ising model, are integrals or sums in dimension proportional to N. These are computed with **Markov chain Monte Carlo** methods such as the Metropolis algorithm.
- **Finance:** the price of an option on a basket of many assets is an expectation over all of their future paths.
- **Bayesian statistics and machine learning:** posterior means are integrals over all unknown parameters.
- **Computer graphics:** in modern animated films, the colour of each pixel is an integral over all paths that light can take through the scene, computed by Monte Carlo *path tracing*. The grainy "noise" in a render with too few samples is exactly the $n^{-1/2}$ Monte Carlo error.

6.2.6 Variance reduction: importance sampling

The rate $n^{-1/2}$ is fixed, but the constant σ is not. The same quantity μ can be written as the expectation of many different random variables, some with much smaller variance than others. Methods that exploit this are called **variance reduction** techniques, and one of the most important is importance sampling.

Its main use is for **rare events**. Suppose we want $p = \mathbb{P}[Z > 4] \approx 3.17 \times 10^{-5}$ for $Z \sim \mathcal{N}(0,1)$. The plain Monte Carlo estimator averages $A(Z) = \mathbb{1}\{Z > 4\}$, which has $\sigma^2 = p(1-p)$, so the error relative to p is about

$$\frac{\sigma}{p\sqrt{n}} \approx \frac{1}{\sqrt{pn}}.$$

With $n = 10^4$ samples we expect fewer than one sample to exceed 4, and we need $n \approx 100/p \approx 3 \times 10^6$ for 10% relative accuracy. Almost every sample lands where $A = 0$ and is wasted.

The idea of importance sampling is to draw samples from a different density g that puts more of them where they matter, and then correct for this with a weight. If $g(x) > 0$ wherever $A(x)f(x) \neq 0$, then

$$\mu = \int A(x) f(x)\, dx = \int A(x) \frac{f(x)}{g(x)}\, g(x)\, dx = \mathbb{E}\big[A(Y)\, w(Y)\big], \qquad Y \sim g,$$

where $w = f/g$ is called the **likelihood ratio** or **weight**.

Theorem 6.4: Importance sampling

Let $Y_1, \dots, Y_n$ be independent with density g, and let

$$\hat{\mu}_n^{\mathrm{IS}} = \frac{1}{n} \sum_{i=1}^{n} A(Y_i)\, w(Y_i), \qquad w = \frac{f}{g}.$$

Then $\hat{\mu}_n^{\mathrm{IS}}$ is an unbiased estimator of μ, with variance σ_g^2/n, where

$$\sigma_g^2 = \int \frac{A(x)^2 f(x)^2}{g(x)}\, dx - \mu^2.$$

If $A \geq 0$ and $\mu > 0$, the choice $g^* = Af/\mu$ gives $\sigma_{g^*}^2 = 0$.

Proof. Unbiasedness follows from the identity above. For the variance,

$$\mathrm{Var}[A(Y)w(Y)] = \mathbb{E}[A(Y)^2 w(Y)^2] - \mu^2 = \int A^2 \frac{f^2}{g^2}\, g\, dx - \mu^2 = \int \frac{A^2 f^2}{g}\, dx - \mu^2,$$

and the variance of the average of n independent copies is this divided by n. With $g = g^*$, we have $A^2 f^2 / g^* = \mu A f$ wherever $A > 0$ (and both sides vanish where $A = 0$), so $\sigma_{g^*}^2 = \mu \int Af\, dx - \mu^2 = 0$. □

The optimal density g^* cannot be used in practice, since it requires knowing μ. But it tells us what to aim for: g should be roughly proportional to $|A|f$, so that samples are concentrated where the integrand is large. A poor choice of g, for example one with much lighter tails than Af, can instead make the variance much *larger*, or even infinite.

Example 6.9: Importance sampling for a rare event

To estimate $p = \mathbb{P}[Z > 4]$, we shift the samples into the rare region by taking g to be the $\mathcal{N}(4, 1)$ density. The weight is

$$w(y) = \frac{f(y)}{g(y)} = \frac{e^{-y^2/2}}{e^{-(y-4)^2/2}} = e^{8-4y}.$$

```
rng(2)
n = 1e4;
Z = randn(n, 1);  A = (Z > 4);                % plain Monte Carlo
Y = 4 + randn(n, 1);                          % samples from g = N(4,1)
AW = (Y > 4).*exp(8 - 4*Y);                   % A(Y) w(Y)
fprintf('plain: %.2e +/- %.1e\n', mean(A), 1.96*std(A)/sqrt(n))
fprintf('IS:    %.3e +/- %.1e\n', mean(AW), 1.96*std(AW)/sqrt(n))
```

Plain Monte Carlo gives $(1.0 \pm 2.0) \times 10^{-4}$, based on the single sample that exceeded 4, which is useless. Importance sampling gives $(3.23 \pm 0.13) \times 10^{-5}$, consistent with the exact value 3.167×10^{-5}. For this choice of g, the variance is about 7000 times smaller than for plain Monte Carlo, so we would need about 7000 times as many plain samples for the same accuracy.

Exercise 6.2

Estimate $\mathbb{P}[Z > 6]$ for $Z \sim \mathcal{N}(0,1)$ by importance sampling with g the $\mathcal{N}(6,1)$ density, and compare with the exact value `0.5*erfc(6/sqrt(2))`. Roughly how many plain Monte Carlo samples would be needed for the same accuracy?

i Other ways to beat plain Monte Carlo

Other variance reduction techniques include **control variates**, which subtract a correlated quantity with known mean, and **antithetic variables**, which pair each sample U with $1-U$. **Quasi-Monte Carlo** methods go further, replacing random points by carefully constructed low-discrepancy sequences (e.g. `sobolset` in MATLAB), and achieve errors close to $O(n^{-1})$ for smooth integrands.

6.2.7 Application: random walkers solve the heat equation

We can now connect this section to the heat equation from earlier in the chapter. Recall the method-of-lines system with Neumann boundary conditions, and apply forward Euler with the *largest stable timestep*, $\Delta t = (\Delta x)^2/(2\alpha)$. Writing $u_j^k \approx u(x_j, t_k)$, the scheme becomes

$$u_j^{k+1} = u_j^k + \frac{\alpha \Delta t}{(\Delta x)^2}\left(u_{j-1}^k - 2u_j^k + u_{j+1}^k\right) = \frac{1}{2}u_{j-1}^k + \frac{1}{2}u_{j+1}^k, \qquad j = 1, \dots, N,$$

with ghost nodes $u_0^k = u_1^k$ and $u_{N+1}^k = u_N^k$. Each new value is simply the *average of the two neighbouring values*.

This has a probabilistic interpretation. Consider a **random walker** on the nodes $1, \dots, N$ that, at each timestep, moves one node left or right with probability $\frac{1}{2}$ each. If a step would take the walker onto a ghost node, it stays where it is instead (it is **reflected**). Let J_k be the walker's node after k steps, and define

$$v_j^k = \mathbb{E}\big[\, u(x_{J_k}, 0) \mid J_0 = j \,\big],$$

the expected initial temperature at the walker's position after k steps, starting from node j.

Theorem 6.5

For all j and k, $v_j^k = u_j^k$: the forward Euler solution is the expected value of the initial data at the end of a random walk.

Proof. For $k = 0$ the walker has not moved, so $v_j^0 = u(x_j, 0) = u_j^0$. For $k \geq 0$, condition on the walker's first step. With probability $\frac{1}{2}$ it moves to node $j-1$, after which it performs a k-step walk from there; similarly for $j+1$. Hence

$$v_j^{k+1} = \tfrac{1}{2} v_{j-1}^k + \tfrac{1}{2} v_{j+1}^k,$$

where a step onto a ghost node is replaced by staying at the boundary node, i.e. $v_0^k = v_1^k$ and $v_{N+1}^k = v_N^k$. This is exactly the recursion satisfied by u_j^k, with the same initial values, so $v_j^k = u_j^k$ by induction on k. □

So we can estimate the solution at a single point by Monte Carlo: release M independent walkers from node j, run each for $K = t/\Delta t$ steps, and average the initial data at their final positions. As $\Delta x \to 0$, the random walk converges to Brownian motion, and we obtain $u(x,t) = \mathbb{E}[u(X_t, 0)]$, where X_t is a (reflected) Brownian motion started at x. This is a special case of the **Feynman–Kac formula**, which links a large class of PDEs to expectations over random paths.

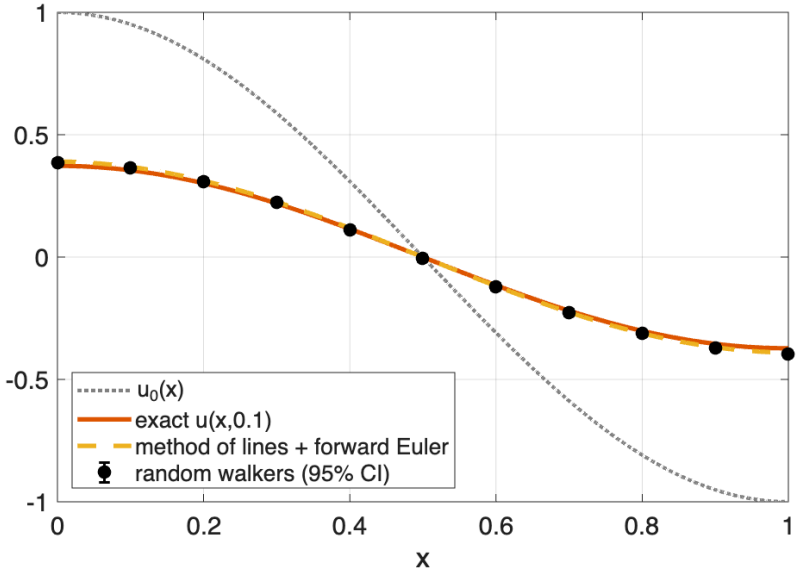
Example 6.10: MATLAB demonstration: random walkers for the heat equation

We take $L = \alpha = 1$, $u(x,0) = \cos(\pi x)$ and $t = 0.1$; the exact solution is $u(x,t) = e^{-\pi^2 t}\cos(\pi x)$. The following function estimates $u(x_{j_0}, t)$ using M walkers on a grid of N nodes:

```
function [uHat, SE] = walkers(j0, M, N, K, u0)
x = linspace(0, 1, N);
j = j0*ones(M, 1);                          % every walker starts at node j0
for k = 1:K
    j = j + 2*(rand(M, 1) < 0.5) - 1;       % step left or right
    j = min(max(j, 1), N);                  % a step off the grid stays put
end
vals = u0(x(j));                            % initial data at final positions
uHat = mean(vals);
SE = std(vals)/sqrt(M);
end
```

With $N = 51$ we have $\Delta x = 0.02$, $\Delta t = 2 \times 10^{-4}$ and $K = 500$ steps. Running `walkers(j0, 1e4, 51, 500, @(x) cos(pi*x))` for every fifth node gives the estimates below. They agree with the method-of-lines solution to within their error bars (as the theorem predicts),

and that solution in turn approximates the exact solution.



Why would we ever do this, when the method of lines gives the whole solution at once? A grid method must compute the solution everywhere, on a grid of m^d points in d dimensions. A walker estimates the solution at just the points we care about, and its cost per step barely depends on dimension: in d dimensions it simply chooses among $2d$ neighbours. This is the curse of dimensionality again, and it is why random-walk methods are used for high-dimensional PDEs such as those arising in finance and quantum mechanics. They also handle complicated geometries easily, since the walker only needs to know whether it has hit a boundary. Other boundary conditions and equations correspond to other walker rules: for example, Dirichlet boundary conditions correspond to walkers that are *absorbed* when they reach the boundary, and the expected time for a walker to hit a target region (such as a foraging animal searching for food) satisfies a Poisson equation.

Exercise 6.3

Extend `walkers` to the heat equation on the unit square $[0,1]^2$ with Neumann boundary conditions, where each walker moves to one of its four neighbours with equal probability. Take $u(x,y,0) = \cos(\pi x)\cos(\pi y)$, for which $u = e^{-2\pi^2\alpha t}\cos(\pi x)\cos(\pi y)$. What timestep does forward Euler need to make the scheme an average over neighbours? How does the standard error change as you increase M?

Knowledge checklist

Key topics:

1. The method of lines for the heat equation: finite-difference discretisation in space, ghost nodes for Neumann boundary conditions, and the stability of forward and backward Euler.
2. Pseudorandom number generators: seeds, periods, linear congruential generators and their lattice structure.
3. The chi-squared test for uniformity, and why tests must probe independence as well as the distribution of single values.
4. Sampling from other distributions by the inverse transform method.
5. The Monte Carlo estimator: unbiasedness, the law of large numbers, the $n^{-1/2}$ error from the central limit theorem, standard errors and confidence intervals.
6. The curse of dimensionality: grid-based quadrature errors of $O(n^{-2/d})$ compared with the dimension-independent $O(n^{-1/2})$ for Monte Carlo.
7. Variance reduction by importance sampling, and the optimal sampling density.
8. The random-walk representation of the forward Euler scheme for the heat equation.

Key skills:

- Implementing the method of lines with explicit and implicit timestepping, and choosing a stable timestep.
- Implementing a linear congruential generator and testing its output with the chi-squared test.
- Generating samples from a given distribution by the inverse transform method.
- Writing integrals as expectations, and computing Monte Carlo estimates with confidence intervals at a given level.
- Deciding when Monte Carlo is preferable to quadrature.
- Choosing an importance sampling density and computing the corresponding weights.
- Simulating random walks and using them to estimate expectations.

Projects Showcase

The largest component of the assessment of this module is the **project**. There are approximately eight different project options to choose from across different areas of mathematics (e.g., pure, applied, probability, mathematical physics, etc.); each project has a distinct member of the Maths Department as supervisor. Each project consists of two parts: a **guided component** that is completed as part of a small group and an **extension component** that is open-ended and completed as an individual.

This page showcases some of the outstanding student projects (**individual extension component**) that were awarded prizes for their excellence. All prize-winning project videos are available on the course YouTube channel.

We are very grateful to MathWorks for generously providing the prizes!

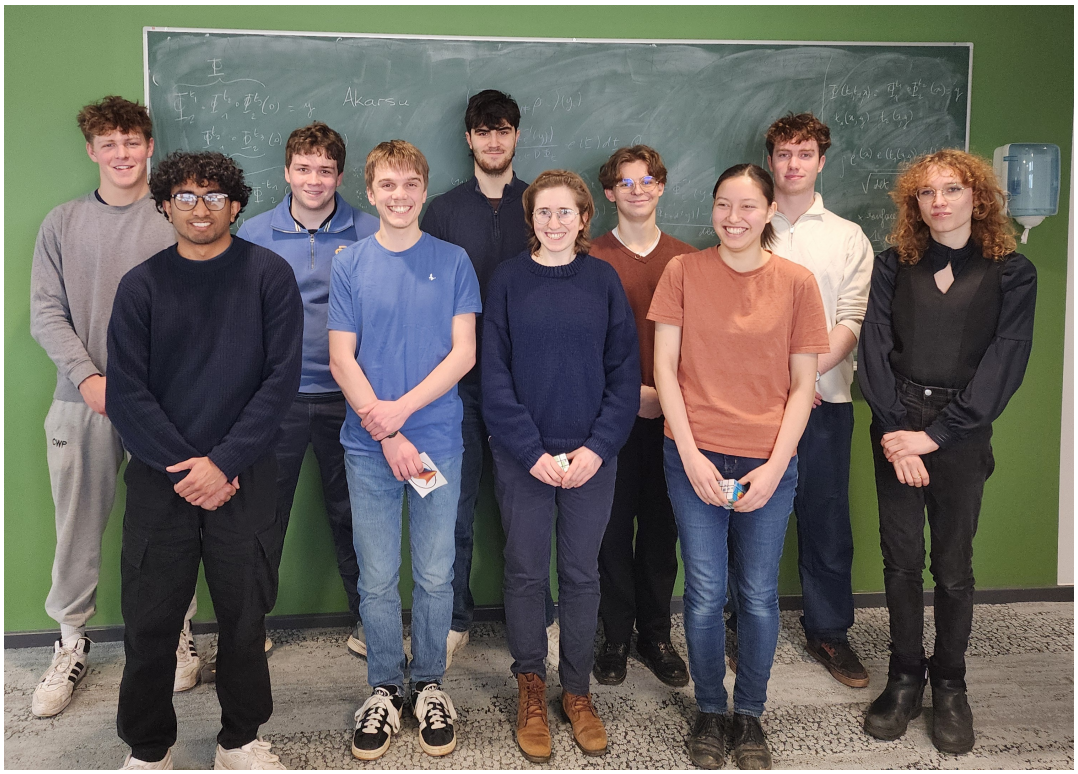


Figure 6.3: Some of our prize-winning students with their MathWorks prizes.

Outstanding Projects 2025/2026 (selection)

Jack Guilfoyle: "Dependence of Initial Speed on Kink-Antikink Collisions"

https://youtu.be/KC3DfIlTm70?si=L4ZokO1Y-G2rH7Wa

Talia Tobias: "Paper Aeroplanes and Dynamical Systems"

https://youtu.be/xJmd4d4s780?si=4k9b0GviL3cY7o5t

Abigail Weersing: "Power Output of a Wind Farm"

https://youtu.be/SJZ00sMg6bM?si=QrhE99O-LTkGMJra

Thomas Newbury: "Finite Size Scaling in the 2D Ising Model"

https://youtu.be/AubzcrxZSYY?si=yFiGkbPtdtFpZajU

Sam Todd: "Gradient and Invasion Percolation"

https://youtu.be/VU289wUaajE?si=ONpkdOj7Cas4ptbF

Ashwin Iyer: "The Foucault Pendulum"

https://youtu.be/mpOjZGvJYck?si=OVHz_RibkYmGUj7f

Joshua Thompson: "Random Braid Words and Link Properties"

https://youtu.be/ZB9UXGwqjXA?si=OQslzJzPaEtFH0-x

Cameron Connor: "Structural Credit Risk Models"

https://youtu.be/JUfVuVp3QfU?si=I0zXNizm_1RF6x_P